Code Cache Management in Dynamic Optimization Systems

A thesis presented

by

Kim Hazelwood Cettei

to

The Division of Engineering and Applied Sciences

in partial fulfillment of the requirements for the degree of

Doctor of Philosophy

in the subject of

Computer Science

Harvard University

Cambridge, Massachusetts

May 2004

©2004 - Kim Hazelwood Cettei

All rights reserved.

Thesis advisor
Michael D. Smith

Author
Kim Hazelwood Cettei

Code Cache Management in Dynamic Optimization Systems

Abstract

Dynamic optimization systems store optimized or translated code in software-managed code caches in order to maximize reuse of transformed code. Code caches store superblocks that are not fixed in size, may contain links to other superblocks, and carry a high replacement overhead. These additional constraints reduce the effectiveness of conventional cache management policies. This dissertation investigates the code cache management problem in dynamic optimization systems and presents three major advances that cover the design space of cache management decisions.

Through code cache simulations, we show that a `FIFO` replacement policy outperforms other traditional policies, as it enables contiguous cache evictions, allows for a simple circular buffer implementation, and results in comparable cache miss rates to `LRU`. Furthermore, a *pseudo-circular FIFO* algorithm is presented, which handles the problem of un-deletable cache blocks.

An investigation of cache eviction granularities also reveals that evicting more than the minimum number of superblocks from the code cache at a time results in less run-time overhead than other alternatives. In high cache pressure situations, the choice of medium-granularity evictions translates into a significant reduction in overall execution time versus both fine and coarse granularities, as the fixed costs of invoking the eviction mechanism can be amortized across multiple cache insertions.

A study of the ideal lifetimes of superblocks in a code cache motivates the design of a global cache management algorithm that leverages generational heuristics to provide for separate code caches for short- and long-lived superblocks. Using this algorithm, short-lived superblocks can easily be removed from a code cache without introducing fragmentation and without suffering the performance penalties associated with evicting long-lived superblocks. The generational code cache algorithm results in an average miss rate reduction of 18% over a unified cache, which translates into 19% fewer instructions spent in the dynamic optimizer.

Finally, an implementation of generational code caches in the DynamoRIO framework reveals important design decisions that affect the performance of code cache management policies, such as support for relocatable code, lightweight link adjustment, and lightweight profiling insertion and removal. As dynamic optimization systems become more prevalent, effective code cache management policies will be essential to reliable, scalable performance. This dissertation presents the first steps in the direction of understanding the behavior of code caches and designing effective code cache replacement policies for modern applications, and opens the doors for follow-up research on cache insertion policies, phase detection, and persistent caches.

Contents

Title Page i
Abstract iii
Table of Contents iv
List of Figures vii
List of Tables ix
Citations to Previous Publications x
Acknowledgments xi
Dedication xii

1 Introduction **1**
1.1 The Role of a Software-Based Code Cache 2
1.2 Current Issues in Code Caching 3
1.3 Solving the Code Cache Management Problem 3

2 Software-Based Code Caches **6**
2.1 Overview of Code Caches 6
2.1.1 Basic Blocks and Superblocks 8
2.1.2 Superblock Creation Heuristics 9
2.1.3 Multiple Code Caches 10
2.2 Motivating Bounded Code Caches 10
2.2.1 Maximum Size of Unbounded Caches 11
2.2.2 Superblock Generation Frequency 11
2.2.3 Moving Forward: Cache Size Trends 13
2.3 Challenges for Code Cache Management 13
2.3.1 Caching Varying-Sized Superblocks 16
2.3.2 Superblock Chaining 18
2.3.3 Superblock Regeneration Overhead 19
2.3.4 Unmapped Memory 19
2.4 Summary 20

3 Traditional Replacement Policies **21**
3.1 Optimal Replacement 22
3.2 Full Flush 22

3.3 Least-Recently Used . . . 23
3.4 Least-Frequently Used . . . 26
3.5 First-In First-Out . . . 26
3.6 Best-Fit Element . . . 27
3.7 Random Element . . . 27
3.8 Summary . . . 28

4 Local Code Cache Management 29
4.1 Pseudo-Circular FIFO Replacement . . . 30
4.2 Code Cache Eviction Granularity . . . 32
4.2.1 Cache Miss Rates by Granularity . . . 33
4.2.2 Code Cache Evictions by Granularity . . . 34
4.2.3 Cache Miss and Eviction Overhead by Granularity . . . 36
4.2.4 Analysis of Superblock Chaining . . . 37
4.2.5 Characteristics of Superblock Links . . . 38
4.2.6 Interconnectivity of Superblock Links . . . 40
4.2.7 Resulting Impact on Overhead . . . 41
4.2.8 Granularity Discussion . . . 42
4.3 Summary . . . 43

5 Generational Code Caches 44
5.1 Superblock Lifetimes . . . 45
5.2 Parallels in Garbage Collection . . . 47
5.3 The Probation Cache . . . 47
5.4 Code Relocation . . . 49
5.5 Evaluation . . . 49
5.5.1 Code Cache Miss Rates . . . 51
5.5.2 Resulting Performance . . . 52
5.6 Lifetime versus Access Frequency . . . 53
5.7 Discussion . . . 54

6 Implementation Details and Challenges 55
6.1 Challenges . . . 55
6.1.1 Support for Relocatable Code . . . 55
6.1.2 Efficient Link Repair . . . 56
6.1.3 Hash Tables . . . 57
6.1.4 Profiling Insertion and Removal . . . 58
6.2 Performance Results . . . 59
6.3 Initial Cache Setup Overhead . . . 61
6.4 Superblock Promotion Overhead . . . 62
6.4.1 Reserving Code Cache Space . . . 62
6.4.2 Memory Copy Overhead . . . 63

6.4.3 Link Repair Overhead 63
6.4.4 Overhead Summary 64
6.5 Validating Simulated Miss Rates 64
6.6 Discussion 65

7 Related Work – Systems Using Code Caching **66**
7.1 Transparent Optimization 66
7.1.1 Performance Optimization Focus 66
7.1.2 Security, Sandboxing, and Introspection 67
7.2 Dynamic Binary Translation 68
7.3 High-Level Language Virtual Machines 69
7.4 Hardware Caches 69
7.5 Related Areas 70

8 Conclusions **71**
8.1 Broader Impact 72
8.2 Future Work 73
8.2.1 Code Cache Insertion Policies 73
8.2.2 Hardware and OS Support for Code Caches 73
8.2.3 Persistent Storage of Code Cache Contents 74
8.2.4 Enabling Other Technologies 74

A Persistent Caching **75**
A.1 Inter-Execution Optimization Persistence 76
A.2 Inter-Application Persistence 79
A.3 Discussion 81

B Experimental Approach **82**
B.1 SPEC 2000 Benchmarks 83
B.2 Interactive Benchmarks 83
B.3 Using the PAPI Interface 84
B.4 Raw Data 85

Bibliography **91**

List of Figures

1.1 High-level view of a dynamic optimization system. 2

2.1 Control flow of dynamic optimization systems 7
2.2 A superblock example 9
2.3 Maximum cache size required to avoid cache management. 12
2.4 Amount of code generated per second. 14
2.5 Code expansion with an unlimited code cache. 15
2.6 Static size distribution of superblocks. 16
2.7 Median superblock size 17
2.8 Code cache with superblock links. 18
2.9 Evictions due to unmapped memory. 20

3.1 Optimal code cache miss rates 23
3.2 Traditional algorithm miss rates under low cache pressure 25
3.3 Traditional algorithm miss rates under high cache pressure 25
3.4 Traditional algorithm miss rates under varying cache pressure 26

4.1 An algorithm for pseudo-circular FIFO code cache eviction. 31
4.2 Conceptual view of medium-grained cache eviction units. 32
4.3 Distinction between intra-unit and inter-unit links. 32
4.4 Code cache miss rates at varying granularities 34
4.5 Miss rates at varying granularities as cache pressure increases. 35
4.6 Relative number of evictions compared to finest-grained FIFO eviction. 36
4.7 Relative overhead of code cache eviction granularities (including miss rate and eviction overheads) 37
4.8 Average number of outbound links originating from each superblock. 38
4.9 Percentage of links that target superblocks in different cache units. . 40
4.10 Relative overhead of eviction granularities (including cache miss penalties, eviction penalties, and superblock link maintenance) 41
4.11 Relative overhead (including link maintenance) as cache pressure increases. 42

5.1 Conceptual view of the generational code cache architecture. 45

5.2 Superblock lifetimes. 46
5.3 Generational code cache architecture using a probation cache. 48
5.4 The generational code cache algorithm. 48
5.5 Cache miss rate reduction: generational vs. unified. 50
5.6 Generational code cache misses eliminated. 51
5.7 Overhead of generational code caches. 53

6.1 A superblock with exit stubs 56
6.2 Superblock promotion and insertion of profiling counters 59
6.3 Runtime performance of generational code caches 60
6.4 Breakdown of cache accesses for GCC 61
6.5 Relative generational code cache performance 62
6.6 Code block promotion overhead breakdown 63
6.7 Validating simulated miss rates 65

A.1 Inter-execution persistence with changing inputs 78
A.2 Inter-application persistence 80
A.3 Dynamic instruction overlap 81

B.1 Trace-driven simulation framework used in Chapters 3–5. 82
B.2 The overhead of code cache evictions 85
B.3 Interconnectivity of superblocks within a code cache. 88
B.4 Accesses over time for the five oldest superblocks per benchmark. . . 89
B.5 Accesses over time for the five hottest superblocks per benchmark. . . 90

List of Tables

3.1 Number of cache flushes during execution 24
3.2 Evictions per cache insertion 24
3.3 Summary of traditional replacement algorithms 28

4.1 Slowdown resulting from disabling superblock chaining 39

5.1 Overheads used in the evaluation of generational code caches. 52

6.1 Average overhead of run-time generational tasks 64

B.1 SPEC 2000 benchmarks used 83
B.2 Interactive windows benchmarks used 84
B.3 Raw Data: Traditional miss rates at low cache pressure 86
B.4 Raw Data: Traditional miss rates at high cache pressure 86
B.5 Raw Data: Results of eviction granularities 87

Citations to Previous Publications

Large portions of this dissertation appeared in the following publications:

Kim Hazelwood and Michael D. Smith. "Generational Cache Management of Code Traces in Dynamic Optimization Systems," 36th Annual International Symposium on Microarchitecture, December 2003, San Diego, California, pp. 169–179.

Kim Hazelwood and James E. Smith. "Exploring Code Cache Eviction Granularities in Dynamic Optimization Systems," 2nd Annual International Symposium on Code Generation and Optimization, March 2004, Palo Alto, California, pp. 89–99.

Kim Hazelwood and Michael D. Smith. "Characterizing Inter-Execution and Inter-Application Optimization Persistence," First Workshop on Exploring the Trace Space for Dynamic Optimization Techniques, June 2003, San Francisco, California, pp. 51–58.

Kim Hazelwood and Michael D. Smith. "Code Cache Management Schemes for Dynamic Optimizers," Sixth Workshop on the Interaction between Compilers and Computer Architecture, February 2002, Boston, Massachusetts, pp. 102–110.

Acknowledgments

Looking back over ten years of higher education, there are countless people that should be acknowledged for their contributions to my educational path. I've always thought of myself as someone who is tempted by the path of least resistance, therefore it was ultimately the impact of others that enabled me to progress this far in my education.

I should start by thanking my family. Growing up with eight siblings is a truly unique experience that undoubtedly resulted in the drive and endurance to complete a Ph.D. I learned from a very young age the value of collaboration and compromise, techniques for getting along with a variety of personality types, the importance of standing out in a crowd, and most importantly, persistence.

My husband Matt has been there for me throughout my entire graduate school experience. By providing a listening ear to all of my daily gripes about officemates, coursework, and bureaucracy, Matt allowed me to spend my days focusing on work and (usually) pleasantly interacting with colleagues. Matt gave me a reason to work normal business hours and to be home in time for dinner and reality television.

There were several professors along the way who stand out in my memory. Walt Ligon was the first to mention graduate school to me during my sophomore year at Clemson. Walt went on to be my honors thesis advisor and during the course of that project, he introduced me to the world of computing research, which I quickly learned was much more fun than coursework. Tom Conte, my master's thesis advisor, subsequently made a huge impact on my research career. Tom introduced me to the key players, made sure I was always working on a conference paper, and ultimately convinced me that I should continue for a Ph.D.

None of my recent research would've been possible without the support of my Ph.D. advisor, Mike Smith. I am truly grateful to Mike for being what many called "the most mellow advisor around" and for never doubting or criticizing me (out loud, at least). I once heard Mike's own advisor Monica Lam say, "Do what you love, and be nice." I think Mike is a living example of that comment. He is an extremely sincere, grounded individual, and an excellent role model.

I would like to thank the rest of my dissertation committee: Margo Seltzer, for showing me that it's possible to balance family and a great career, and David Brooks, for having the time and energy for numerous brainstorming sessions and for getting my name out when it came time to search for a postdoctoral research position.

Finally, I can't thank our trusty staff member, Glenn Holloway, enough for endless hours of proofreading before deadlines, operating system installations, and technical support for anything and everything.

Dedicated to my mother,
for working so hard over the years,
and always maintaining a smile.

Chapter 1

Introduction

In order to keep pace with Moore's Law[1] [79], hardware designers make dramatic low-level changes to every new microprocessor released to consumers. Yet software packages are generally limited to a single version designed to execute on any processor featuring the same instruction-set architecture (ISA). These trends have resulted in two key inefficiencies for hardware-software interaction. From a software perspective, compiling and scheduling software packages for all processors within a single ISA inherently leads to conservative assumptions about individual processor features and latencies, which results in inefficient execution on a given processor. And from a hardware perspective, microprocessor designers are forced to conform to the most widespread ISA, regardless of the inefficiencies of doing so, or run the risk of market rejection due to software incompatibility.

One solution to these inefficiencies has been the introduction of dynamic optimization systems—including dynamic optimizers [8, 15, 34, 87], dynamic binary translators [2, 23, 32, 38, 55], and just-in-time compilers [4, 13]. These systems create a modified version of a program's code image while it executes. Dynamic optimizers enable performance improvements by leveraging information that is not available at static compile time, including run-time constants, user preferences and access patterns [95], and traits of the particular processor architecture or memory system. This additional information is used to apply link-time [80, 91], run-time [8, 46, 77], and/or temporal optimizations [89, 90] to programs, often without the need for source code. Dynamic binary translators extend this idea by converting the code image from one ISA to another (and applying optimizations) before each instruction is executed. This technique allows for the greatest flexibility during the hardware-design phase, but still provides code compatibility for existing software packages. Because the three systems above—dynamic optimizers, dynamic binary translators, and just-in-time compilers—all must and do perform optimizations to executing applications to improve the resulting performance, they are often collectively referred to as *dynamic*

[1] In 1965, Gordon Moore observed that the $transistors/inch^2$ on integrated circuits had doubled every year. Since then, Moore's Law has been self-fulfilling, although it slowed to every 18 months.

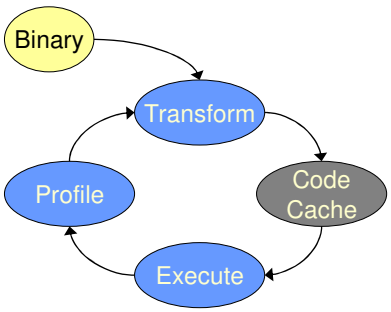


Figure 1.1: High-level view of a dynamic optimization system.

optimization systems.

The potential of dynamic optimization systems as a standard layer in hardware-software co-design cannot be realized until the full system can efficiently execute large applications. This dissertation tackles one of the major issues that is currently impeding that goal.

1.1 The Role of a Software-Based Code Cache

Dynamic optimization systems generally perform four major tasks. First, they analyze the program's instruction stream to determine the flow of execution. Second, they perform analysis, translation and/or optimization on the frequently executed code sequences. Third, they cache the transformed code in a *software-based code cache* to enable reuse and to amortize the costs of code transformation. Finally, they execute code directly from the code cache for the remainder of program execution or until the code is evicted from the code cache to make room for newly transformed code. Figure 1.1 provides a high-level view of each of the four tasks within a dynamic optimization system.

The steady-state performance of dynamic optimization systems is improved by increased instruction locality and code optimization. Offsetting this steady-state performance increase are the *overhead costs* of observing run-time behavior, performing transformations, updating program code, and managing the code cache.

One of the major impediments to the widespread acceptance of dynamic optimization systems as a standard layer in hardware-software co-design has been the performance degradation of many client applications executing under the control of a dynamic optimization system. The performance degradation has resulted from scalability issues in the implementation of these systems. When executing large applications, much of the run-time budget ends up being spent performing overhead tasks, such as code transformation and cache management, and less time is spent

actually executing the application. For very large applications, overhead costs can potentially dominate, resulting in temporarily unresponsive software. As software complexity increases, as it undoubtedly will over time, the problem will become more aggravated. Therefore, optimizing the various components that make up a dynamic optimization system is an important problem.

1.2 Current Issues in Code Caching

The code cache is one such component that is particularly important because it also impacts several other overhead costs within a dynamic optimization system, such as the rate of code regeneration and reoptimization. An effective code cache management policy should incur low overhead while making eviction decisions. It should also maintain a good working set of optimized code in the code cache by evicting code that is unlikely to be used in the future. This will help to reduce the number of times that code must be regenerated and reoptimized in response to non-compulsory code cache misses and will improve performance by allowing execution to remain in the cached code.

In the past, code cache designers have used simple management techniques, such as flushing the cache when a significant change in the instruction working set is detected [7] or creating an unbounded code cache [15]. The designers evaluated each of these cache management schemes using the SPEC 2000 benchmark suite. As we will see in Chapter 2, the problem and importance of code cache management changes dramatically as we move away from the SPEC 2000 benchmarks and look at today's large interactive Windows applications, such as Microsoft Word and Adobe Acrobat. Not only do the number of code blocks generated at run time increase dramatically, but the rate at which they are generated also increases dramatically. While the 500% code expansion that generally occurs in an unbounded code cache can be tolerated for the SPEC 2000 benchmarks whose footprints are smaller than five megabytes, this level of code expansion is unacceptable for applications with code footprints on the order of 10's to 100's of megabytes. These observations both motivate and justify the design of sophisticated cache management techniques.

1.3 Solving the Code Cache Management Problem

This dissertation focuses on the problem of code cache management in the context of dynamic optimization systems and introduces several solutions to the problem. Chapter 2 provides an overview of software-based code caches, discusses the challenges that are specific to code caches, and motivates the need for bounded caches.

After introducing and evaluating the miss rate and fragmentation of several traditional code cache eviction policies in Chapter 3, it will become clear that a First-In First-Out (FIFO) replacement policy is an effective choice. It rivals a Least-Recently

Used (LRU) policy for the lowest miss rate, but also enables a simple circular buffer implementation that requires no profiling or defragmentation.

After choosing FIFO as the underlying replacement policy, there are many opportunities for extending this policy. These extensions can be classified into two main categories – *local policies* and *global policies*. Local policies define the replacement strategy and details for a single code cache. A cache hierarchy may contain multiple code caches, however, and a global policy is necessary for defining the movement strategy between the caches.

Chapter 4 introduces and explores two local extensions to a FIFO replacement policy. The first extension is a pseudo-FIFO eviction policy for effectively handling undeletable code in a code cache. A second extension leverages the fact that a FIFO replacement policy can range from very fine grained (replace a single block of code at a time) to very coarse grained (flush the entire cache when it fills). Thus far, implementations have been designed with either miss rates or implementation complexity in mind, and have tended to use one of the extreme granularities. By exploring the wide range of medium-grained replacement policies, we will be able to understand the cost factors that come into play when choosing an eviction granularity. The analysis reveals that the best eviction policy is one that balances code cache miss rates, eviction overheads, and design complexity. In the case of the DynamoRIO dynamic optimization infrastructure, the trade-off curve suggests that partitioning the cache into several distinct *units*—where each unit contains multiple superblocks that are flushed together as a group to make room for new code—results in an effective balance of miss penalties and the opposing costs of invoking the eviction mechanism and removing dangling references to the evicted code.

With an effective local policy in place, a global policy involving multiple code caches can be designed and investigated. In Chapter 5, we will see that a global policy of maintaining different code caches for code blocks that exhibit different *lifetimes* is an effective solution. The analysis of the ideal lifetime of cached code shows that the majority falls into one of two categories: (1) a lifetime of less than 20% of the program execution, and (2) a lifetime of more than 80% of the program execution. This observation motivates an algorithm for employing *generational code caches* that allows the system to easily evict short-lived code from the code cache without introducing unnecessary fragmentation or suffering the performance penalties associated with evicting and regenerating long-lived code. After an analytical evaluation using simulation and overhead analysis, we will find that generational code caches provide notable overhead reductions.

Chapter 6 reveals several important design considerations derived from a direct implementation of generational code caches within DynamoRIO. Support for relocatable code, lightweight link adjustment, and lightweight profiling insertion and removal are introduced as valuable techniques for approaching the performance potential of generational code cache management.

We review related work in Chapter 7 by discussing several existing dynamic op-

timization systems and their solutions to code caching and cache management. We will then discuss broader impacts and future work that can extend the work of this dissertation in Chapter 8.

Finally, Appendix A introduces some follow-up work that takes generational code caches one step further. While the long-lived superblocks reside in a generational code cache until the application ends, *persistent code caches* allow superblocks to live beyond the end of a single application run. Appendix A investigates this potential for *inter-execution persistence* by studying the amount of code that is identically optimized during subsequent executions of the same program and therefore should be stored in secondary storage between executions. Appendix B provides a supplement to the experimental setup descriptions used throughout the dissertation.

The novel contributions of this dissertation include:

1. A description of the problem of code cache management in dynamic optimization systems and the issues that complicate the problem domain.
2. A characterization of the different code caching needs of SPEC 2000 and large interactive applications.
3. An evaluation of the miss rates and fragmentation of traditional code cache replacement policies using both SPEC 2000 and interactive applications.
4. An exploration of medium-grained code cache eviction policies for balancing overhead, code space, and cache miss rates.
5. A proposal for and evaluation of generational code caches for easily evicting short-lived code without incurring fragmentation.
6. Implementation details of sophisticated code cache management policies within the DynamoRIO framework.
7. A study of the potential for persistent code caches to store optimized code between program executions.

These contributions directly address the problem of code cache management, which is one of the main impediments to building robust dynamic optimization systems capable of executing large, interactive applications.

Chapter 2

Software-Based Code Caches

This chapter provides an overview of the role of code caches in dynamic optimization systems and introduces various challenges that are unique to code caches. We will review the basic elements that are stored in code caches – superblocks – including the design decisions that are made during their formation as well as their basic size and structure. We will discover why a cache management policy is essential for modern applications by investigating the size that the code cache will reach if we allow it to grow without bound. Finally, we will learn why code cache management is a unique challenge by reviewing the major issues that cache management algorithms must tolerate.

The overview will highlight various features that are specific to the DynamoRIO dynamic optimization system, as it was the research infrastructure used to gather all results presented throughout this dissertation.

2.1 Overview of Code Caches

Code caches are a vital element in any dynamic optimization system, as they enable reuse of transformed code and therefore help amortize the cost of code transformation over the entire run time of a program. Figure 2.1 illustrates the role of the code cache as well as code block lookup and execution in a typical dynamic optimization system.

As Figure 2.1 indicates, dynamic optimization systems begin by *interpreting* the executing application. Native interpretation involves discovering control flow, discovering instruction boundaries if necessary, and maintaining counters to recognize when code regions become frequently executed. In a dynamic binary translator, interpretation also includes a translation step, where each instruction is converted to an equivalent instruction (or set of instructions) in the native instruction-set architecture. In DynamoRIO [7, 8], where interpretation is native—from IA-32 to IA-32—the interpretation process is optimized, and we'll talk more about this optimization in

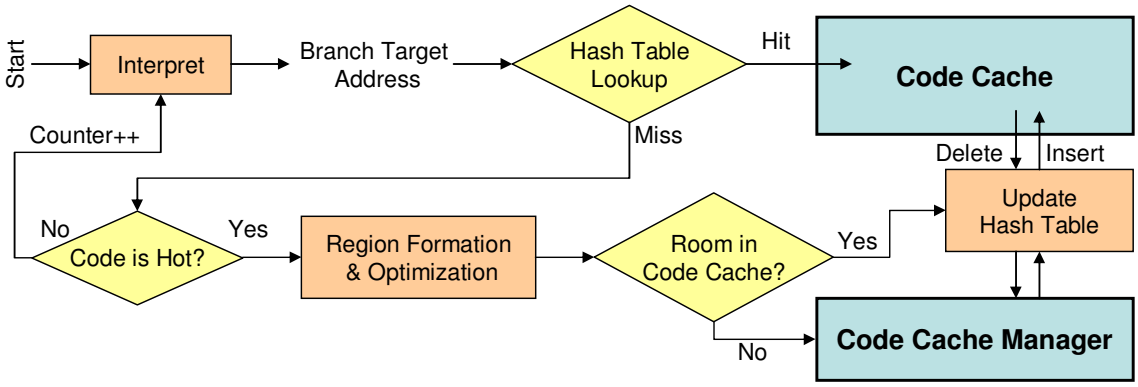


Figure 2.1: Control flow of dynamic optimization systems including a code cache and cache management system.

Section 2.1.3.

When a taken branch is encountered during interpretation, dynamic optimization systems perform a lookup to determine if the branch target is already present in the software-based code cache. A hash table—which maps original program addresses to code cache addresses containing the transformed code blocks—is used to perform the code cache lookup. A hit in the hash table results in an immediate jump to a previously transformed block in the code cache. A table miss results in one of two outcomes depending on whether the next instruction has been frequently executed in the past. (Heuristics for determining whether an address is considered "frequently executed" will be discussed in Section 2.1.2.) An infrequently-accessed (cold) address results in a counter increment and a return to interpretation mode. A frequently-accessed (hot) address results in transformation of the target code block, insertion of the resulting transformation into the code cache, and finally a jump into the code cache to execute the new code.

Each exit from a block in the code cache is appended with a series of instructions—also known as an *exit stub*—that return control to the dynamic optimization system, passing information about the next instruction to execute. The dynamic optimization system then determines whether to return control to the interpretation engine, or whether the next set of instructions is already present in the code cache and execution should resume at the start of that cached target block.

Eventually, the system will link together blocks of cached code that execute in succession to improve performance. This process—called *chaining*—entails patching the exit-stub branch with a branch to the target block in the cache. Chaining can be done *lazily*—links are formed only when execution flows from one block to another—or *proactively*—all possible links are formed when a block is first inserted into the code cache. Chaining will be discussed in more detail in Section 2.3.2.

Code cache management comes into play because it won't always be the case that a new code block can be directly inserted into the code cache. If the cache is full, one or more contiguous code blocks may require eviction to make room for the new block. In this case, the system must decide what to evict from the cache. The eviction decision may be based on temporal locality guidelines[1], implementation complexity, overhead considerations, or any combination thereof.

The best performance will result from maintaining an optimized copy of the current instruction working set in the code cache. The working set is generally much smaller than the original executable, which along with the improvements in code layout, will potentially improve instruction-cache fetch performance. Additionally, optimizations such as *constant propagation*, *strength reduction*, and *dead-code elimination* will improve performance by customizing the program to the run-time context.

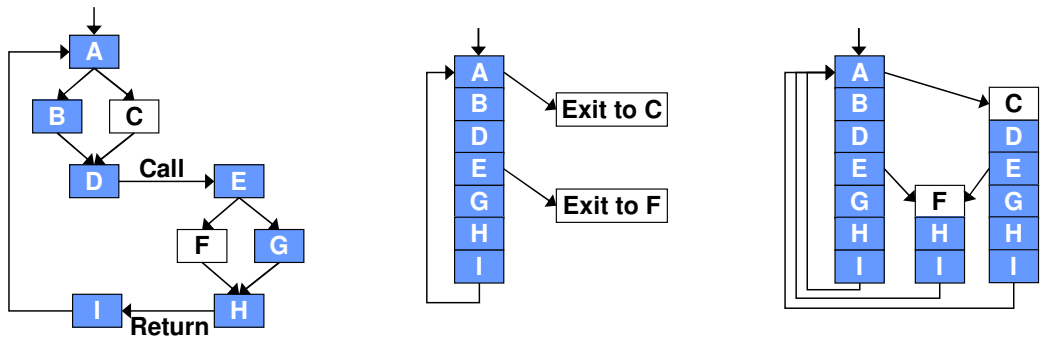
2.1.1 Basic Blocks and Superblocks

Most dynamic optimization systems begin by handling code at the level of *basic blocks*. A basic block of code is a maximal sequence of non-transfer instructions followed by a single control-transfer (jump) instruction. Basic blocks can be entered only at the beginning and exited only at the end, but can be combined to form numerous structures including superblocks [57], hyperblocks [76], treegions [45], or traces [41].

Once a code region is determined to be frequently executed, a dynamic optimization system will combine basic blocks into these larger regions to enable additional optimization opportunities. Many systems, including DynamoRIO, choose to form superblocks—single-entry, multiple-exit instruction sequences—as they are conducive to many traditional compiler optimizations. Figure 2.2 shows an example of a control-flow graph and the equivalent superblock.

Part (a) of Figure 2.2 represents the original control-flow graph of a sequence of basic blocks. As this figure shows, *Basic Block A* is generally followed by a branch to *Basic Block B* then a join point at *Basic Block D*. A function call to *Basic Block E* results in a similar fork and join before returning to *Basic Block I* and a loop back-edge to the top of *Basic Block A*. Figure 2.2(b) shows a resulting superblock that eliminates the fork and join as well as the call and return instructions and results in a straight-line sequence of instructions. The exit stubs *Exit to C* and *Exit to F* initially handle the case of a side-exit from the superblock during execution. The exit stubs return control to the dynamic optimizer to determine the next point of execution and to form the relevant superblocks. Exit stubs contain at least three instructions per exit, and exits tend to occur every six instructions. Over time, the exit stubs are replaced by direct jumps (often referred to as *links*) to the relevant superblocks inside the cache as shown in Figure 2.2(c).

[1] Temporal locality suggests that recently-accessed items are likely to be accessed again soon.



(a) Original control-flow diagram

(b) Superblock with exit stubs

(c) Linked superblocks

Figure 2.2: A superblock example. Each box represents a basic block of instructions. On the left is the control-flow graph representing the dynamic instruction execution – the frequently-executed blocks are shaded. The middle figure represents the corresponding superblock with exit stubs returning control to the dynamic optimizer. The right figure represents multiple superblocks chained together within the code cache.

2.1.2 Superblock Creation Heuristics

Superblock creation heuristics describe *when* and *how* to generate and cache a superblock. While the design details may vary slightly by implementation, the general structure and flow is common among most dynamic optimization systems. This section describes the implementation in DynamoRIO that will be assumed throughout the remainder of this dissertation.

Superblock Heads During program execution, certain basic blocks in an application are marked as *superblock heads* because they are either (a) the target of a backward branch (signaling a loop) or (b) an exit from an existing superblock. These basic blocks will eventually become the start of a new superblock.

Superblock Creation Thresholds Counters are associated with each superblock head, and each execution of this basic block will increment the corresponding counter. A superblock is formed when the counter exceeds a threshold. In DynamoRIO, this threshold is set at 50 executions. This threshold was empirically derived in a study performed on DynamoRIO's predecessor—Dynamo [7]. Other systems have different thresholds for different types of superblock heads. In Mojo (see Chapter 7), the threshold for back-edge targets is 32 executions, while superblock exits have a threshold of 10 executions [22]. In any implementation, the thresholds are defined to indicate that a region has become hot and should be optimized and cached.

Path Selection Once we have determined that a region is hot, a corresponding superblock can be formed, and there are several alternatives for forming superblocks from a series of basic blocks. One option is to profile the execution counts for each basic block, and to create a superblock containing the most-frequently executed basic blocks. Another option is to do path-based profiling [97] and to create a superblock containing the most frequently-executed path through the code. A final option that is implemented in systems such as Dynamo, DynamoRIO, and Mojo is the *Next-Executed Tail* (NET) policy described by Duesterwald and Bala [37]. NET speculatively follows the path of execution during the *nth* execution of a superblock head, where n is the threshold for determining that the region is hot. This policy requires no profiling aside from the counters associated with each superblock head. The result is a speculative sequence of blocks. This policy has been shown to work well in dynamic optimization systems because even if the speculative path is not the most-frequently executed path, the frequently-executed side exit from the superblock will soon become hot and the compensating superblock will be added to the code cache and linked, as was shown earlier in Figure 2.2(c).

2.1.3 Multiple Code Caches

Until this point, dynamic optimization systems have been described as having a single code cache per process. However, many implementations consist of multiple code caches. DynamoRIO, for example, includes two code caches. A *basic-block cache* stores all cold blocks that have been encountered during execution, which allows DynamoRIO to avoid the high overhead of interpretation during every execution of a basic block. The frequently-executed superblocks are stored in a separate *superblock cache*. Therefore, the two code caches in DynamoRIO are distinguished by (a) their execution count, and (b) the type of code they contain – basic blocks or superblocks.

Unless otherwise indicated, we will now focus exclusively on the superblock cache as it contains the more important *hot code* that leads to the most performance improvement. While Chapter 5 will extend the basic cache design to support multiple superblock code caches that are distinguished by the lifetimes of the superblocks they contain [51], all of the code caches described will store frequently-executed superblocks.

2.2 Motivating Bounded Code Caches

Before moving forward with the exploration of code cache management algorithms, it is first important to motivate the need for a replacement algorithm. Cache management is only necessary if we must bound the size of the code cache, which is not immediately apparent when we consider that a code cache is software based. In this section, we will find out why code cache management has been largely overlooked

until this point, and why it is so important today.

Until recently, dynamic optimization systems have largely been confined to the research arena. Therefore, previous code cache management solutions have been driven by performance results on the SPEC 2000 benchmarks. The limited number of superblocks generated while executing the SPEC 2000 benchmarks actually implied that code cache management was not an important issue, because unbounded code caches or unsophisticated replacement policies—such as flushing the cache when it became full—were reasonable solutions. But this situation changes dramatically when we begin to think about production systems that must execute all kinds of software applications. Large interactive applications, for example, where every mouse movement or screen refresh translates into massive amounts of code will truly test the bounds of a code cache replacement algorithm.

While other researchers have discussed salient features of interactive applications and their impact on research in other contexts [19, 69], we will learn about the specific features of these large applications that make them so challenging for a code cache management policy. We will also see why certain existing solutions, such as unbounded code caches, are impractical for such large applications.

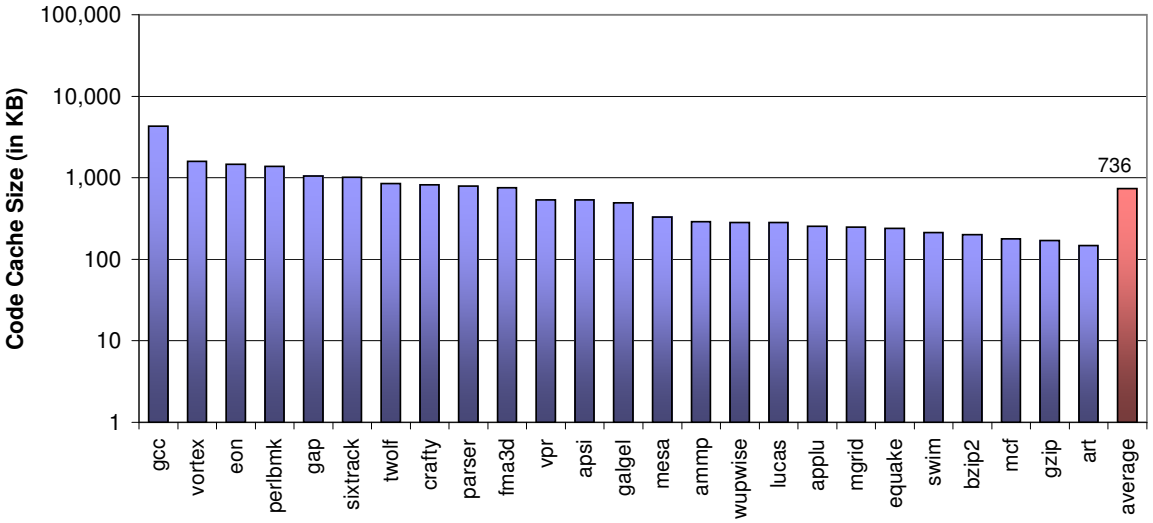
In the following sections, we will investigate the salient features of SPEC 2000 and Windows applications in several respects. All data reported in this section (and throughout the dissertation) was gathered using DynamoRIO.

2.2.1 Maximum Size of Unbounded Caches

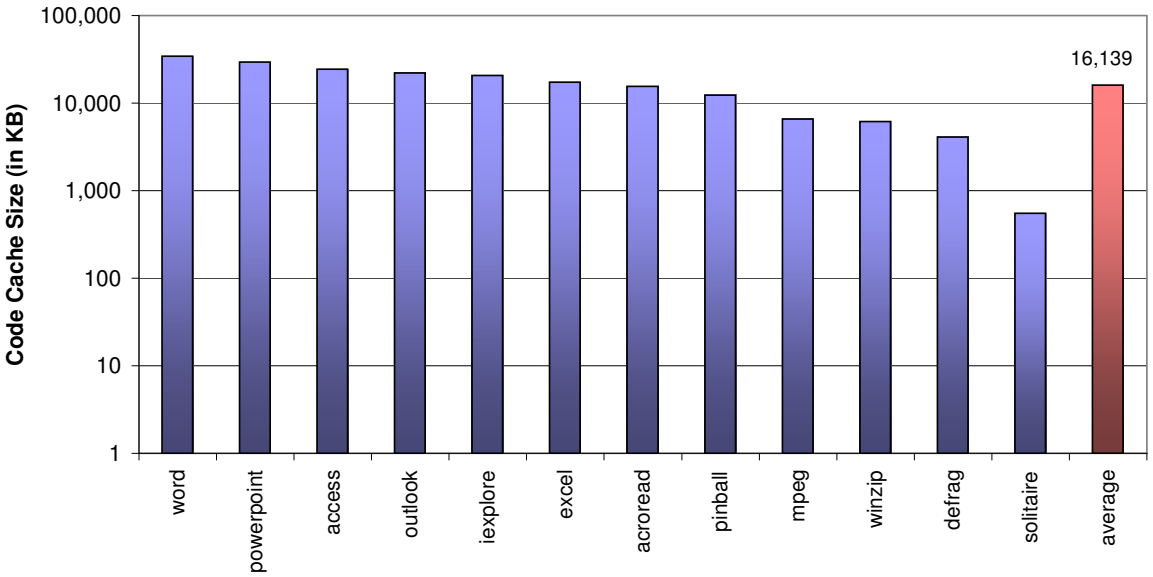
We first look at the amount of memory required in order to support unbounded code caches. Figure 2.3 shows the maximum code cache size reached when DynamoRIO executed various benchmarks with an unbounded cache. On average, the SPEC 2000 benchmarks resulted in a 736 KB code cache. This value is *in addition to* the size of original executable and the DynamoRIO executable. The largest of the SPEC 2000 benchmarks, `gcc`, required a 4.3 MB cache, followed by `vortex` which required a 1.6 MB cache. If we were to focus only on the SPEC 2000 benchmarks (Figure 2.3a), we might conclude that `gcc` is an outlier, and that most applications will fit in less than a 1 MB cache. However, as we move to the interactive benchmarks, we see a twenty-fold increase in the required code cache size. On these benchmarks, the code cache averaged 16.1 MB, with the `word` benchmark requiring a 34.2 MB code cache to avoid code cache management. While most users today have more than 34 MB available, it's important to realize that this value is per application, and most users tend to run more than one application at a time.

2.2.2 Superblock Generation Frequency

Next, we will learn the rate at which superblocks were inserted into the code cache. This metric will allow us to evaluate the strain that will be placed on the cache



(a) SPEC 2000 Benchmarks



(b) Interactive Windows Benchmarks

Figure 2.3: Maximum cache size required to avoid cache management plotted on a logarithmic scale. Note that the maximum cache size of the interactive benchmarks averaged over 20 times larger than SPEC 2000.

management system when cache-size limitations are imposed. Figure 2.4 compares the superblock insertion rate, measured in kilobytes per second (KB/s), of SPEC 2000 and interactive applications. This figure shows that most of the SPEC 2000 benchmarks generate less than 5 KB of superblocks each second—with exceptions including `gcc` which generated 232 KB/s and `perlbmk` which generated 89 KB/s. On the other hand, only one of the interactive applications — `solitaire` — generated less than 5 KB/s. These results suggest that larger applications typically result in a higher frequency (as well as number) of superblocks generated at run time.

2.2.3 Moving Forward: Cache Size Trends

Code caches experience a large amount of code expansion when compared to the original executable size for several reasons. First, code caches store superblocks of original code. Superblocks simplify optimization, but have the unfortunate side-effect of causing tail duplication within a code cache (which could be observed in Figure 2.2(c) where three superblocks contained blocks H and I). Second, all superblocks have exit stubs appended to them to return control to the dynamic optimization system (see Figure 2.2(b)). Each exit stub comprises at least three instructions, and an exit occurs roughly every six instructions. Tail duplication and exit stubs contribute to a significant code bloat within the cache.

The next study therefore examines the size of the unbounded code cache as a multiple of the original code footprint. The application code footprint is defined as the size of the static code executed by DynamoRIO, including system libraries. The code size expansion was calculated using the following equation:

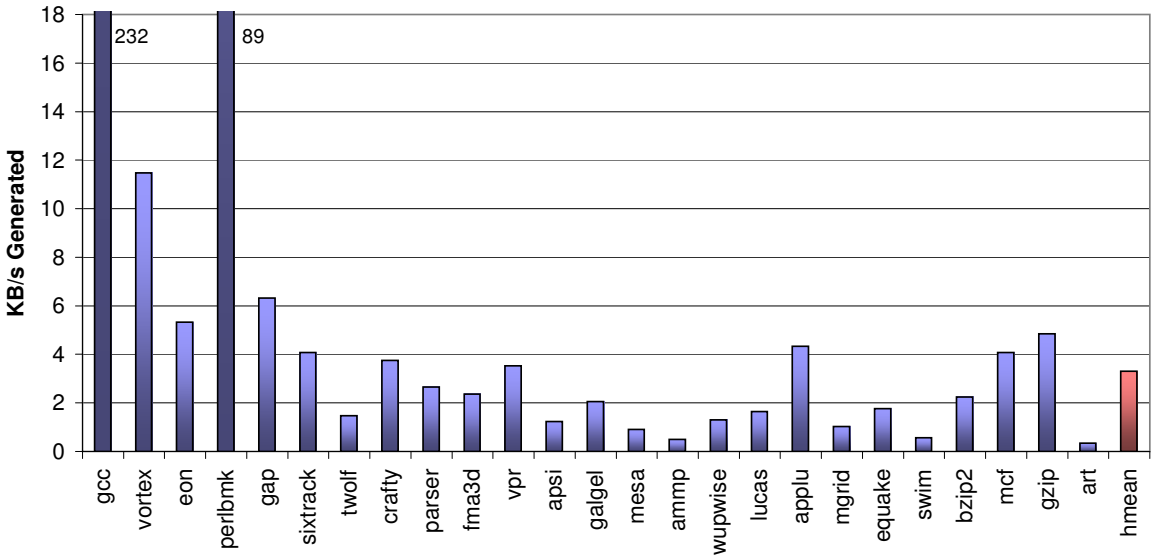
$$codeExpansion = \frac{finalCacheSize}{applicationFootprint} \tag{2.1}$$

A code expansion of 100%, for example, means that we've doubled the size of the original application (original application code + cached code).

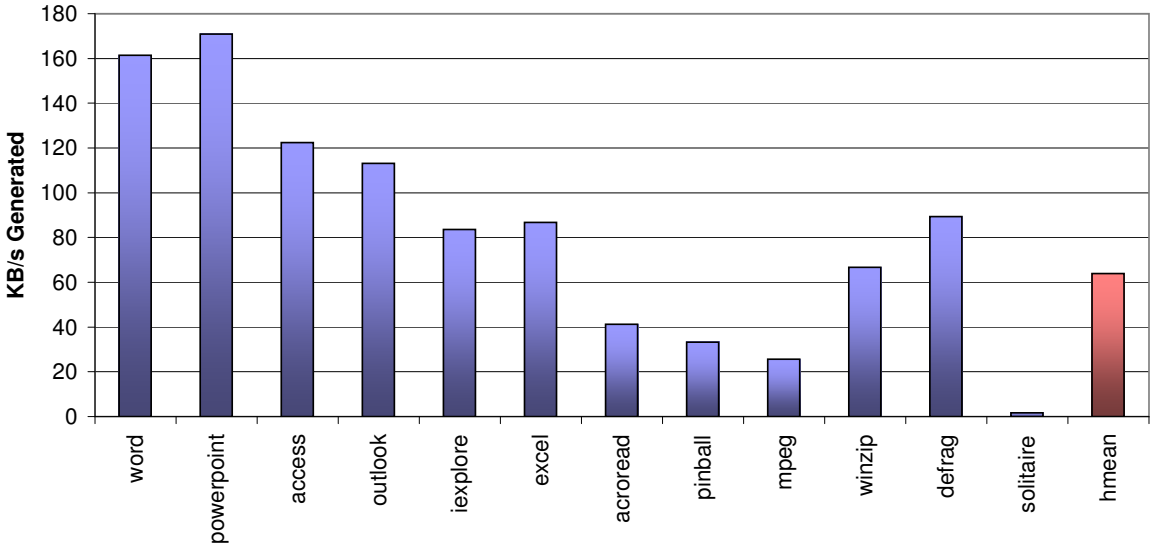
As we see in Figure 2.5, the code expansion factor is roughly 500% for both SPEC 2000 and the interactive benchmarks. Therefore, this indicates that the main factor contributing to the final code cache size is the original application size, which is dramatically larger for the interactive applications. The expansion factor is fairly steady, with only a 59% and 111% standard deviation for the interactive and SPEC 2000 benchmarks, respectively. This implies that as the sizes of new software releases grow, unbounded code cache sizes will grow proportionately.

2.3 Challenges for Code Cache Management

There are several similarities between the code cache management problem and other cache management problems that have been studied in the research community

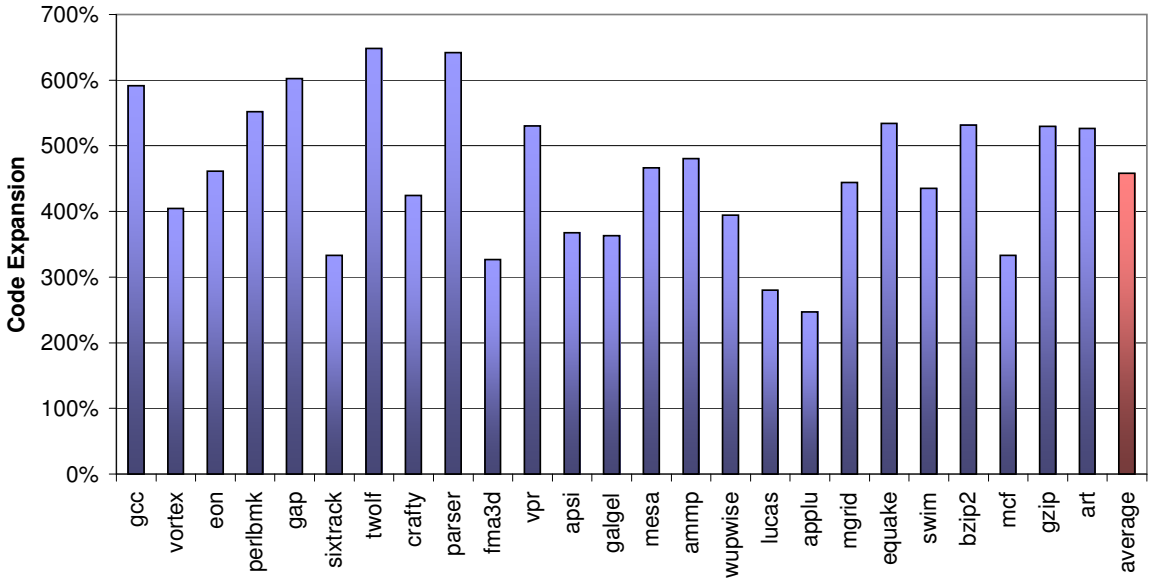


(a) SPEC 2000 Benchmarks

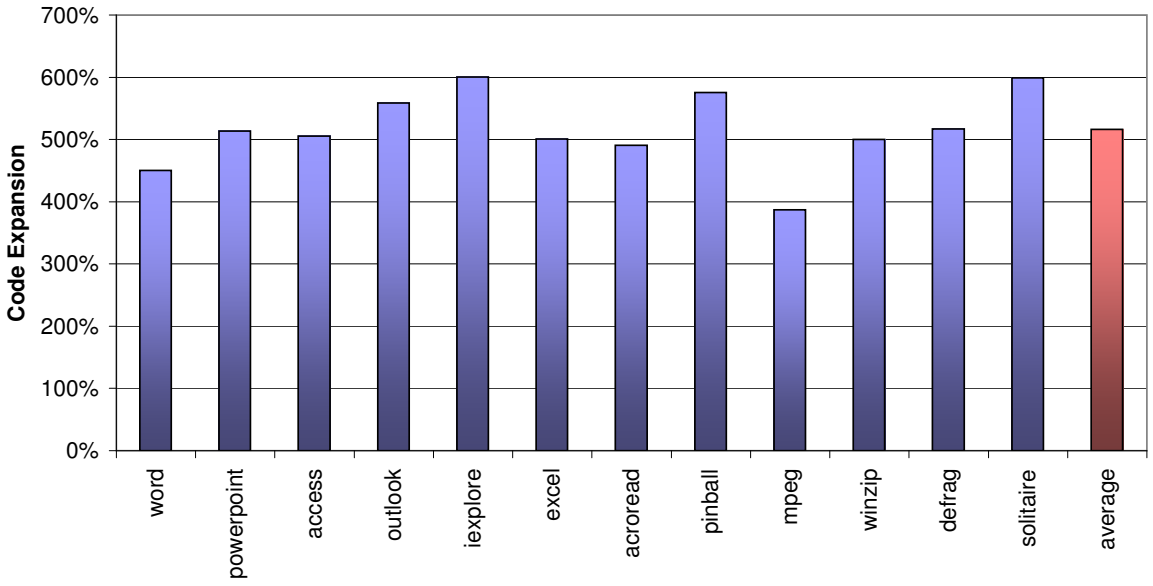


(b) Interactive Windows Benchmarks

Figure 2.4: Amount (in KB) of code generated in the code cache each second. Note that the vertical-axis of the Windows benchmarks is ten times larger than the corresponding axis in the SPEC 2000 graph.



(a) SPEC 2000 Benchmarks



(b) Interactive Windows Benchmarks

Figure 2.5: Code expansion with an unlimited code cache.

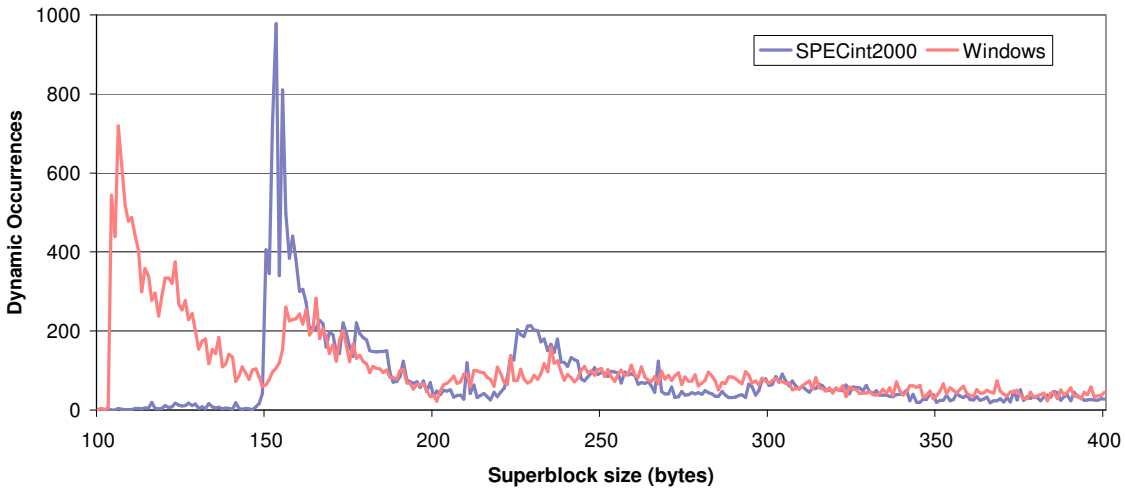


Figure 2.6: Static size distribution of superblocks. The horizontal axis shows a truncated view of all possible superblock sizes. The vertical axis shows the static number of superblocks of each size that would appear in an unbounded code cache. (The truncated portion of the figure is very similar to the 350–400 byte range and extends to several kilobytes.) Results are aggregated over an entire benchmark family.

over the years. Above all, the general idea of maintaining the current working set using limited resources parallels many other caching problems. Any time spent maintaining the cache directly impacts the performance of the executing application. However, several features of code caches necessitate more sophisticated management methods than those used in hardware-based instruction or data caches. In this section, we will look at the specific features of code caches to understand why hardware cache solutions are less effective in this domain.

2.3.1 Caching Varying-Sized Superblocks

An important property of code caches that distinguishes them from conventional hardware caches is that the cached superblocks vary in size rather than being fixed-length blocks. As Figure 2.6 illustrates, the superblock creation heuristics defined in Section 2.1.2 result in cached superblocks that vary significantly in size (from under 100 bytes to several kilobytes).

Implications for Code Cache Management Variable superblock sizes mean that many replacement algorithms, such as LRU, will lead to fragmentation in the code cache. Not only would compaction (to remove fragmentation) be too costly to perform at run time, it would also require adjusting all the branches that allow execution to flow from one superblock to another. Therefore any code cache replacement

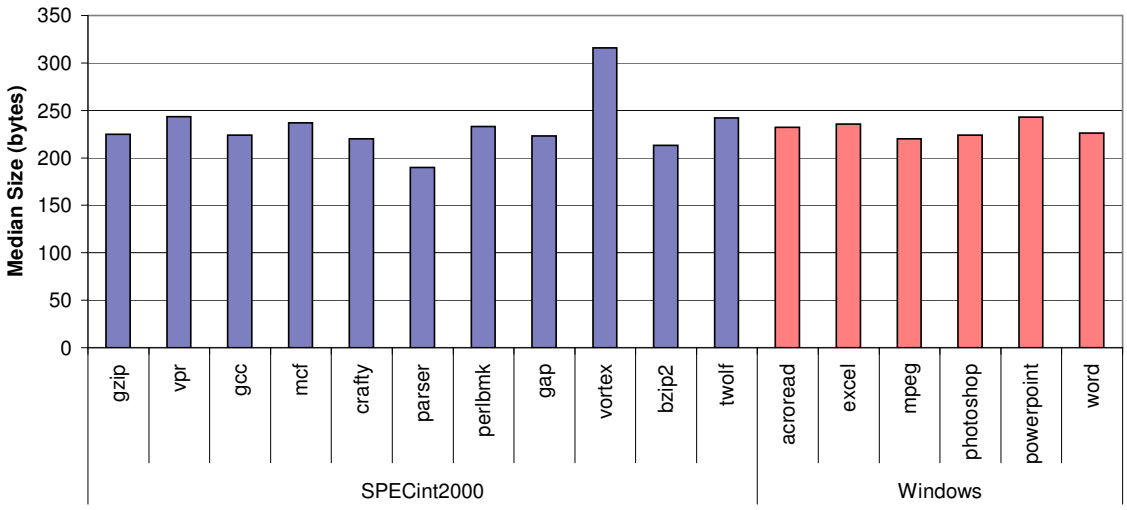


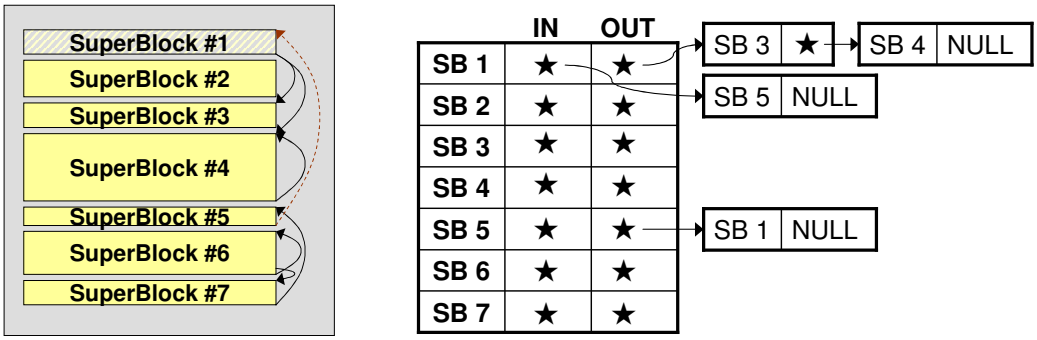
Figure 2.7: Median size (in bytes) of all superblocks in the code cache.

algorithm must attempt to minimize the amount of fragmentation that will occur, or a lightweight compaction operation will be required.

Limiting Superblock Sizes There are several key reasons that superblocks must vary in size and shouldn't be arbitrarily limited to a common size. One major reason is that there is not a common size of superblocks. Figure 2.7 shows that the median superblock size varies—often significantly—across benchmarks. Furthermore, some studies show that the single largest performance benefit of dynamic optimization comes from the improvement in instruction fetch and instruction cache hit rates of forming superblocks [8]. Restricting the size of these superblocks will limit this performance benefit. Finally, limiting the size of superblocks limits the opportunities for applying optimizations to the superblock.

Subdividing Superblocks It is not a good idea to subdivide superblocks after they are cached to enable traditional cache management algorithms for several key reasons. The main reason is that correct execution cannot be guaranteed unless both halves of a superblock always reside in the code cache. The optimizations applied to superblocks assume that control will continue inside the superblock until a defined exit point. Therefore, arbitrarily dividing superblocks and inserting compensating branch instructions will not solve the problem unless the subdivided portions of the superblock can be guaranteed to co-exist. Modern dynamic optimization systems do not maintain enough context information to regenerate a missing portion of a superblock and still guarantee consistency with the assumptions made while generating the existing portion of a superblock.

A final reason not to subdivide superblocks concerns the overhead of doing so.



(a) Code Cache

(b) Back-Pointer Table

Figure 2.8: Code cache with links allowing control to flow directly from one superblock to another. On the right is the corresponding back-pointer table with several pointers initialized. The back-pointer table is a 2-by-N array of linked lists, where N is the number of superblocks in the code cache.

The superblocks stored in a code cache consist of machine instructions. Therefore, correctly subdividing superblocks would require decoding each instruction to determine a safe division point on variable-length instruction-set architectures, such as IA-32, and this decode process is expensive.

2.3.2 Superblock Chaining

One of the major performance boosts in a dynamic optimization system results from *superblock chaining* [8, 25]. This technique allows execution to remain inside the code cache by chaining together superblocks that execute in succession. Good performance is achieved because it is not necessary to return control to the dispatch system to determine the next superblock to execute. Superblock chaining is supported by patching jump instructions to the ends of superblocks residing inside the code cache. Figure 2.8(a) depicts a code cache containing links between elements. From this point forward, the overall process will be referred to as "chaining", while individual pointers will be called "links".

Eviction of superblocks can result in dangling link pointers unless there is a method for determining and eliminating incoming links. For example, to evict superblock #1 from the code cache in Figure 2.8(a), there must be a way to determine that superblock #5 has an outgoing link to superblock #1, so that link can be removed before evicting superblock #1. Otherwise, superblock #5 will contain a dangling link pointer, which could result in incorrect program execution. Therefore, for any cache management policy to ensure code correctness, it must provide support for removing

any incoming link pointers to eviction candidates.

A common solution to this problem is to provide a side table of back pointers, which is depicted in Figure 2.8(b). Before evicting a superblock from the cache, the eviction mechanism can look in the back-pointer table to determine all other superblocks that are linked to the eviction candidate. These incoming links can be removed before proceeding with eviction. Unfortunately, this table carries run-time overhead for lookups and takes up memory that could otherwise be used for code caching. A back-pointer table is unnecessary in code caches that are managed by flushing the entire cache, as all links will be flushed along with the superblocks. Therefore, fine-grained code cache management policies must take into account the performance and memory requirements of supporting link removal.

2.3.3 Superblock Regeneration Overhead

Servicing code cache misses is different from servicing hardware cache misses because elements stored in a code cache do not exist in their identical form anywhere else; that is, there is no backing store. Servicing a code cache miss requires regeneration of the cached code, which carries a high overhead. The specific steps that must be completed before resuming execution of the application are:

1. Saving the processor state of the running program
2. Transforming the next sequence of instructions from the source program
3. Storing the transformed code in the code cache
4. Updating any hash tables and links
5. Restoring the processor state of the executing program and transferring control

As we will soon see, this process takes on the order of 50,000 instructions for a typical SPEC 2000 superblock in the DynamoRIO system [51]. Due to this high code regeneration overhead, designers of dynamic optimization systems will go to great lengths to minimize code cache misses.

2.3.4 Unmapped Memory

Finally, an issue arises whenever we deal with programs that dynamically link and unlink libraries. Anytime a region of memory containing program code is unmapped, the corresponding superblocks must be deleted from the code cache. This is because other code could be loaded into the same address space, resulting in a stale code-cache entry (because superblocks are tagged by their starting address in memory). This phenomenon occurs frequently in Windows applications. In fact, Figure 2.9 shows that an average of 15% of the interactive benchmark's code must be deleted from the dynamic optimizer's code cache due to unmapped memory. This trait of Windows applications can result in inefficient code caches, because *holes* appear in the code

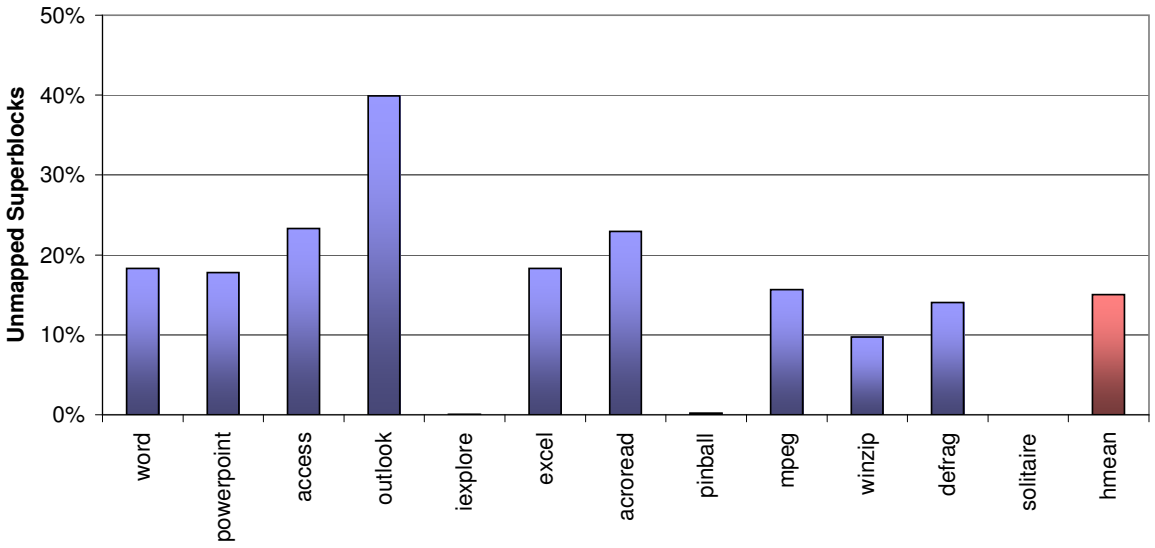


Figure 2.9: Percentage of superblocks that must be removed from the code cache due to unmapped memory.

cache due to the deleted superblocks. Therefore, an effective code cache management mechanism must support superblock deletion, while minimizing fragmentation.

2.4 Summary

While the simplest code cache management policy is not to allow evictions at all, and to allow code caches to grow without bound, this policy is not feasible because:

- Code caches tend to grow to five times the size of the executed application code.
- Modern interactive Windows applications can execute almost 40 MB of code during the course of a few minutes of execution.

Combining these findings with the observation that users tend to execute several programs at once, we can conclude that code cache sizes must be limited, especially as application sizes increase. Managing code caches entails several unique challenges. The cached superblocks vary in size, contain links to one another, and carry a high replacement overhead.

Now that we can recognize code cache management as an important problem with several unique challenges, we can move on to a discussion of code cache management mechanisms designed to overcome these challenges. The remainder of this dissertation explores the many options for managing a limited code cache in the lightweight, efficient manner that dynamic optimization systems require in light of all of the challenges we have outlined in this chapter.

Chapter 3

Traditional Replacement Policies

The driving force behind any code cache management policy is that it must include all of the following attributes: a low overhead implementation, an emphasis on temporal locality, and minimized fragmentation. The first motivating factor—low overhead—is important because the overhead of code cache management directly impacts the application's run-time performance. Therefore, management policies that require a walk of the entire code cache or a complex calculation are not feasible. It will soon be clear that the second factor—temporal locality—is an important attribute of code caches that significantly impacts the performance of code cache management policies. Finally, the third factor—minimized fragmentation—is important for two reasons. First, we are inserting and deleting superblocks that vary in size, therefore fragmentation can occur. Second, in applications that unmap memory, deletions must occur immediately. Therefore, a policy must either include a defragmentation step, or make efforts to minimize the fragmentation that occurs.

This chapter examines several traditional cache management algorithms that have been proposed over the years, and investigates the performance of these algorithms on cached code. We will begin by performing an idealized study of code cache replacement (assuming an omniscient predictor) to form a baseline for comparison. This will be followed by an investigation of six traditional algorithms. The investigation will include a quantitative evaluation of the miss rate of each algorithm and a qualitative evaluation of the resulting fragmentation and implementation complexity.

Gathering Results DynamoRIO and a code cache simulator were used to gather the results in this chapter. DynamoRIO was used to execute the various benchmarks and to gather a trace of code cache insertions and accesses. This trace was stored offline and was later used as input to our trace-driven code cache simulator. The code cache simulator is written in `C` and is parameterizable to support the seven cache replacement policies discussed in this chapter, as well as a range of cache sizes. More details on the experimental setup can be found in Appendix B.

Cache Sizes In addition to providing code cache miss rates, this chapter investigates

how well each of the policies scales under increasing cache pressure. In practice, the size of a dynamic optimizer's code cache is generally fixed for all applications. The effect is that code cache management strategies result in bimodal cache performance. For applications that fit within the code cache, the choice of code cache management policy makes no difference. However, for applications that do not fit within the code cache, performance can suffer precipitously.

We are most interested in studying code cache management behavior when it is being stressed, i.e. when the program working set exceeds the size of the code cache. Therefore, for all results in the next three chapters, the code cache is sized to ensure that the replacement mechanism will be stressed. This will be done by introducing a *cache pressure factor* that determines the size of the code cache. The cache sizes that will be used in the next few chapters are defined as:

$$cacheSize = maxCacheSize/cachePressureFactor \tag{3.1}$$

where $maxCacheSize$ is defined as the size that the cache would reach if we allowed it to grow without bound for a particular benchmark. The $maxCacheSize$ term ranges from 171 KB for the smallest benchmark—`gzip`—where 301 superblocks were cached, to 34.2 MB for the largest benchmark—`word`—where 18,043 superblocks must be cached. The cache pressure factor will vary incrementally from 2 to 10, where 2 represents low cache pressure, and 10 represents a smaller cache with high cache pressure. A pressure factor of 2 means that the cache size is 1/2 as large as is necessary to avoid cache management while a pressure factor of 10 means that the cache size is 1/10 as large as is necessary.

3.1 Optimal Replacement

An ideal code cache replacement algorithm is one that can see into the future, and will evict the superblock that will not be accessed for the most amount of time. This optimal replacement policy was first studied by IBM's Laszlo Belady in 1966 [10]. While there is no way to implement such a policy in a real cache, the results acquired from simulating this policy are useful for understanding how close other algorithms come to the best possible miss rate for each cache size. The resulting optimal miss rates are shown in Figure 3.1, where we see that the miss rates vary from 0.15% under low cache pressure to 1.10% under high pressure.

3.2 Full Flush

One cache management scheme that was designed with a simple implementation in mind is the full code cache flush mechanism. Superblocks begin filling the code cache at its lowest address and continue filling toward higher addresses. As soon as a

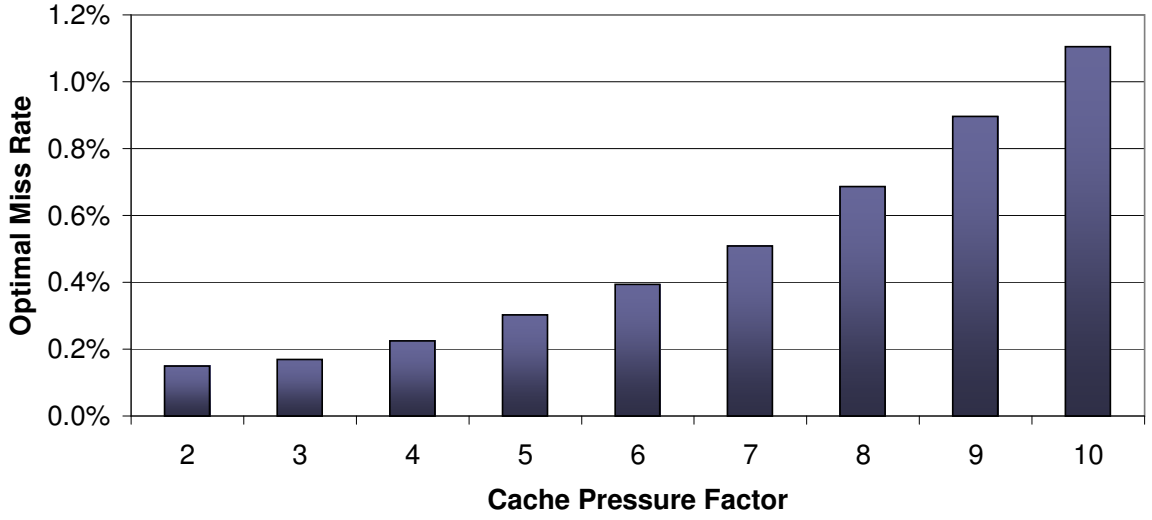


Figure 3.1: Optimal code cache miss rates averaged across the SPEC 2000 integer benchmarks. Cache pressure varies from 2 to 10.

superblock is encountered that cannot be inserted into the cache, all superblocks are flushed, and the new superblock becomes the first item inserted into the empty code cache.

While this is a low-overhead cache management strategy, it has the adverse side effect of flushing hot superblocks from the cache. If these hot superblocks are subsequently rebuilt and reinserted into the cache, unnecessary overhead is encountered. Table 3.1 shows the number of full cache flushes that occur during execution of each of the SPEC 2000 benchmarks. The table shows that the number of cache flushes increases exponentially as we add cache pressure.

Furthermore, Table 3.1 also shows us that 61.5% of the flushed superblocks (on average) are regenerated in the cache, clarifying the point that useful superblocks are often flushed from the cache and unnecessary overhead is encountered to regenerate those superblocks. The miss rate of this policy is shown in Figures 3.2–3.4, where we see that it varies from 0.39% under low cache pressure to 4.52% for high pressure.

3.3 Least-Recently Used

Another common cache management policy is the Least-Recently Used (LRU) policy. For this scheme, superblocks are again inserted into the cache in the order that they are created. However, when the code cache becomes full, the block that has not been accessed in the greatest amount of time becomes the first candidate for eviction. In the event that this block will not free enough space to hold the newly

Bench-mark	**Cache Pressure Factor**									Regener-ation
	2	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	
gzip	4	11	15	24	42	225	568	800	905	84.6%
vpr	2	4	6	10	13	17	28	117	191	38.7%
gcc	8	63	121	166	206	251	300	361	423	42.9%
crafty	5	27	116	825	3066	7651	14498	23971	36560	29.9%
parser	4	102	230	893	1123	1505	1792	2085	2382	58.8%
eon	2	4	7	24	329	2796	7999	11799	15227	80.5%
perlbmk	4	9	19	33	51	72	100	126	154	61.5%
gap	3	9	19	32	58	130	231	355	510	63.3%
vortex	2	14	31	350	518	1079	1660	2862	5816	64.5%
bzip2	2	5	8	10	12	16	20	23	30	81.3%
twolf	2	5	8	13	44	165	193	244	265	66.1%

Table 3.1: Number of cache flushes during execution as cache pressure increases. The rightmost column lists the percentage of flushed superblocks that are later regenerated in the code cache (when pressure=2).

Replacement Policy	LRU	FIFO	LFU	BFE	Flush	Random	Optimal
Evictions / Insertion	1.47	1.62	1.45	1.09	1.00	1.64	1.81

Table 3.2: Evictions per cache insertion averaged over all benchmarks.

optimized block, the subsequent block(s) following the LRU element in the code cache are also evicted (since we need to free enough contiguous space for the new block). These subsequent evicted superblocks will be referred to as *victim superblocks* because they are not necessarily the next LRU element.

This scheme has the benefit of leveraging temporal locality, yet it has the unfortunate side-effect of removing innocent victim blocks from the cache in order to create adequate contiguous space. Table 3.2 gives an indication of the average number of evictions that typically occur in order to make room for the insertion of a single block. From this figure, we see that it is usually necessary to evict one to two blocks from the cache for each new block that is inserted (or 47% of the time, an eviction will result in the eviction of one additional victim block.) A second side-effect of this scheme is that it will create code cache fragmentation.

The miss rate of this mechanism is shown in Figures 3.2–3.4 where we see that it varies from 0.17% under low cache pressure to 2.34% under high pressure.

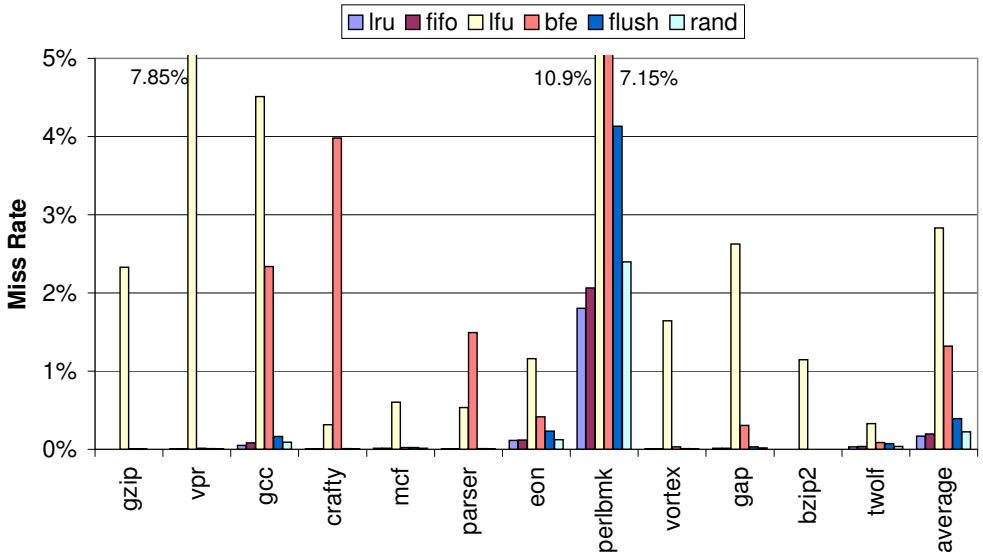


Figure 3.2: Miss rates of traditional cache algorithms. Cache pressure factor = 2. (See Table B.3 in Appendix B for raw data.)

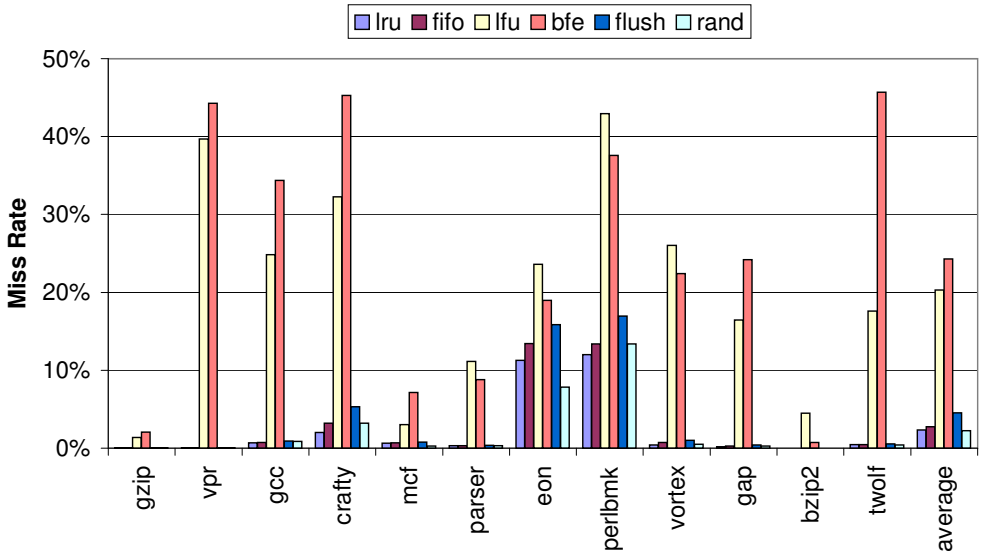


Figure 3.3: Miss rates of traditional cache algorithms. Cache pressure factor = 10. (See Table B.4 in Appendix B for raw data.)

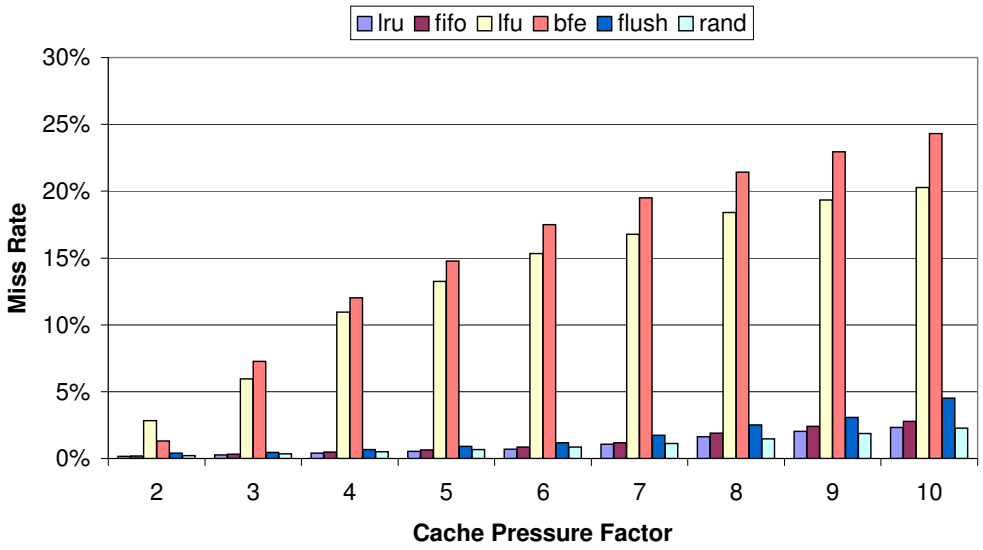


Figure 3.4: Miss rates of traditional cache algorithms averaged over all benchmarks. Cache pressure factor varies from 2 to 10.

3.4 Least-Frequently Used

An alternative to LRU is the LFU policy, which evicts the *Least-Frequently Used* superblock from the code cache. By maintaining a counter that tracks the number of accesses to each optimized superblock in the code cache, we can determine the LFU element. In the event of a tie, FIFO (see Section 3.5) is used as a tie breaker.

Just as under the LRU policy, we evict the LFU block and any subsequent superblocks necessary to make enough room for the replacement block. Also like the LRU scheme, LFU will suffer from the effects of code cache fragmentation. Table 3.2 shows that 45% of evictions result in a victim block eviction, thus victims are evicted at a slightly lower rate than LRU. Finally, while this scheme will effectively recognize hot superblocks and allow them to remain in the code cache, it has the adverse effect of removing new superblocks from the cache, which haven't yet acquired a high access count. As we can see from Figures 3.2–3.4, this policy doesn't fare well, resulting in a 2.83% miss rate under low cache pressure and a 20.29% miss rate under high pressure.

3.5 First-In First-Out

A simple code cache management strategy that leverages temporal locality is a model that treats the code cache as if it were a circular buffer. Called the First-In First-Out (FIFO) method, superblocks in the code cache are replaced in the same order as they were inserted. In the case where the next candidate for replacement will

not free enough space for the new block, subsequent superblocks are also removed.

Though the average number of fragments removed is larger than LRU and LFU (shown in Table 3.2), the victim superblocks removed in this scheme were already next in line for eviction. In the case where an evicted block frees much more space than is needed for the newly optimized block, the free space will be used by the next block inserted into the code cache, thus avoiding fragmentation. In terms of bookkeeping overhead, we merely update a pointer after each block insertion. And, by proactively removing additional FIFO superblocks from the code cache, as we will discuss in Chapter 4, additional cache management overhead can be eliminated. The miss rate of this mechanism is shown in Figures 3.2–3.4 where we see that it varies from 0.20% under low cache pressure to 2.77% under high pressure.

3.6 Best-Fit Element

Another strategy is one that attempts to minimize fragmentation by searching the code cache for the best-fit block to evict. In the Best-Fit Element (BFE) scheme, the code cache is scanned in search of the smallest superblock that is greater than or equal to the size of the newly optimized superblock. When the best-fit block is found, it is evicted from the cache, ideally leaving just enough room for the new block. In the event of a tie, FIFO is used as tie breaker. In the case where all superblocks in the cache are smaller than the new block, superblocks are then grouped in subsequent pairs of two, and the best-fit search continues.

Table 3.2 shows that most of the time (91%), only one best-fit block must be evicted from the cache to make room for an incoming block. But this scheme will require a more sophisticated mechanism for finding the next element to evict, such as forming buckets containing a list of blocks within a certain size range. More importantly, the miss rate suffers from ignoring temporal locality, with a 1.32% miss rate under low cache pressure and a 24.30% miss rate under high pressure.

3.7 Random Element

The final replacement policy will provide insight into the benefits of temporal locality by determining what happens when we implement a policy that is somewhat fair (all elements have an equal chance of eviction, regardless of access times) but no priority has been placed on temporal locality when evicting superblocks.

With the help of the `drand48()` function in the `C` standard libraries, a random superblock was selected for eviction whenever a new superblock required insertion into the code cache. Table 3.2 shows that this policy has one of the highest percentages of victims at 64%. Most interestingly, we see from the miss rates in Figures 3.2–3.4 that under low cache pressure, Random performs worse than LRU and FIFO. But as pressure increases Random begins to match the miss rates of LRU and FIFO. This

Policy	Fragmentation	Victims	Miss Rate	Implementation
Optimal	Yes	81%	1.10%	not possible
Random	Yes	64%	2.27%	hash table
LRU	Yes	47%	2.34%	priority queue
LFU	Yes	45%	20.29%	priority queue
BFE	Minimal	9%	24.30%	multiple sorted lists
Flush	None	N/A	4.52%	pointer
FIFO	None	62%	2.77%	pointer

Table 3.3: Summary of traditional replacement algorithms (under high cache pressure.) The miss rates of LFU and BFE are particularly high because both policies ignore temporal locality. Flush and FIFO do not suffer from fragmentation and have a simple implementation.

is likely because the cache is too small to capture enough of the working set of the application, therefore LRU and FIFO essentially resemble Random replacement.

3.8 Summary

The high cost of preparing superblocks for insertion into a cache, combined with the fragmentation issues involved in replacing superblocks that vary in size has caused many dynamic optimization system developers to sidestep the issue and implement either an enormously large code cache or an unsophisticated replacement scheme such as flushing the entire cache. This chapter explored several other algorithms and discussed the benefits and trade-offs of each model, which are summarized in Table 3.3.

Weighing the factors of (1) code cache miss rate, (2) fragmentation, and (3) implementation complexity, it is clear that treating the cache as a circular buffer and implementing a FIFO replacement policy is an effective approach to code cache management. In the next chapter, we will explore the trade-offs of performing a full flush versus performing FIFO replacement in more detail.

Chapter 4

Local Code Cache Management

Now that traditional replacement policies have been evaluated and FIFO has been identified as an effective policy with a simple implementation, we can move on to the novel contributions of this dissertation that build upon a FIFO replacement policy. The extensions are designed to overcome many of the challenges of code caches outlined in Chapter 2 while providing scalable performance under high cache pressure.

The code cache replacement strategies that will be discussed in the next two chapters fall into two different categories. *Local code cache management* refers to replacement policies implemented within a single code cache, with examples including the LRU and FIFO policies explored in Chapter 3. *Global code cache management* refers to the hierarchy and interaction policy between multiple code caches, such as the basic block and superblock caches described in Section 2.1.3. It is important to investigate both effective local and global policies that may be implemented, as the two are often complementary.

This chapter takes the pure FIFO scheme identified in the previous chapter and extends it to handle issues incurred in the design of a real dynamic optimization system. The chapter first describes some of the run-time issues that were not incorporated into the simulations of the previous chapter and introduces a variant of the baseline FIFO algorithm that will overcome these issues. Then, we will look at a variant of FIFO that evicts large numbers of superblocks in mass to avoid the bookkeeping overhead of managing single superblocks.

Through a combined simulation and analytical study of cache management overheads, we will see that employing a medium-grained FIFO eviction policy results in an effective balance of cache management complexity and cache miss rates, which under high cache pressure can translate into a significant reduction in overall execution time versus both coarse and fine granularities.

4.1 Pseudo-Circular FIFO Replacement

While some of the challenges of code cache management were carefully considered when evaluating the FIFO policy in Chapter 3, several other issues are present that are difficult to simulate, but nonetheless must be handled in any real implementation. These issues include *undeletable blocks* and *program-forced evictions*, and they result in making a pure circular-buffer implementation unachievable in a real system.

Undeletable Blocks Several situations may arise that require a dynamic optimizer to mark a block undeletable from the code cache. For instance, during an exception that occurs in a block inside the code cache, execution of that block must be suspended while the exception is handled. The exception handling code itself is also run under the control of the dynamic optimization system and therefore, may cause insertions and evictions within the code cache. Yet, because control will later return to the superblock that caused the exception, it is vital that it is not deleted in the interim. Furthermore, control may have left the excepting superblock at an unexpected point, therefore the dynamic optimization system doesn't have enough state information to regenerate the return point.

Program-Forced Evictions Figure 2.9 showed that code cache management is not the only reason for evictions from the code cache; unmapped code regions require deletions of corresponding blocks from the code cache. Such evictions introduce fragmentation in all code cache mechanisms, including the circular buffer.

These complications warranted the design of a *pseudo-circular FIFO* policy, which is a variant of the standard circular buffer FIFO scheme. From a distance, this policy behaves as a circular buffer. A simple cache pointer is maintained to keep track of the next element up for eviction. Insertion of a new superblock into the cache will evict zero or more existing superblocks in the cache depending on the free cache space and the size of the superblock to be inserted.

However, the pseudo-circular scheme deviates from normal circular eviction when an undeletable block is detected in the list of potential eviction candidates. At this point, the circular buffer mechanism resets its eviction pointer to begin evicting directly after the undeletable fragment, and the eviction process begins again. The algorithm is presented in Figure 4.1.

This policy does not change its behavior in response to program-forced evictions, which inherently violate the circular eviction policy. While it would be possible to maintain a list of holes in the code cache, and to try inserting into those before evicting superblocks located at the eviction pointer, this complicates the cache management design, and may reduce the benefits of temporal locality. Instead, a simple solution is to add the size of any holes resulting from program-forced evictions to the size of the predecessor superblock in the code cache, then continue as normal.

Pseudo-Circular FIFO Eviction

```
insertNewBlock {
  free_space = leftover_space
  eviction list = NULL
  full_circle_check = FALSE
  while (free_space < new_superblock_size) {
    if (new_superblock_size > (end_of_cache - fifo_ptr)) {
      fifo_ptr = top_of_cache
      eviction list = NULL
      free_space = 0
      if (full_circle_check == TRUE) return ERROR
      full_circle_check = TRUE
    }
    while (fifo_ptr points to undeletable superblock) {
      fifo_ptr += size of undeletable superblock
      eviction list = NULL
    }
    if (fifo_ptr points to empty space) {
      free_space += size of empty space
      fifo_ptr += size of empty space
    }
    else {
      eviction list += superblock at fifo_ptr
      free_space += size of superblock at fifo_ptr
      fifo_ptr += size of superblock at fifo_ptr
    }
  }
  evict elements in eviction list
  insert new superblock in code cache
  leftover_space = (free_space - new_superblock_size)
}
```

Figure 4.1: An algorithm for pseudo-circular FIFO code cache eviction. The `insertNewBlock` routine is called when a new superblock is inserted into the cache. The variable `fifo_ptr` points to the next insertion point in the circular buffer.

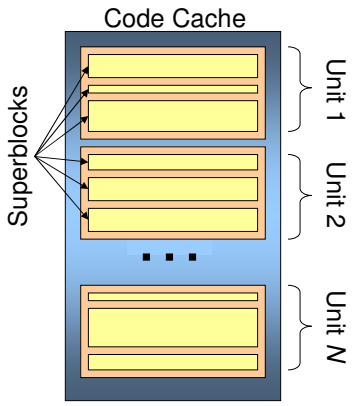


Figure 4.2: Conceptual view of medium-grained cache eviction units.

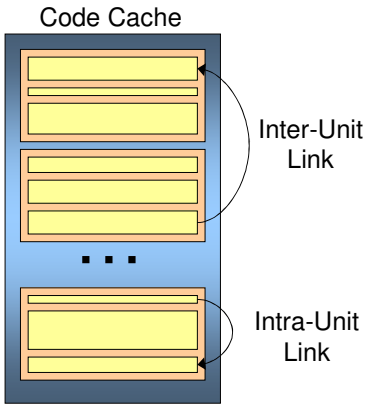


Figure 4.3: Distinction between intra-unit and inter-unit links.

4.2 Code Cache Eviction Granularity

There is an important trade-off with respect to evicting code from a code cache. Evicting a single superblock at a time will minimize the miss rate, but will lead to relatively high overhead for two reasons. First, there is some fixed overhead in invoking the eviction code, no matter how many superblocks are evicted. Evicting single superblocks will lead to a high number of invocations and therefore a larger total amount of fixed overhead. Second, as we shall see, evicting single elements tends to maximize the number of dangling links that must be fixed via a back-pointer table.

At the other end of the spectrum, one can employ the cache management scheme where the entire cache is flushed when either (a) the cache becomes full, or (b) when a program phase shift is detected. This policy will greatly reduce the fixed overhead of invoking eviction, and it will eliminate the need to adjust link pointers. However, flushing the cache will result in noticeably higher miss rates, as was shown in Chapter 3.

In this section, we explore the spectrum of eviction granularities, including the middle ground between flushing the entire cache and eliminating a single element. We refer to this middle ground as medium-grained code cache evictions [48] and the idea is illustrated in Figure 4.2. As this figure indicates, the cache is partitioned into *cache units* of roughly equal size, each containing several superblocks. Instead of evicting a single code block, an entire cache unit is evicted at a time, typically leaving enough room for several individual code blocks to be inserted before the code cache manager must be re-invoked.

Furthermore, organizing the cache into units will simplify the number of links that must be repaired during cache eviction. As Figure 4.3 illustrates, cache links can now be partitioned into two distinct types – *intra-unit links* and *inter-unit links*. Intra-unit links target superblocks that reside in the same cache unit. Interestingly, these

links do not require repair when flushing a cache unit because the dangling links will be flushed along with the target superblocks. Inter-unit links that target superblocks in other cache units will still require fix-up after a code cache eviction.

To better understand the trade-offs of evicting small vs. large code units, we will look at the impact of eviction granularity on cache miss rates and eviction overheads. The experimental setup involves the use of the DynamoRIO dynamic optimization system and a code cache simulator. We used both the Windows and Linux versions of DynamoRIO release 0.93. We used the PAPI [72] performance counter interface to the Pentium processors to collect the overhead estimates, as outlined in Appendix B.3. In addition, we used the verbose output from DynamoRIO to drive the code cache simulator; therefore we were able to represent the actual code regions that a code cache would manage including actual region sizes and inter-region links. We were able to save and reuse the DynamoRIO logs to allow for repeatability in the experiments.

As we will see, a combination of SPEC 2000 and Windows benchmarks were executed for this study. (Table B.1 and B.2 in the Appendix list the specific benchmarks, including all 12 SPEC 2000 integer benchmarks that were run on Linux and eight interactive Windows applications that were driven by manual user interaction performing everyday tasks.) As we learned in Chapter 2, interactive applications are particularly important in code cache studies because the rate and amount of generated code in these applications tests the limits of code cache management systems.

4.2.1 Cache Miss Rates by Granularity

For all results, the code cache eviction granularity was varied from a full code cache flush down to the finest-grained FIFO eviction policy, which evicts only enough superblocks to make room for the inserted superblock. Again, to ensure code cache pressure, the size of the entire code cache was set to be $maxCache/cachePressureFactor$. As described in Chapter 3, $maxCache$ is the size that the code cache would reach if it was allowed to grow without bound for the particular benchmark, and $cachePressure-Factor$ reduces the size of the cache by a factor of 2–10 to ensure that the cache management policy is truly stressed.

A weighted average was used to calculate a single cache miss rate across all benchmarks in a benchmark family (SPEC 2000 and Windows). For instance, the weighted average across the Windows benchmarks was determined using Equation 4.1.

$$unifiedMissRate = \frac{\sum_{i=\texttt{iexplore}}^{\texttt{word}} (cacheMisses_i)}{\sum_{i=\texttt{iexplore}}^{\texttt{word}} (cacheAccesses_i)} \tag{4.1}$$

As was suggested earlier, the code cache miss rate is expected to increase as the grain size for evictions increases. This was verified using the code cache simulator,

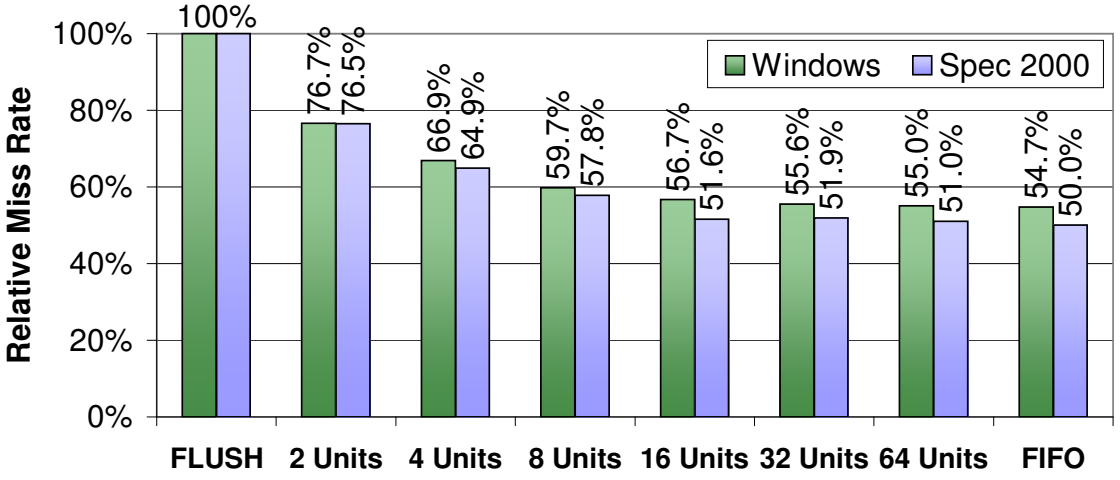


Figure 4.4: Relative miss rates at varying granularities (with respect to FLUSH). Cache pressure fixed at 2. This figure shows that for both Windows and SPEC 2000, finer-grained evictions result in a lower relative miss rate. See Table B.5 in Appendix B for individual benchmark results.

and the results are shown in Figure 4.4 for a code cache pressure of 2. The leftmost bar represents the coarsest granularity possible—treating the cache as a single unit and flushing it entirely. Moving to the right, the cache is split into two equally-sized cache units and each is flushed separately in a FIFO fashion. Finally, the rightmost bar represents the case where each individual superblock is treated as a single eviction unit, and only enough superblocks are evicted to make room for the new one being inserted. As the figure illustrates, miss rates decline as the cache evictions become more fine grained.

Figure 4.5 then shows how the miss rate of each eviction granularity scales under pressure. As the figure indicates, the differences in miss rates become much more pronounced as pressure increases. However, miss rates tell only part of the story.

4.2.2 Code Cache Evictions by Granularity

Reduction in miss rate through fine-grained cache evictions is balanced by the increased overhead of performing more evictions. Figure 4.6 shows the impact of eviction granularity on the number of times that the eviction mechanism must be invoked. The values shown in Figure 4.6 are relative percentages with the baseline set to the number of times that eviction would be necessary had a fine-grained, minimum-element eviction scheme been implemented. It is interesting to note that moving from the finest-grained FIFO to a slightly coarser 64-unit eviction policy increases the miss rate only slightly (Figure 4.4) but the number of evictions decreases by nearly a factor

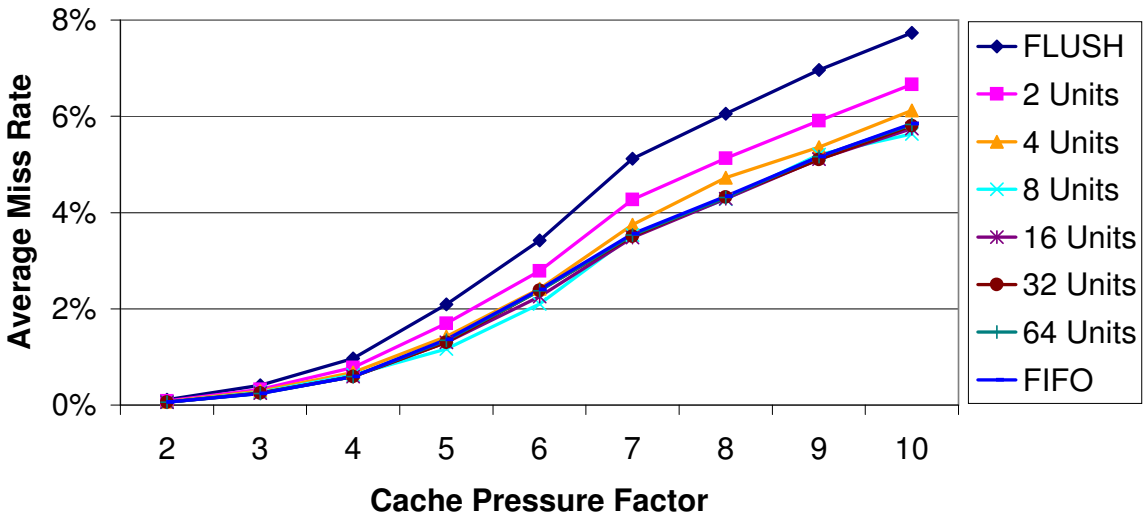


Figure 4.5: Combined SPEC 2000 and Windows miss rates at varying granularities as cache pressure increases.

of 3 compared to the fine-grained implementation (Figure 4.6).

Next, we used the PAPI performance counter interface to study the actual overhead of code cache evictions in the DynamoRIO framework. We collected a log of over 30,000 code cache evictions, including their eviction size (in bytes) and the number of instructions required to perform the eviction. We then used a least-squares linear regression trend-line (described in Appendix B.3) to develop Equation 4.2.

$$evictionOverhead = 2.77 * sizeBytes + 3055 \tag{4.2}$$

This equation tells us the average number of instructions required to evict a superblock of a given size (in bytes) from the code cache. Earlier in Chapter 2 we saw that the median superblock size is about 230 bytes in DynamoRIO, and rarely do we encounter a superblock larger than about 500 bytes (from Figures 2.6–2.7). Therefore, using Equation 4.2, we see that an eviction of 230 bytes of code would require 3,690 instructions. We also see from the equation that the main factor contributing to the overhead of evictions is the start-up cost (i.e. the constant term 3055 in the equation) and the dependence on the number of bytes evicted is a much less substantial portion of the overhead. This implies that it is advantageous to evict larger blocks of code from the code cache.

We repeated the process of instrumenting the code to determine the expected overhead of a cache miss. We did so by collecting data on superblock generation, code cache insertion, and hash table updates as they occurred in DynamoRIO. Using this methodology, the overhead of a cache miss can be estimated by Equation 4.3.

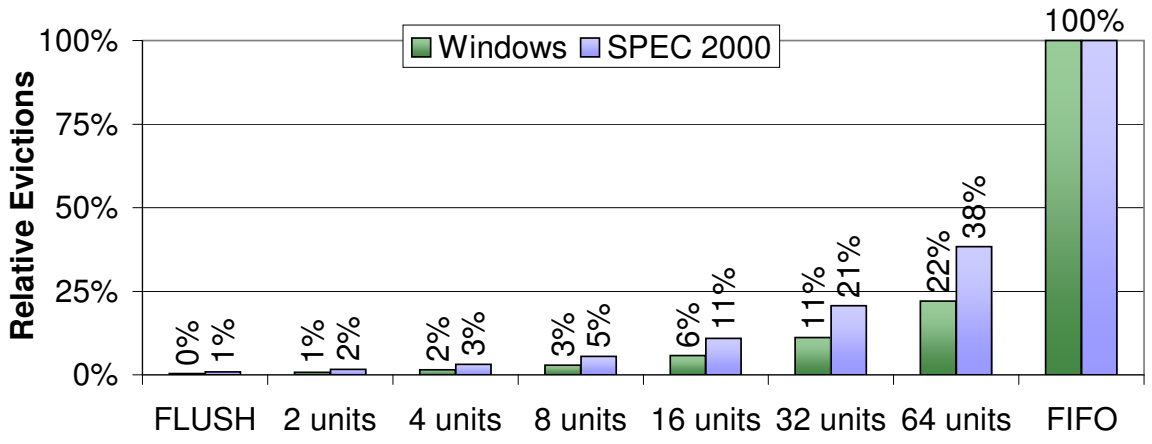


Figure 4.6: Relative number of evictions compared to finest-grained FIFO eviction.

$$missOverhead = 75.4 * sizeBytes + 1922 \tag{4.3}$$

This equation tells us the expected overhead of a code cache miss for a superblock of a given size (in bytes). We see from this equation that the size of the superblock plays a much larger role in the overhead of a cache miss (as compared to Equation 4.2) because superblock formation itself is highly dependent on the amount of code that must be modified and cached. Servicing a cache miss for a 230-byte superblock, therefore, tends to require 19,264 instructions.

By combining the instruction penalty of eviction (Equation 4.2) with the instruction penalty of a cache miss (Equation 4.3), we can begin to understand the overhead trade-offs of code cache eviction granularity.

4.2.3 Cache Miss and Eviction Overhead by Granularity

We introduced the notion of *overhead penalties* to the code cache simulator and re-executed the benchmarks to study the trade-offs encountered as the code cache granularity is varied. We used the overhead penalties defined in Equations 4.2–4.3, and set the cache pressure to be 10 (the cache was sized at $maxCache/10$).

The results are shown in Figure 4.7 normalized to the coarsest-grained policy (`FLUSH`). This figure exhibits several interesting trends. The eviction policies on the far left perform worst because their high code cache miss rates are not offset by the reduction in the number of evictions. Moving to finer-grained evictions, there are reductions in overhead as a result of the improvement in hit rate. However, moving to the finest-grained eviction policies results in an increased overhead due to frequent invocations of the cache eviction mechanism. There is clearly a delicate balance between reducing cache misses and reducing eviction overhead. While fine-grained

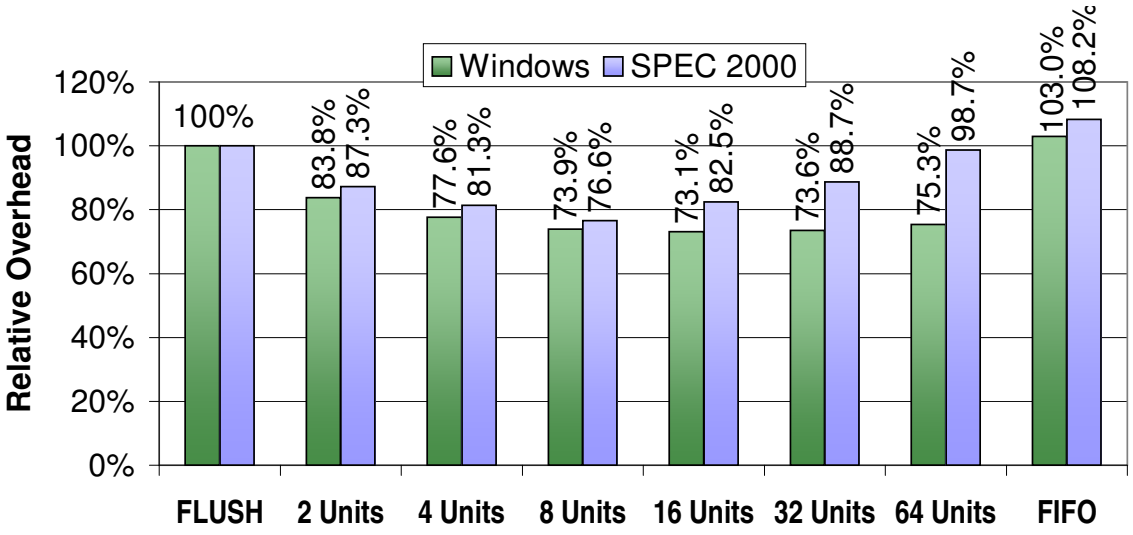


Figure 4.7: Relative overhead of various code cache eviction granularities including miss rate overhead and eviction overhead. The code cache is sized at $maxCache/10$.

cache eviction minimizes the miss rate, it also maximizes the number of evictions that must occur. Therefore, at this point results indicate the most robust and scalable eviction policy is to aim for medium-grained evictions.

There is an additional factor that must be considered before making concrete conclusions about the benefits of various code cache eviction granularities, however. Superblock chaining—as described in Section 2.3.2—is a subtle feature, but one that could result in a significant difference in the complexity of the eviction mechanism and skew the resulting run-time overhead.

4.2.4 Analysis of Superblock Chaining

As discussed in Section 2.3.2, one of the advantages of the full cache flush mechanism is that there are no dangling links resulting from individual superblock deletions. As the code cache is flushed, all superblock links are eliminated simultaneously with the cached code. Therefore, there is no need to maintain a table of link back-pointers. This eliminates both the memory required for the back-pointer table and the overhead to maintain it. Yet, in Section 4.2.3, we found the miss rate of the full flush mechanism is higher than with other granularities.

Since we are exploring partitioning the code cache into several units, each of which will be flushed entirely, we can still achieve some of the superblock chaining benefits of the full flush mechanism. Several superblock links will be eliminated as an entire cache unit is flushed, therefore back pointers are unnecessary for those *intra-unit superblock links*—links that connect two superblocks residing in the same cache unit.

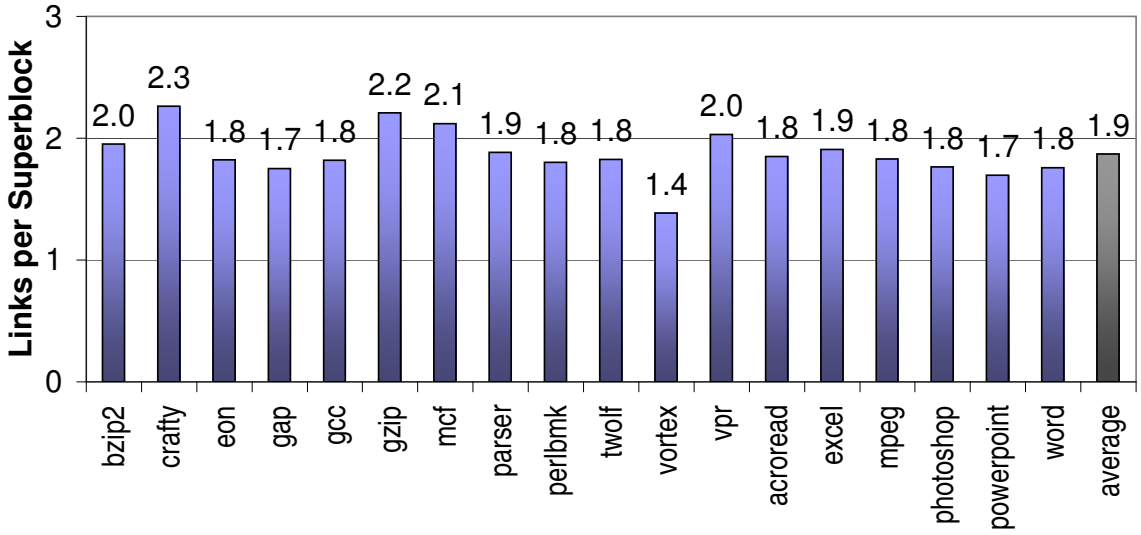


Figure 4.8: Average number of outbound links originating from each superblock.

However, there are two options when dealing with *inter-unit superblock links*—those that span cache unit boundaries. We can provide a back-pointer table exclusively for those inter-unit links, or we can eliminate inter-unit chaining from the code cache management system altogether. The two link types were illustrated in Figure 4.3.

4.2.5 Characteristics of Superblock Links

In this section, we begin by investigating the feasibility of eliminating inter-unit links (links that span the boundaries of a cache unit and therefore must be removed when the unit is flushed.) Intuitively, there are two scenarios where it would be feasible to eliminate inter-unit superblock links. The first situation would occur if superblock chaining was not highly beneficial to run-time overhead. In this case, the elimination of the back-pointer table and its subsequent maintenance overhead would outweigh the benefits of chaining. The second situation would occur if there turned out to be a very small number of inter-unit superblock links, such that even if the benefits of chaining were high, the case of inter-unit superblock chaining was so rare that it made sense to eliminate the memory and run-time overhead of maintaining back-pointer tables.

We begin by providing perspective on the memory footprint required to support back-pointer tables. We collected the average number of outbound links from each cached superblock and show the results in Figure 4.8. As the figure indicates, there are an average of 1.9 links originating from each superblock. Combining this with

Benchmark	Linking Enabled	Linking Disabled	Slowdown
gzip	230 sec	7951 sec	3357%
vpr	333 sec	2474 sec	643%
gcc	206 sec	3284 sec	1494%
mcf	368 sec	2014 sec	447%
crafty	215 sec	3547 sec	1550%
parser	350 sec	6795 sec	1841%
perlbmk	336 sec	6945 sec	1967%
gap	195 sec	4231 sec	2070%
vortex	382 sec	4655 sec	1119%
bzip2	287 sec	4294 sec	1396%
twolf	658 sec	6490 sec	886%

Table 4.1: Slowdown resulting from disabling superblock chaining. Benchmarks were executed under the control of DynamoRIO on Linux.

the observation that each back pointer requires roughly 16 bytes of memory[1], we can determine that the memory overhead of a complete back-pointer table is generally 11.5% the size of the code cache.

Next, we provide a concrete perspective on the benefits of superblock chaining. To this end, we executed the SPEC 2000 benchmarks[2] under the control of DynamoRIO with and without superblock chaining enabled on a dual-Xeon 2.4 GHz machine with 1 GB RAM. The results, shown in Table 4.1, are quite compelling. While the median execution time of `gzip` with chaining enabled was 230 seconds for the reference inputs, disabling chaining resulted in a execution time of 7,951 seconds—a dramatic 3357% slowdown. The cost is not in the hash table lookup but is caused by the memory protection changes (and associated system calls) that the DynamoRIO system does in order to protect the translation manager from the user code. In systems where this is not necessary, the slowdown is reduced, but is still significant.

A visualization of superblock links in a code cache (see Figure B.3 in Appendix B) indicates that very little spatial locality exists with regard to interconnectivity (links do not currently tend to target nearby superblocks). Therefore, although Table 4.1 showed the effects of disabling all links rather than inter-unit links in isolation, this metric can be seen as an indicator of the importance of inter-unit chaining.

At this point, the first scenario has been eliminated from the feasibility study on removing back pointers—we have shown that superblock chaining is *crucial* to run-time performance, and removing superblock chaining altogether is not an option.

[1]Back pointers are stored in a linked list. At least 16 bytes are necessary for a superblock identifier, the branch address, status flags, and a pointer to the next element in the linked list.

[2]Interactive application results were not included in this study, as we are not equipped to measure the important interactive metric—response time.

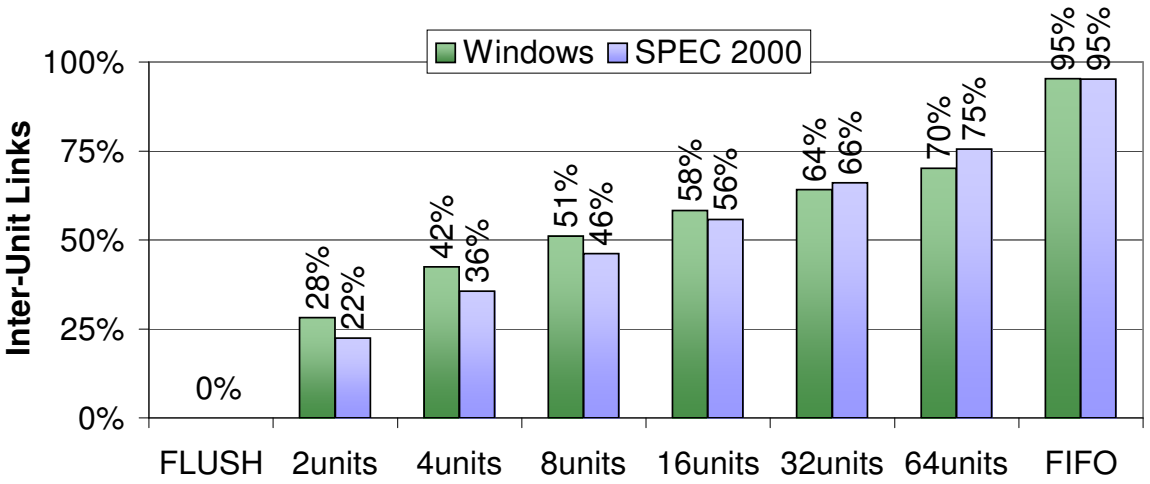


Figure 4.9: Percentage of links that target superblocks in different cache units.

4.2.6 Interconnectivity of Superblock Links

The previous section took an extreme approach and determined the impact of eliminating all superblock links from the dynamic optimizer's code cache in order to provide a perspective on the impact of superblock chaining. What is really important for the investigations, however, is to determine the impact of removing only the *inter-unit* superblock links, as the contrasting *intra-unit* superblock links will be eliminated upon flushing the unit and therefore are not problematic.

Figure 4.9 shows the percentage of links that span the cache unit boundaries. There are no inter-unit links in the `FLUSH` scheme because the entire cache is a single unit. However, as the cache is split into two separate units, a quarter of the links now span unit boundaries. On the right side of the figure, we note that although the `FIFO` mechanism places each individual superblock in a separate unit, not all links span unit boundaries because a superblock can link to itself (i.e. a loop). This now removes the second scenario from the back-pointer elimination feasibility study—it is clear that even with only two cache units, a non-trivial number of links span the unit boundary. Now that we have determined that a back-pointer table is unavoidable in all but the case of the full flush mechanism, we extend the analytical overhead study to include the overhead of maintaining a back-pointer table and removing link pointers upon cache evictions.

Using once again the methodology from Section 4.2.2, we inserted PAPI instruction count monitors around the DynamoRIO code that removes incoming link pointers to an eviction candidate. Equation 4.4 approximates the overhead of removing a given number of links pointing to an evicted superblock.

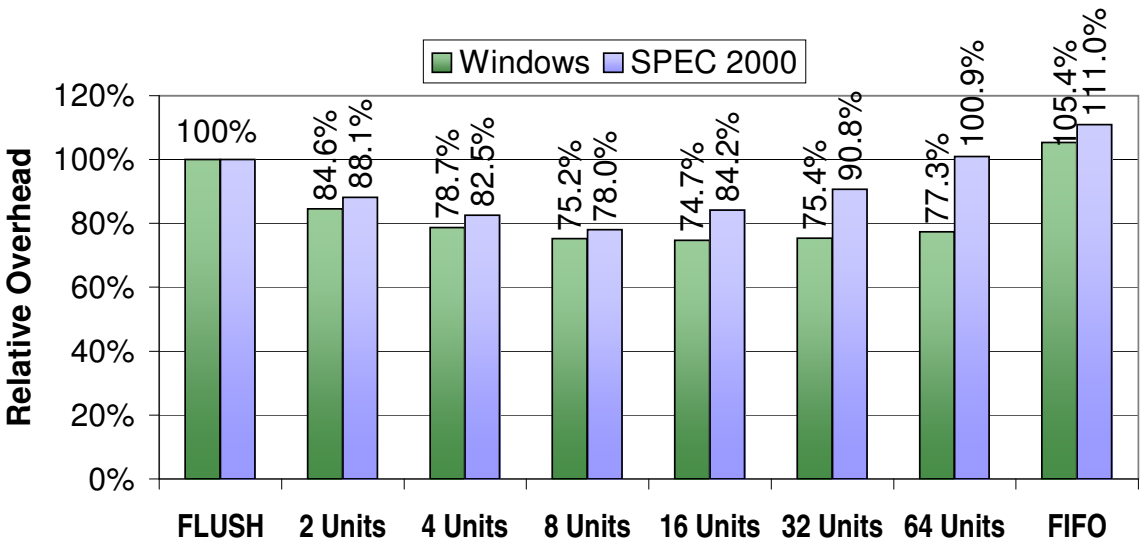


Figure 4.10: Relative overhead including cache miss penalties, eviction penalties, and superblock link maintenance. Cache size set to $maxCache/10$.

$$unlinkingOverhead = 296.5 * numLinks + 95.7 \tag{4.4}$$

We note that a large part of the overhead in Equation 4.4 can be directly attributed to the number of links that must be removed from an eviction candidate. Therefore, minimizing this number is in our best interest. Since we showed that eliminating links results in a significant decrease in performance, the best bet is to minimize the number of inter-unit superblock links that occur. As Figure 4.9 showed, this is achieved by increasing the grain size for evictions.

4.2.7 Resulting Impact on Overhead

We introduced superblock link maintenance overhead (shown in Equation 4.4) to the code cache simulator and re-executed the benchmarks. For each cache eviction, the system was penalized for every inter-unit link that required removal. This resulted in the final overhead estimates, shown in Figure 4.10. One observation from this figure is that the overheads of all of the finer-grained policies have moved closer to `FLUSH` as a result of inter-unit superblock links. This is a result of the penalty for maintaining inter-unit links that is unnecessary in `FLUSH`. The largest changes occurred in the finer-grained policies, as they contained more inter-unit links than the coarser-grained policies.

Figure 4.11 shows the performance of each granularity under increasing cache pressure. It's interesting to note that the finest-grained policy starts out performing

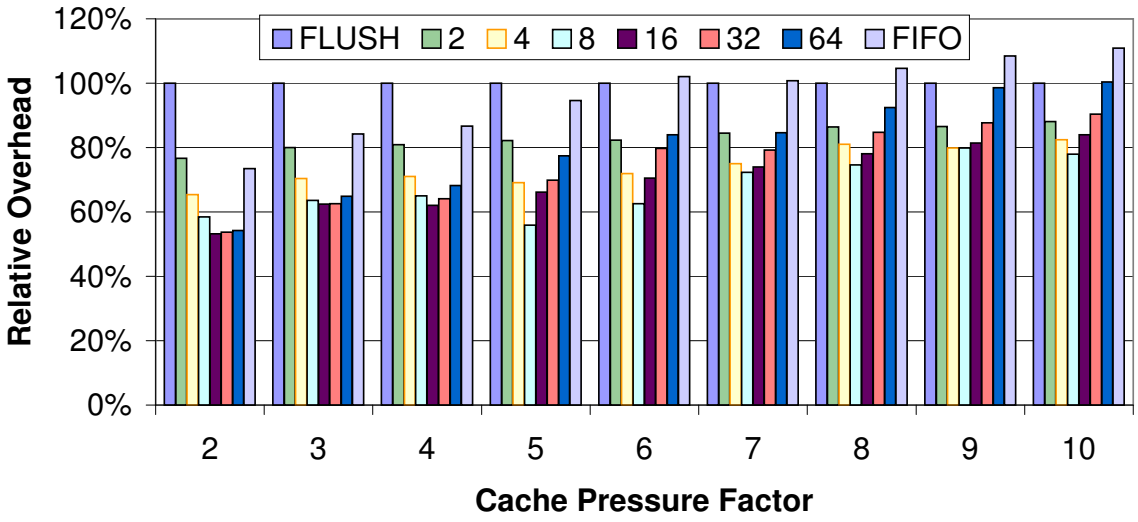


Figure 4.11: Relative overhead (including link maintenance) as cache pressure increases. Results are averages of both Windows and SPEC 2000 benchmarks.

better than `FLUSH`, but as cache pressure increases, its performance decreases until it eventually begins to perform worse. While this study took into account the performance penalties of maintaining a back-pointer table, it did not account for the memory overhead of storing the table. Increasing the size of the code cache that implements coarse-grained evictions to account for the back-pointer table savings is left for future study.

In high cache pressure circumstances, the overhead of cache management becomes a dominant factor in the overhead of a dynamic optimization system. To provide a perspective on the overhead reductions shown in Figures 4.10 and 4.11, we calculated the potential impact on final execution performance, using the calculated instruction overheads, the measured CPI, and the processor clock frequency. With a cache pressure factor of 10, benchmarks such as `crafty` and `twolf` experience a 19.33% and 19.79% reduction in overall execution time, respectively, by simply changing the eviction granularity from `FLUSH` to `8-Unit FIFO`. In practice, we expect applications that stress the cache management system to experience higher changes in execution time than others.

4.2.8 Granularity Discussion

We can conclude that a policy where the code cache manager evicts medium-grained units from the cache can outperform both the fine-grained `FIFO` eviction policy and the coarse-grained `FLUSH` policy. Compared to a fine-grained policy, the expected increase in miss rate of medium-grained evictions is offset by a reduction in eviction

invocations and reduced link pointer maintenance. This results in a smaller number of code cache management interruptions, a smaller back-pointer table for maintaining link pointers, and as the estimates suggest, improved run-time performance. These results bring us one step closer to the end goal of designing robust, scalable dynamic optimization systems by understanding the inputs and trade-offs involved in choosing a code cache eviction policy.

While the actual trade-offs may vary slightly by system implementation, we feel that the insight gained from this work will be useful for dynamic optimization system designers as they consider the options for code cache management and recognize that they should not limit their options to the extreme eviction granularities.

Currently, we advocate treating the cache as a circular buffer and inserting elements in FIFO order to leverage the temporal locality of the code cache accesses. Our future work includes a more detailed analysis of the interconnectivity of superblocks within the cache. This study will help us to determine whether a better method exists for determining the placement of superblocks into the cache units to minimize inter-unit superblock links while still achieving low miss rates.

4.3 Summary

Code cache management policies have been primarily limited to two extreme granularities. To make room for new superblocks in the cache, they evicted either the minimum necessary to make room for the new superblocks, or they flushed the entire cache. Using trace-driven simulation driven by the cache accesses and overheads of an existing dynamic optimizer, we explored several intermediate granularities of code cache evictions.

Through a combination of simulation and analytical means, we investigated the factors that make up the overhead of code cache management. We determined that the major contributor to cache eviction overhead is a fixed cost to invoke the eviction mechanism, which indicated that it is advantageous to evict larger regions from the code cache at a time. We also determined that implementation details of superblock chaining make a significant impact on the memory and run-time overhead of the implemented eviction policy. In the end, we concluded that medium-grained code cache evictions are the most scalable under pressure, ultimately resulting in a balance between cache miss rates, cache management complexity, and run-time overhead.

Chapter 5

Generational Code Caches

Now that we have an effective local cache management policy in place for a single code cache, we can begin to consider the structure and interaction policy between multiple code caches. We use the term *global code cache management* to refer to this hierarchy and policy of interaction. While the *local policies* outlined in Chapters 3–4 are necessary for determining the eviction strategy for a single cache, a *global policy* will build upon the local policy by determining when and what code regions should be relocated to a different cache during execution of a program.

Our proposal for global cache management is based on superblock generations. Section 5.1 motivates the use of generational code caches by showing that there is a clear tendency in our benchmark suites toward superblocks living either for a short time (short-lived) or for most of the program's execution (long-lived). We have borrowed where appropriate from the literature on generational garbage collection, as we will discuss in Section 5.2, and based the architecture on a simple partitioning of the existing superblock cache into two distinct and separately managed regions: a *nursery code cache* that stores all newly generated superblocks; and a *persistent code cache* that stores the long-lived superblocks. Figure 5.1 illustrates this basic architecture.

We assume that each code cache in the generational architecture adheres to the local pseudo-circular management policy described in Section 4.1. One could, of course, choose to implement a different local management policy; we leave this as a question for future studies.

As in generational garbage collection, the nursery is where superblocks stay until they have reached an appropriate age for promotion to the persistent cache. In this design, an eviction from the nursery cache indicates that a superblock has "come of age." Section 5.3 describes the technique we use to know if we should promote the evicted superblock or delete it altogether. The decision is non-trivial because, as discussed in Sections 5.2 and 5.3, we do not have sufficient information to determine whether the superblock evicted from the nursery will ever be referenced again. In other words, borrowing from the concept of liveness in compilers, we must ask if the evicted superblock is live or dead.

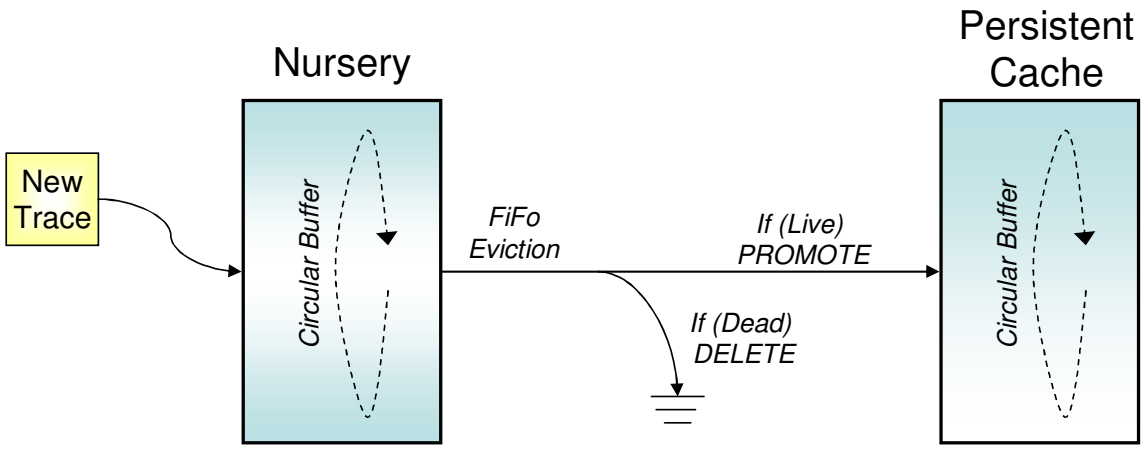


Figure 5.1: Conceptual view of the generational code cache architecture.

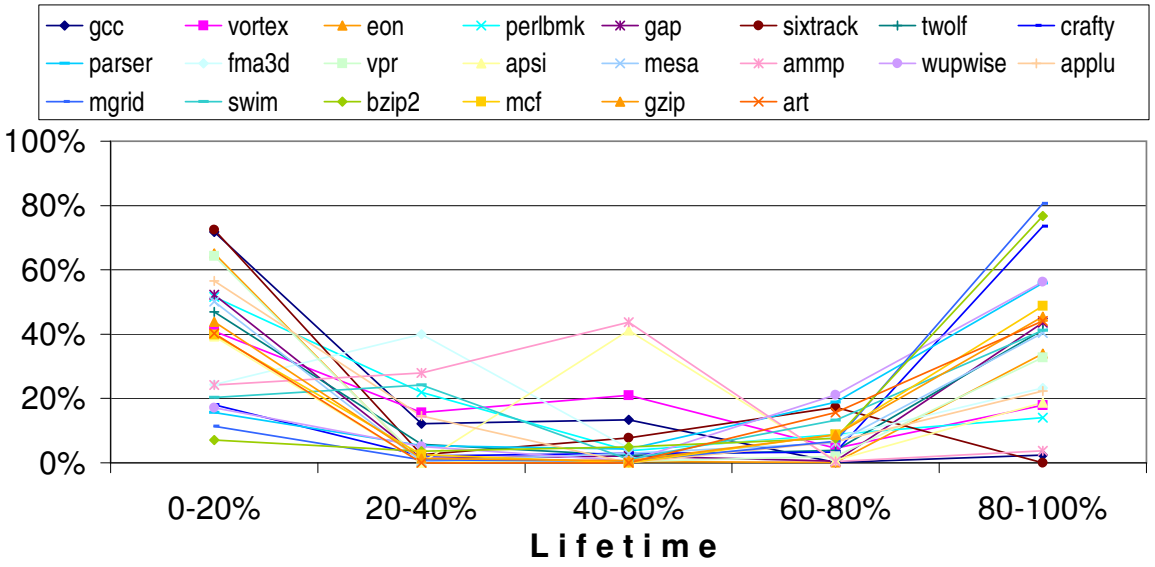
Before continuing, we note that DynamoRIO already supports multiple code caches per thread. The separation of the basic block and superblock caches (described in Section 2.1.3) can be considered a form of generational code cache management, where the execution count determines which cache stores the dynamically generated code segments. We extend this philosophy to identify storage for superblocks based on their expected lifetime. Because the superblock cache contains the frequently-executed code we apply the generational approach only to it and thus propose to move the design of dynamic optimization systems from a single unified superblock cache to multiple distinct superblock caches per thread.

5.1 Superblock Lifetimes

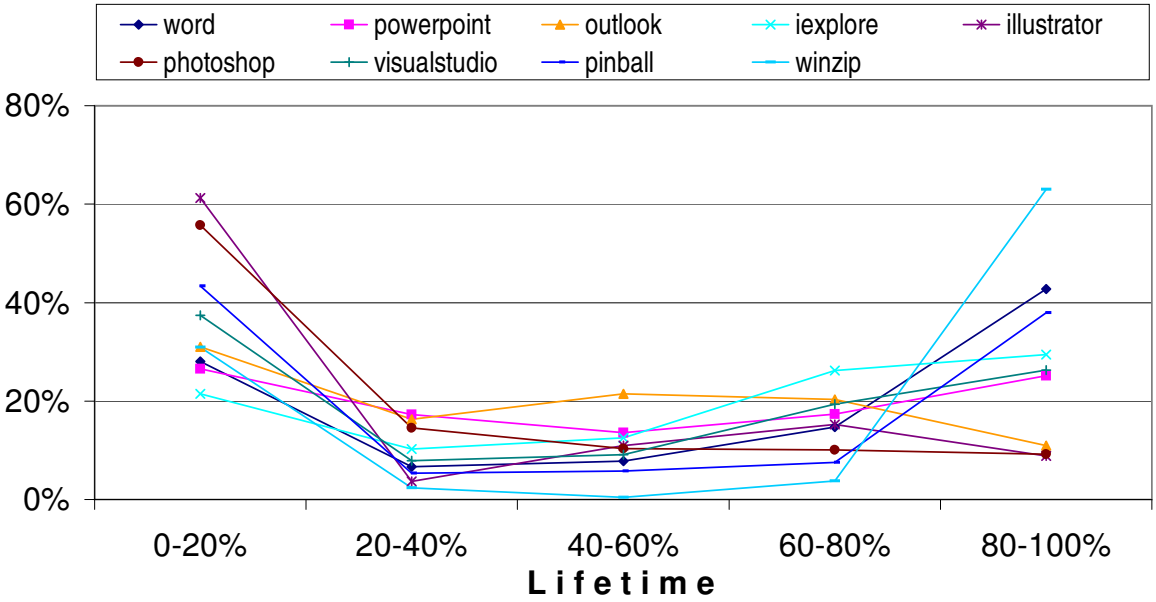
To justify the need for generational code caches, we investigate the *lifetimes* of superblocks generated during benchmark execution. A superblock lifetime is defined by Equation 5.1. This equation is intended to determine the duration of time that $superblock_i$ should reside in the code cache in the ideal case, to provide a sense for the amount of code that is short- or long-lived.

$$lifetime_i = \frac{lastExecutionTime_i - firstExecutionTime_i}{totalApplicationExecutionTime} \tag{5.1}$$

Figure 5.2 shows the typical lifetimes of superblocks calculated using Equation 5.1. The main observation from this figure is that the majority of frequently-executed superblocks fall into one of two categories: short-lived (shown as less than 20% of execution time) and long-lived (shown as greater than 80% of execution time) with a relatively small percentage of superblocks falling in the middle. The fact that so



(a) Spec2000 Lifetimes



(b) Windows Application Lifetimes

Figure 5.2: Lifetimes of superblocks as a percentage of total execution time. The vertical axis shows the unweighted (static) percentage of superblocks that fall into each lifetime category.

many of the superblocks fall into the extreme categories indicates that it may be advantageous for us to treat these superblocks differently. Furthermore, the fact that the lifetimes in Figure 5.2 exhibit a U-shaped curve implies that two separate but equally-sized caches may work well.

5.2 Parallels in Garbage Collection

The problem of managing varying-sized elements with varying lifetimes already exists in other domains. Code cache management is in many ways similar to the problem of garbage collection. One technique in garbage collection is to take a generational approach to collecting unused memory [3, 93]. Generational garbage collection makes use of the *generational hypothesis*, which states that young objects are much more likely to die than old objects, therefore young objects are typically grouped and collected together.

We have leveraged some of the advances in the area of garbage collection in our approach to the code cache management problem, but our domain differs in several respects. First, a code cache management system has the added constraint that it must immediately evict certain elements from the cache, as in the case of unmapped memory, while garbage collectors can take a passive approach to memory management. Second, garbage collectors know for certain that an object is dead when it is no longer reachable from a root node. There is no such parallel in code cache management. A code object is dead when it will no longer be executed, and this is generally not known before program termination. Finally, we can have the luxury of being incorrect in our decisions without affecting correctness, while a garbage collector must be much more conservative.

5.3 The Probation Cache

An interesting design decision arises in our generational cache architecture shown in Figure 5.1, which is the heuristic for determining whether a superblock is still alive at a given point in time, and should be promoted from the nursery to the persistent cache. We developed one such heuristic that takes into account the features and constraints outlined in Section 5.2. Our heuristic involves the use of an intermediate *probation cache* between the nursery and persistent caches, as shown in Figure 5.3.

The probation cache is similar to a *victim cache* in microarchitecture, which is used to detect and recover from untimely evictions from the instruction or data caches. The main distinction is that superblocks in the probation cache will never be reinserted into the nursery. Instead, the superblock will either be promoted to the persistent cache or deleted, as described in the algorithm in Figure 5.4.

The advantage of a probation cache, from an implementation perspective, is that it eliminates the need for profiling counters and comparisons in the larger nursery cache.

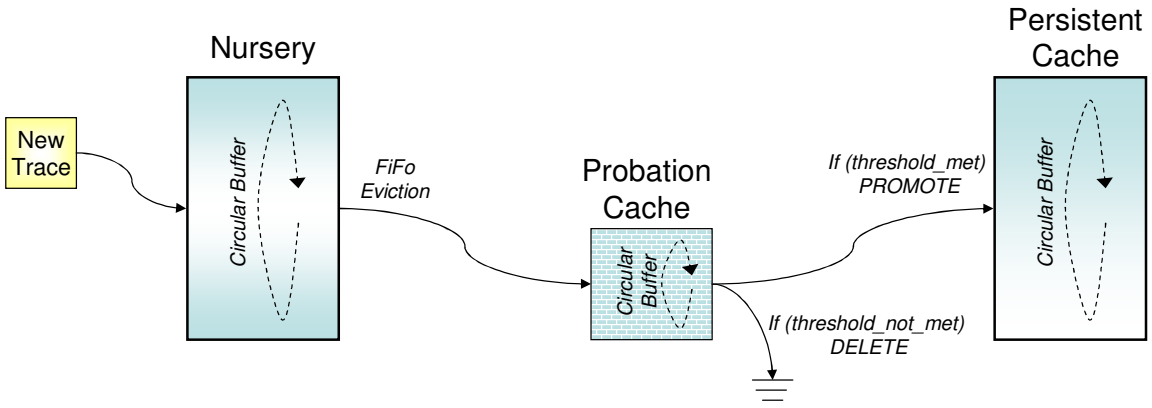


Figure 5.3: Generational code cache architecture using a probation cache.

Generational Code Cache Eviction

```
insertNewBlock {
  if (nursery_fifo points to valid superblock) {
    if (probation_fifo points to valid superblock) {
      if (probation_superblock.accessCount > threshold) {
        if (persistent_fifo points to valid superblock) {
           evict persistent superblock(s)
        }
        promote probation superblock to persistent cache
      }
      else {
        evict probation superblock(s)
      }
    }
    promote nursery superblock to probation cache
  }
  insert new superblock in nursery
}
```

Figure 5.4: An algorithm for generational code cache management with a probation cache. The `insertNewBlock` routine is called when a new block is inserted into the nursery. The `*_fifo` variables point to the next insertion point in each circular buffer.

Furthermore, it is possible to eliminate access counters in the probation cache as well by allowing each hit in the probation cache to trigger an upgrade to the persistent cache. The actual number of hits that should trigger this upgrade is experimentally determined in Section 5.5.1.

5.4 Code Relocation

In order to support the promotion of superblocks to a persistent cache, the dynamic optimizer must contain the functionality to move instructions from one address to another and provide the necessary fix-up for any address-relative jump instructions contained in the relocated instruction sequences. Fortunately, this is a basic functionality of any dynamic optimizer. By the time a superblock resides inside a superblock cache, it has already been moved from its original program location into the basic-block cache, and once again from the basic-block to the superblock cache.

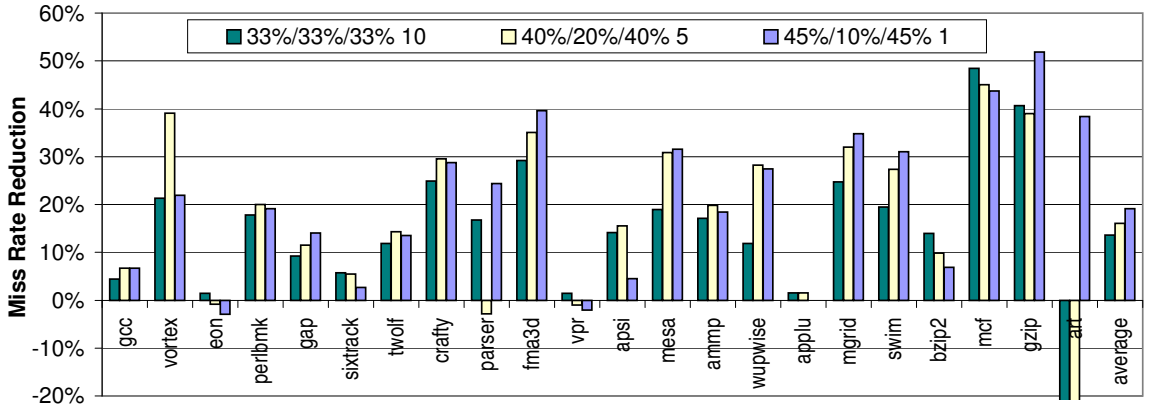
5.5 Evaluation

As in the previous chapters, we evaluated the design using DynamoRIO and a generational cache simulator. DynamoRIO executed the benchmarks using an unbounded code cache, and we used the verbose log of cache accesses to drive our cache simulator. Our baseline for comparison for each benchmark was a single pseudo-circular cache sized such that cache pressure = 3. We compared the miss rate of this single pseudo-circular cache to that of multiple pseudo-circular caches with a generational global policy and sized such that the total size of all generational caches combined (including the probation cache) equaled the size of the unified cache.

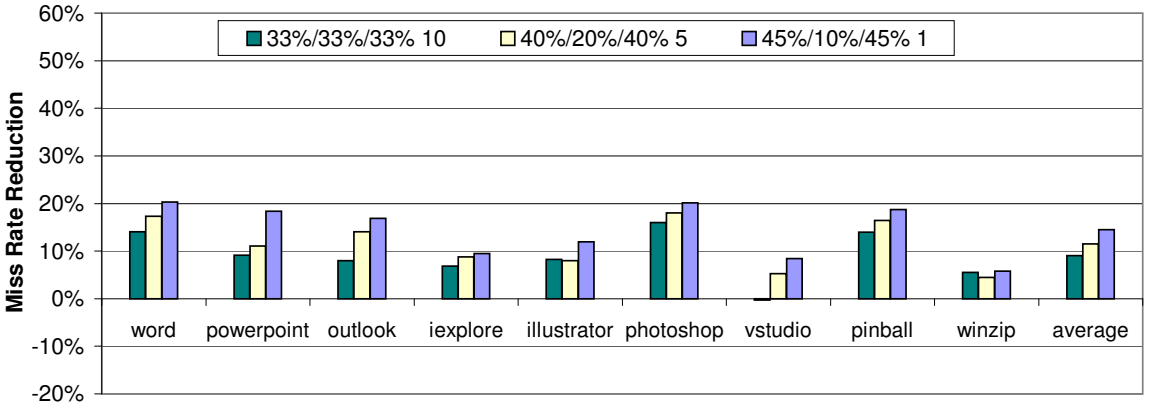
We swept the space of generational code cache sizes to determine the cache proportions that result in the lowest miss rates for each application. While the best proportions varied by benchmark, we made two general observations:

First, there was no clear advantage to an unbalanced sizing of nursery and persistent caches that held across all benchmarks. While some benchmarks such as `gcc` and `sixtrack` preferred larger nursery caches, most of the Windows benchmarks preferred balanced caches. In fact, the superblock lifetimes in Figure 5.2 turned out to be a good predictor of the best cache configuration for each benchmark. In that figure, `gcc` and `sixtrack`, had more of an L-shaped curve indicating that more of their superblocks were short-lived, therefore larger nursery caches were better suited.

Second, we noted an undeniable link between the size of the probation cache and the promotion threshold. As we decreased the size of the probation cache, we needed to lower the promotion threshold. If the threshold was set too high, long-lived superblocks did not reach the promotion threshold before eviction from the probation cache.



(a) SPEC 2000 Benchmarks



(b) Interactive Windows Benchmarks

Figure 5.5: Cache miss rate reduction of generational code caches over a unified cache. The three different bars are the proportions allocated to the generational caches. For example, on the left are results for a generational caching scheme where 33% of the cache real estate is allotted to the nursery, 33% is allotted to the probation cache, 33% to the persistent cache, and evicted superblocks with a minimum of 10 executions are promoted from the probation to the persistent cache. Average values are the unweighted arithmetic mean across all benchmarks.

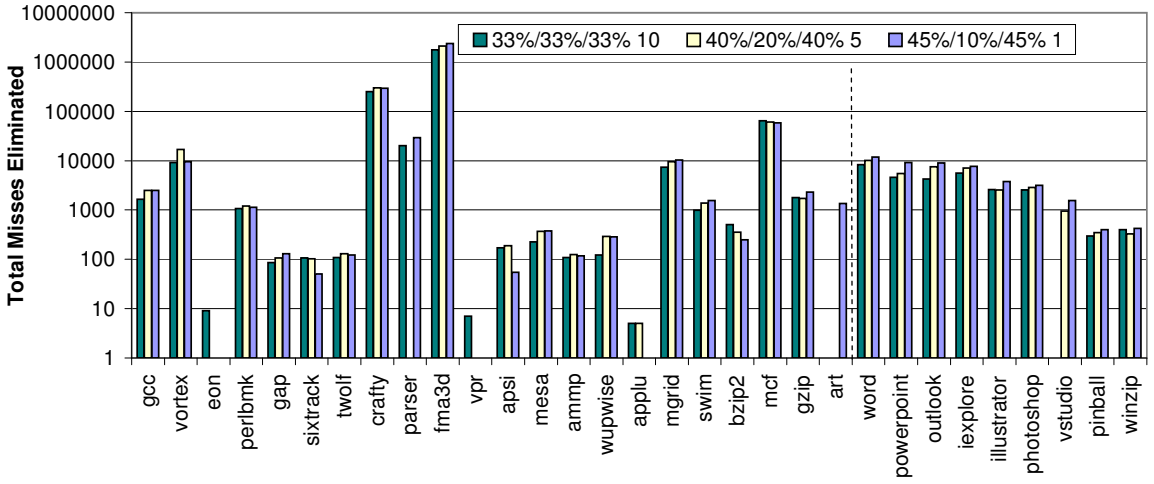


Figure 5.6: Total number of cache misses eliminated using generational code caches, as compared to a single unified cache. Note the logarithmic vertical axis.

5.5.1 Code Cache Miss Rates

Figure 5.5 compares the miss rates of three instances of generational cache layouts to that of a single unified code cache. While the best cache configuration varied by benchmark, a generational cache configuration with a 45%–10%–45% size ratio between the nursery, probation, and persistent cache, respectively, performs best overall. Furthermore, this cache configuration performed best when we applied the policy where a single hit in the probation cache triggered a promotion. This phenomenon is particularly interesting because it simplifies the implementation and obviates the need for complex analysis to determine whether to promote a superblock to the persistent cache.

An outlier in Figure 5.5 was `art`, which is also the smallest benchmark in terms of bytes of superblocks produced at run time. Since we purposely sized the caches so as not to fit the entire code footprint, this will have a greater impact on small programs that are characterized by execution within a small loop body. Interestingly, it is these very benchmarks for which cache management is least critical, because there is less of a need to apply cache management to them.

Finally, to provide perspective to the resulting miss rates, we present the actual *number* of eliminated misses in Figure 5.6. From this figure, we see that the miss rate reductions were often the result of eliminating many thousand code cache misses in the benchmarks.

Description	**Overhead (instructions)**
Superblock Generation	$75.4 * blockSizeBytes + 1922$
Superblock Eviction	$2.77 * blockSizeBytes + 3055$
Link Repair	$296.5 * numLinks + 95.7$
Superblock Promotions	$13 * blockSizeBytes + 5028$
DynamoRIO Context Switch	25

Table 5.1: Overheads used in the evaluation of generational code caches.

5.5.2 Resulting Performance

The importance of minimizing code cache miss rates can only be appreciated when we contemplate the cost of a single code cache miss. Implementors of effective dynamic optimizers have gone to great lengths to keep execution in the code cache. All of this is done to avoid the high overhead of instruction interpretation, code generation and superblock optimization. Conflict misses in the code cache require us to repeat these expensive steps unnecessarily.

We used the Pentium-4 performance monitors to get a realistic sense of the overhead of superblock generation, evictions, promotions, and context switches within the DynamoRIO framework. Using the PAPI interface and the Pentium instruction counters, we read the instruction counts before and after key events. We plotted these measurements against superblock sizes where applicable, and generated the best-fit formulas listed in Table 5.1 using a linear regression as described in Appendix B.

A single conflict miss in the superblock cache results in two DynamoRIO context switches, superblock regeneration, and link repair. Therefore, while the generational cache management scheme includes the additional overhead of promoting superblocks to the probation and persistent caches, this carries far less overhead than premature eviction and superblock regeneration.

We compared the overhead of superblock generation, context switches, evictions and promotions in a unified code cache to that of a generational code cache. In Figure 5.7 we show the overhead ratio between generational caches and a unified cache, calculated using Equation 5.2.

$$overheadRatio = \frac{generationalCacheOverhead}{unifiedCacheOverhead} \tag{5.2}$$

Figure 5.7 shows that the average (geometric mean) overhead ratio is 80.7%, which represents a 19.3% reduction of instructions needed to service code cache misses. Specific benchmarks varied from a 51.1% ratio for `gzip` to a 106.2% for `applu`. Three of the benchmarks—`eon`, `vpr`, and `applu`—resulted in increases in overhead because the code cache miss rate reductions were not large enough to offset the overhead of promotion between the generational caches. These three benchmarks performed better with a larger probation cache, as shown in Figure 5.5a. This result shows that

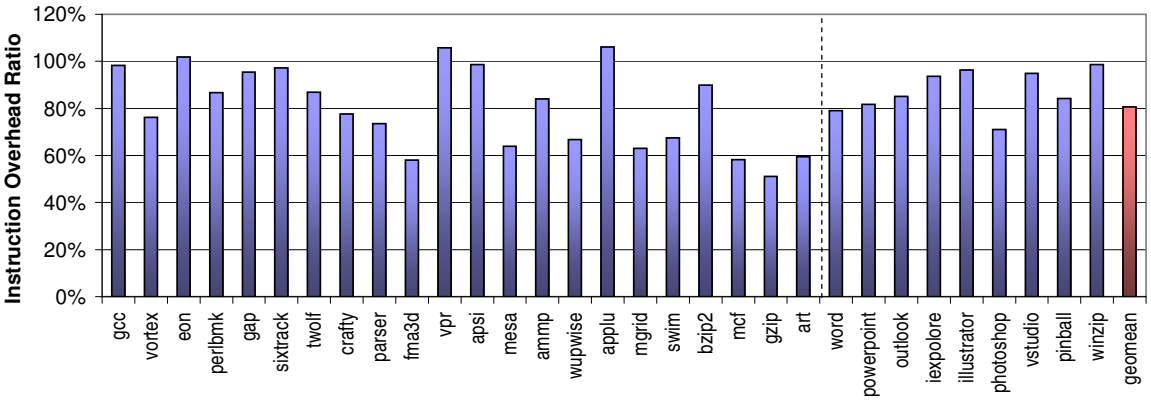


Figure 5.7: Instruction overhead ratio of generational code caches to a unified cache. Values below 100% represent a reduction in overhead, therefore smaller is better. The generational caches have 45%–10%–45% proportions between the nursery, probation, and persistent caches.

further exploration of the configuration space may improve overall performance of the benchmarks. Finally, we were encouraged by the fact that overhead reductions were seen in all of the large Windows benchmarks in Figure 5.7.

To provide a perspective on the overhead reductions, we calculated the impact on final execution performance and found that it was highly dependent on the actual number of mispredictions eliminated (which was shown in Figure 5.6). For `gzip` where generational cache management eliminated 2,288 misses, this resulted in an estimated 0.07% reduction in overall execution cycles. For `crafty` where 292,486 misses were eliminated, an 8.09% reduction in execution cycles is expected.

5.6 Lifetime versus Access Frequency

One natural question that arises is whether the *important* superblocks are being maintained in the persistent cache, or stated another way, how good is *age* as a predictor for *access frequency*. Our notion of the importance of a superblock's lifetime would fail if there existed code that was executed at the beginning of a program run, then again at the end of a program run, but rarely in between. Such superblocks would be marked as *old* and would be promoted to the persistent cache even though they weren't in the working set of the application.

In reality, this doesn't occur. Figures B.4–B.5 in Appendix B show that superblocks with long lifetimes are also steadily executed throughout the run-time of an application. Intuitively, this makes sense. We have a notion of *start-up code* in programs, and we have a notion of *termination code*, but rarely, if ever, are these the same pieces of code. Furthermore, even if they were the same code, the corresponding

superblocks would never pass the threshold test for deciding whether it was alive or dead, therefore it would never be promoted from the probation cache.

5.7 Discussion

In this chapter, a new generational code cache management algorithm was proposed and evaluated. The algorithm categorizes superblocks based on their expected lifetimes and groups superblocks with similar lifetimes together in separate storage areas. This allows short-lived superblocks to easily be removed from a code cache without introducing fragmentation and without suffering the performance penalties associated with evicting long-lived code superblocks. Using trace-driven simulation, generational code caches were determined to result in an average miss rate reduction of 18% over a unified cache, which translates into 19% fewer instructions spent in the dynamic optimizer.

Superblock lifetimes turned out to be an effective guide for determining the best proportions of generational code caches. Applications with a higher percentage of short-lived code preferred larger nursery caches. For a single configuration that works well across all benchmarks, balanced caches are recommended. While it would be interesting develop heuristics for recognizing superblock lifetimes during execution and adjusting code cache proportions on the fly, we leave this as an avenue for future work.

A natural extension to the notion of a generational persistent cache is a truly persistent cache that doesn't simply store superblocks for an entire program execution, but actually stores superblocks between program executions. Appendix A describes some initial work in this direction.

Chapter 6

Implementation Details and Challenges

In order to gain a better understanding of the implementation complexities that must be overcome in order to support sophisticated code cache management algorithms, we implemented the generational code cache algorithm discussed in Chapter 5 in the DynamoRIO dynamic optimization infrastructure [15]. We also used this implementation to validate our simulation results.

6.1 Challenges

As mentioned in Chapter 5, generational code caches were designed as a means for managing DynamoRIO's superblock cache. In building support for generational superblock caches, we made several modifications to the existing DynamoRIO system. In this chapter, we describe the challenges we encountered and the modifications we made. Though the following descriptions make particular reference to DynamoRIO and generational code cache management, many of the issues and complexities are pertinent to any code cache management scheme that moves code items between or within code caches.[1]

6.1.1 Support for Relocatable Code

A basic function in a generational code cache management system is the movement of superblocks from one cache to another. To support this functionality and make it as efficient as possible, superblocks should be created that are conducive to relocation and require as few relocation "fix-ups" as possible. Of course, the task of relocating

[1]For example, the issues described in Sections 6.1.1 and 6.1.2 are relevant to any cache management scheme that moves superblocks in order to compact the storage and remove fragmentation.

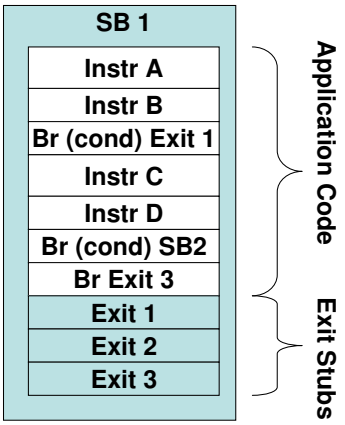


Figure 6.1: A superblock with exit stubs. The first and third branches target the exit stubs, while the second branch is linked to another superblock.

a block of code is simplified when the code uses PC-relative branches. As a result, dynamic optimization systems should use relative branches for all intra-block jumps.

In DynamoRIO, the unit of relocation between code caches is a superblock with its associated exit stubs. Exit stubs (depicted in Figure 6.1) are a mechanism for returning control to DynamoRIO when the target of a branch in the superblock is a piece of code not currently stored in one of the system's code caches.

We can use PC-relative branches and avoid relocation costs for any branches internal to the superblock that target that superblock's entry point or its exit stubs. Unfortunately, branches within the superblock that have been linked to jump directly to other superblocks will have to be adjusted when the superblock is relocated. We discuss this topic further in the next section.

Finally, we can use direct (non-PC-relative) branches in the exit stubs to avoid relocation costs because the target of all exit stubs is DynamoRIO's interpreter, which remains at a fixed address for the entire execution of a dynamically optimized application.

6.1.2 Efficient Link Repair

One of the major performance boosts in a dynamic optimizer comes from chaining within the code cache. Chaining occurs when a branch in one cached superblock is directly linked to the entry point of another cached superblock. This allows execution to stay within the code cache—removing the need to return to the dynamic system and enabling improved instruction-cache performance on the underlying processor.

Though superblock chaining improves the performance of the transformed program's execution while in the code cache, it also causes overhead for the code cache management system. Except in the simplest of code cache management schemes (i.e.,

those that flush the entire code cache), superblock chaining requires code cache management schemes to ensure that no dangling links are created on eviction of one or more superblocks. When a superblock is evicted from the code cache, all outgoing links to other code are automatically eliminated with the purged superblock. However, incoming links to the evicted element (i.e. linked branches originating from other superblocks) must be rewritten to jump to their associated exit stubs. If not fixed, the program's execution could mistakenly branch to a now-invalid region of memory.

In order to support evictions from the code cache, most systems maintain a back-pointer table, which lists all incoming links to a superblock. Using this table, the eviction mechanism in the code cache manager can quickly locate and remove all incoming links before evicting a superblock.

Not surprisingly, we would expect that relocating superblocks from one code cache to another would incur a similar but slightly larger overhead due to chaining as evicting superblocks. Both actions require the management system to adjust all incoming link branches.[2] Unlike eviction however, relocation also requires us to repair the outgoing links of the relocated superblock, since they are implemented as PC-relative branches.

Complicating the matter of repairing the outgoing links is the fact that cached code has already been optimized and translated, and currently exists only in binary form. Decoding the entire relocated superblock to determine its outgoing link locations is impractical at run time, particularly in IA-32 with its variable-length encoding. Therefore we found great utility in maintaining data structures that represent the outgoing links within each superblock. The data structure stores pertinent information such as the addresses of the branch instructions (stored as offsets from the start address), the target of the branch, the branch type (direct, indirect), and the state of the link (linked, unlinked). Each superblock thus contains incoming and outgoing link data structures, and the identical link structure on the outgoing list of one superblock will be accessible on the incoming list of the destination superblock. This allows all pertinent links to be repaired by walking the lists of the promoted superblock, and obviates the need for any expensive decoding during link repair.

6.1.3 Hash Tables

An important design decision that we made early in the implementation process is that we would not allow superblocks to co-exist in different caches. While permitting co-existing superblocks would enable us to maintain several custom versions of superblocks for different *phases* of program behavior, we felt that the complexity of

[2]Though the cost of repair is the same, the manner in which we adjust the PC-relative offset differs between relocation and eviction. In relocation, we adjust the link branch to point to the superblock's new location. In eviction, we change the link branch to point back to its associated exit stub.

generating multiple versions of the code at run time, determining the current phase, and relinking all superblocks during a phase change combined with the added cache contention was unlikely to result in a run-time benefit. We leave this question as an area of future study.

The fact that we have imposed the rule that superblocks may only exist in one of the code caches simplifies the design of a hash-table lookup. We maintain a single hash table which is used to determine whether the code at a particular application program counter exists in the code cache. Now, although we have increased the number of hot superblock caches from one unified cache to three separately-maintained caches, we still require a single lookup to determine a cache hit. And the fact that the hash table furthermore hashes code based on the original program counter, which doesn't change despite a code cache relocation, reduces the maintenance that is required in the table.

6.1.4 Profiling Insertion and Removal

A final crucial element that must exist in a cache management system to support generational code caches is profiling functionality. In the discussion of generational code caches in Chapter 5, we suggested inserting profile counters in the superblocks of the probation cache, and only promoting elements to the persistent cache if their counters exceeded a *promotion threshold*. It is important to recall that profiling occurs only in the probation cache, and not in the nursery or persistent caches.

During implementation, we learned that altering the superblocks themselves to insert profiling counters requires decoding the entire superblock to determine register and processor flag usage, insertion of new instructions, and re-encoding. Furthermore this process would need to be repeated to remove the counters if the superblock were subsequently promoted to the persistent cache. Since this prospect appeared expensive, we decided to use a method similar to that used in DynamoRIO's basic block cache to determine the *hot* blocks that should be converted to superblocks.

The simple profiling technique is shown in Figure 6.2. Instead of linking directly to a target superblock in the probation cache, we instead link the source superblock to a single hand-coded assembly routine. We pass a pointer to the link data structure described in Section 6.1.2 to the assembly routine, which then increments the counter associated with the target of the link, and branches to the target superblock. This technique is very similar to the technique used to support cross-segment procedure calls in the Multics operating system [28].

Using this technique, inserting and removing profiling counters is simply a matter of updating the incoming links to a superblock. In Chapter 5, we learned that profiling was only necessary for elements in the probation cache. This turns out to be beneficial for implementation purposes, because it allows us to link and unlink the profiling counters during the link repair phase that occurs after superblock relocation.

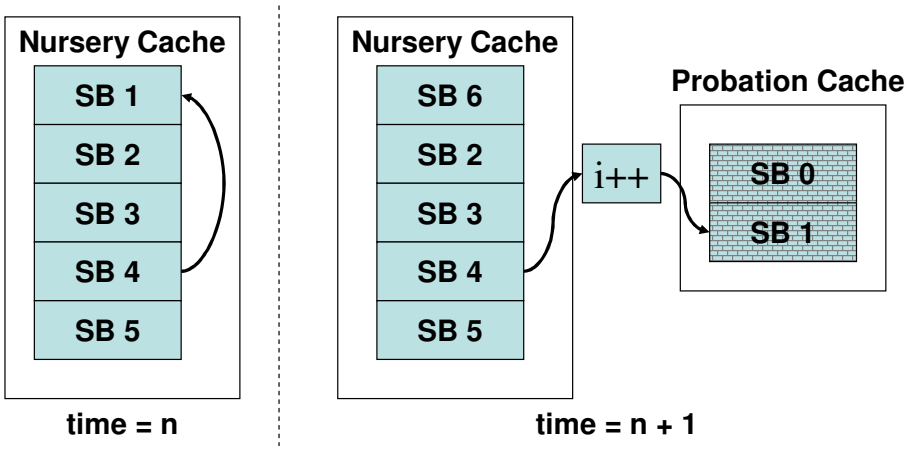


Figure 6.2: The insertion of superblock 6 into the nursery forces a promotion of superblock 1 from nursery cache to probation cache. Profiling counters are inserted in the process.

6.2 Performance Results

We implemented generational code caches in the DynamoRIO dynamic optimization system following the design directions discussed in Section 5.2. We varied the size proportions of the three generational caches such that the sum of the sizes of the three caches were equal to the size of the unified cache to which they were compared. We also varied profile counter thresholds that were used to determine whether to promote a probation block to the persistent cache or to evict it altogether.

In this section, we report the run time performance of each cache configuration. Run time performance values were gathered by executing the entire set of SPEC 2000 benchmarks (with reference inputs) under the direct control of DynamoRIO using the official SPEC 2000 `runspec` script. The `runspec` script invokes each benchmark execution, verifies the validity of the output, and reports the official wall-clock performance results.

A fixed cache size was used for all benchmarks. This choice differs from the approach taken earlier, where code cache pressure was guaranteed by sizing the caches to be some fraction of the cache size required to avoid cache management for each benchmark. In this chapter, we evaluated the result of fixing the cache size for all benchmarks to get a better sense of the cache configurations that work well for many applications. In a real implementation, it is impossible to know *a priori* the maximum cache size that will be reached during execution, thus it will eventually be necessary to design for the common case.

We tested various configurations of generational cache sizes and compared them to a single, unified cache sized at 600 KB[3]. Figure 6.3 reports the run time performance

[3]We chose 600 KB because it allowed us to experiment with many cache proportion configurations

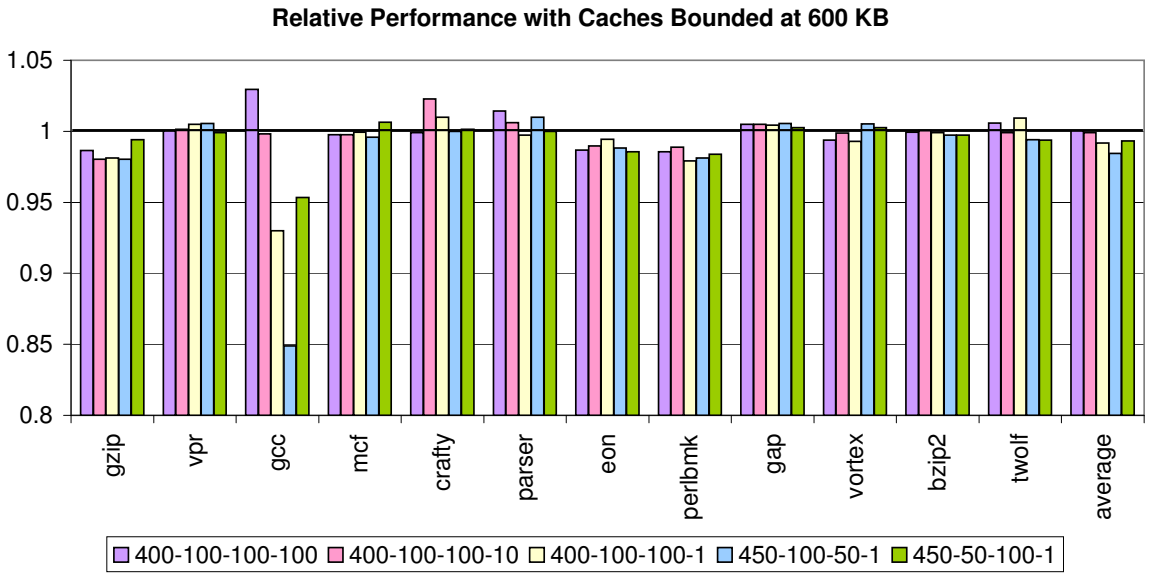


Figure 6.3: Runtime performance of generational code caches for various configurations compared to the baseline performance of a unified cache sized at 600 KB. Lower numbers indicate a performance improvement using generational caches. The configuration 450-100-50-10 represents three generational code caches where the nursery cache is 450 KB, the probation cache is 100 KB, the persistent cache is 50 KB, and the threshold for promoting to the persistent cache is 10 executions.

results of five of the best-performing configurations of those investigated. As the figure indicates, configurations with relatively large nursery caches appear to perform better overall. The best configuration turned out to be one that exhibits a 450–100–50 size ratio between the nursery, probation and persistent caches, respectively.

While Chapter 5 reported that configurations that balance the nursery and persistent caches perform best, we learned that this is only true when cache pressure exists. When cache pressure is not present, as in the case of small benchmarks, the overhead of performing cache promotion was not offset by a reduced cache miss rate. This results in a net loss of performance for certain small benchmarks. In Figure 6.3 where the cache is sized at 600 KB, performance improvements are more evident for the `gcc` benchmark, which not coincidentally is the only benchmark that is significantly stressed at this cache size.

More interesting is the cache configuration that performed best of those tested. If we look at the breakdown of hits for the `gcc` benchmark (shown in Figure 6.4), we see that the configurations where more hits occurred in the nursery cache performed better. Now, if we recall the lifetime study of Chapter 5, we actually see that most of the superblocks in the `gcc` benchmark were actually short-lived, so it makes perfect

without hitting the lower bounds on cache sizes imposed by DynamoRIO.

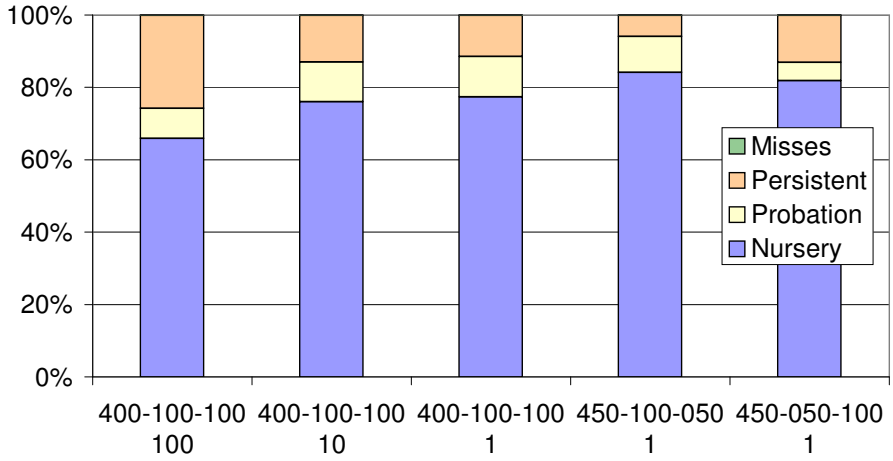


Figure 6.4: Breakdown of generational cache accesses for the `gcc` benchmark with the total cache size set to 600 KB. Misses comprise less than 1% of cache accesses.

sense that a larger nursery would perform best.

This result validates our earlier observation from Chapter 5 where we found superblock lifetimes to be a good predictor of the best cache-size configurations. This result also shows how performing this study with only the SPEC 2000 benchmarks leads us to the wrong conclusion. The SPEC 2000 performance is dominated by the single `gcc` benchmark, which is rare in that it has a large number of short-lived superblocks.

Now that we have an understanding of the final performance of a generational code cache implementation, we can investigate the factors that contribute to the overhead to get a sense of the potential benefits of careful tuning of the implementation.

6.3 Initial Cache Setup Overhead

One interesting metric is the static overhead of generational code cache management. The static overhead includes only the costs of allocating and initializing the three generational caches. Our measure of static overhead does not include the costs of inserting, promoting, or evicting superblocks, which we will see are the dominant costs of cache management. Our static overhead measurement is important for verifying that generational caches do not hinder the performance of small applications. Small applications do not require large code caches and therefore do not benefit from sophisticated code cache management schemes.

Figure 6.5 shows the relative performance of unbounded generational code caches over a single (unbounded) unified cache. The values presented in the figure are those reported by the official SPEC 2000 `runspec` script executing the reference input set to completion.

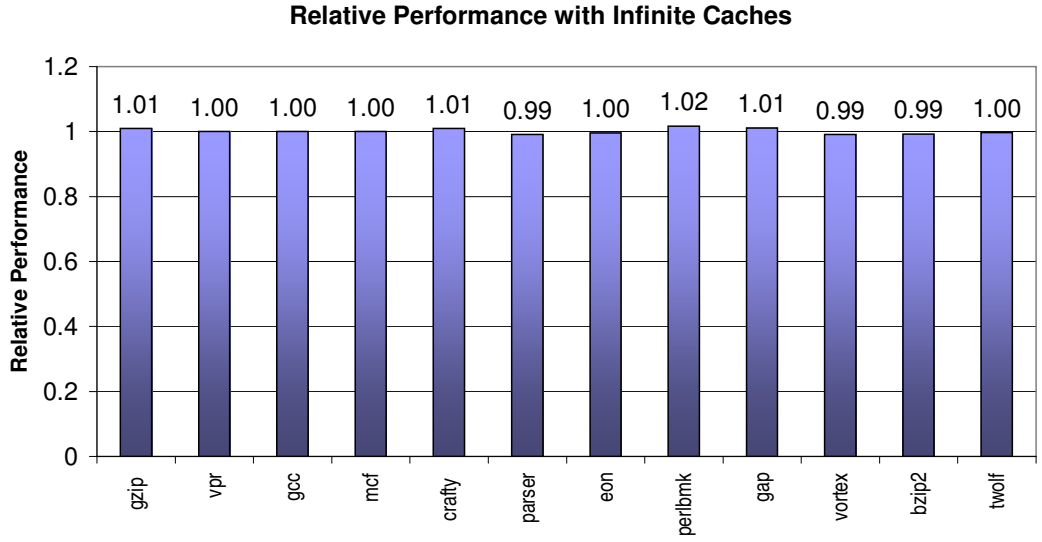


Figure 6.5: Relative performance of generational code caches with an infinite cache. No cache replacements occur. The only cost differences are the start-up overheads.

Again, we emphasize that no cache management related to evictions or promotions occurs in these runs; both sets of experiments simply create and insert a sequence of superblocks into an unbounded code cache. The results illustrated in Figure 6.5 therefore represent the additional static overhead of creating multiple superblock caches. As shown in the figure, implementing generational code caches incurs no noticeable overhead on programs that do not require code cache management due to a lack of contention for the code cache space.

6.4 Superblock Promotion Overhead

The next few sections focus on the overhead experienced when a superblock is promoted from one cache to another. The breakdown is shown in Figure 6.6. These breakdowns are an average of several thousand different superblock promotions.

6.4.1 Reserving Code Cache Space

We start in this section by looking at the cost of reserving space in the code cache. This overhead categorization includes determining the number of elements that must be evicted to make room for the element entering the cache, evicting or promoting other elements, and updating FIFO replacement list in the new cache. As we can see in the "Reserve Space" portion of the pie chart in Figure 6.6, these tasks account for (on average) 26% of the instructions executed in order to promote a superblock. A look at Table 6.1 furthermore shows that this overhead generally amounts to 869

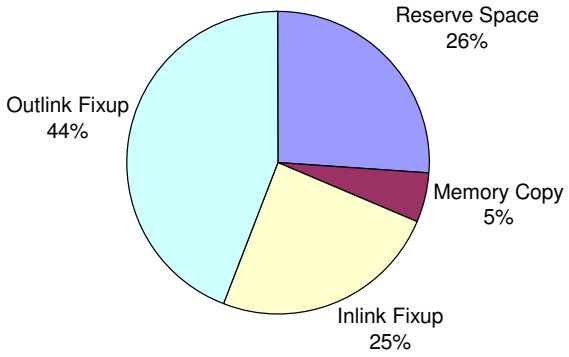


Figure 6.6: Breakdown of the overhead of promoting a superblock from one generational code cache to another, shown as a percentage of assembly instructions executed.

assembly-level instructions, or just under 3,000 clock cycles (per occurrence) on a Pentium-4 processor.

6.4.2 Memory Copy Overhead

After reserving space in the new cache, a memory copy is performed to move the superblock to the new cache. Interestingly, we see from Figure 6.6 that copying the superblock amounts to only 5% of the overhead of the entire relocation process. Regardless of the wide range of superblock sizes, the variance of memory copy overhead was very low. The number of assembly instructions required to perform the memory copy was often identical, and the CPU cycles to perform the copy varied only slightly.

An interesting point to note is that of all the overheads associated with relocating superblocks, the memory copy overhead is the single overhead that is at or close to its lower bound. All of the other overheads are subject to implementation inefficiencies. The fact that the memory copy overhead accounts for such a small percentage of the overhead of relocating superblocks is encouraging, because it implies that implementors can potentially benefit from a great deal of fine-tuning of the other details.

6.4.3 Link Repair Overhead

One interesting observation from the overhead analysis is the significant impact of link repair. This process accounts for the majority of the overhead encountered during superblock relocation. It is important to point out, however, that this value is implementation specific. The complexity of repairing inter-element links is closely tied to the support available for doing so, such as the techniques described in Section 6.1.2. Since DynamoRIO had little to no support for efficiently repairing outlinks, our extensions were not as streamlined as they could have been. We expect that this overhead can be reduced significantly with additional tuning.

Task	Instructions	Cycles
Reserve Space	869	2782
Memory Copy	173	173
Incoming Link Repair	886	2707
Outgoing Link Repair	1543	4009

Table 6.1: Average run-time overhead of various tasks required to support generational code cache management.

6.4.4 Overhead Summary

While Figure 6.6 showed the relative overhead of performing the various tasks necessary for supporting generational code caches, Table 6.1 quantifies the overheads. The values presented in the table are the average instructions and average cycles reported by the PAPI interface to the Pentium performance counters for the regions of the code that perform the indicated actions. As the table illustrates, an average of 3471 (assembly) instructions are required to perform an actual superblock promotion. Except for the memory copy work, the average cycles-per-instruction for each of the other tasks is around 3. There are clear opportunities for improvement here.

6.5 Validating Simulated Miss Rates

In this section, we validate the miss rates gathered in Chapter 5 by collecting the actual miss rates of the generational code cache implementation in DynamoRIO. We are able to gather the miss rates by disabling superblock chaining within the code cache. This forces control to return to DynamoRIO after every superblock execution and enables us to log the outcome of every cache access.

Figure 6.7 shows a comparison of simulated and actual miss rates for the `gcc` benchmark. While the resulting miss rates are fairly comparable, there are several explanations for the differences between simulated and actual miss rates.

The main reason is that DynamoRIO actually forms different superblocks when the cache size changes, due to a parameter that ensures a formed superblock will fit in the cache. In our simulations, we used a trace of superblocks that was fixed regardless of the cache size. Limiting a single superblock's size will result in an entirely different series of superblocks to appear in the code cache. Furthermore, it is not always the case that limiting a superblock's size will hurt performance. If that superblock turns out to be rarely executed, the overhead of superblock formation will not be amortized. In addition, more superblocks can fit in the code cache if individual superblocks are smaller. These variables combine to cause our simulator to overestimate the miss rate in some instances (the 400-100-100-100 configuration) and underestimate in others.

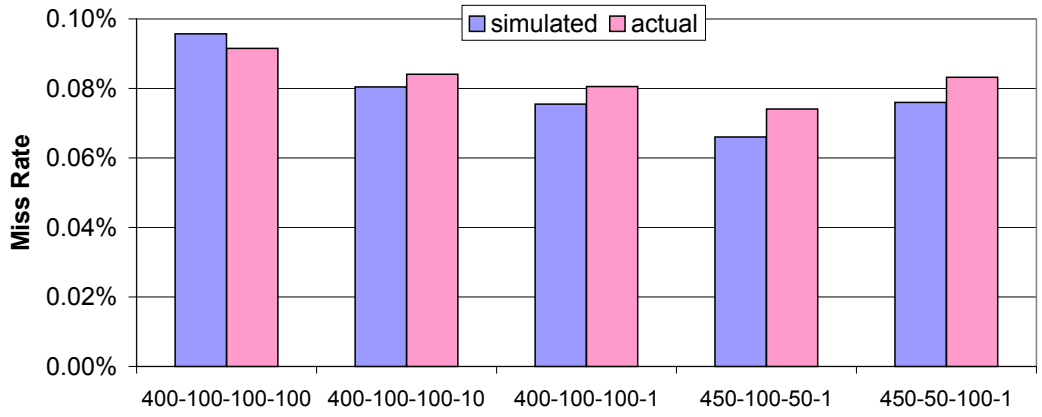


Figure 6.7: This graph compares the simulated and actual miss rates of `gcc` for several configurations of generational code caches.

6.6 Discussion

We described an actual implementation of generational code cache management in DynamoRIO, a mature dynamic optimization system. Using this implementation, we measured the performance advantage of generational code caching over the less sophisticated circular FIFO approach found in DynamoRIO. We also analyzed the overhead of supporting generational code caches and characterized what these insights might mean for other code cache management schemes.

The breakdown of the run-time performance overhead of generational code caches in a dynamic optimization system presents three major contributions to the dynamic optimization, dynamic binary translation, and JIT compilation communities. First, the analysis of the run-time overhead contributors provides significant insights to designers of code cache management algorithms. Algorithms that reduce the *important* overheads are always desired, but until now, it was not clear which overheads were important from a run-time performance perspective. Second, for caching algorithm implementors, overhead analysis provides an indication of which functionalities can be fine-tuned to squeeze out more run-time performance. Finally, the run-time overhead averages presented here can be used to improve the predictive accuracy of future efforts in trace-driven code cache simulation.

Chapter 7

Systems Using Code Caching

In this section, we will review several of the existing software systems for dynamic optimization, dynamic binary translation, and just-in-time compilation. We will focus on the existing solutions to code caching as implemented in these systems to provide a perspective on the impact of the work in this dissertation. We will also review how this dissertation relates to the large body of work in adjacent areas, such as instruction caching in microprocessors or web caching in computer networks.

7.1 Transparent Optimization

A large set of dynamic optimization systems are designed to be completely transparent to the software, the hardware, as well as the user of the system. These systems are characterized by a native optimization engine that is designed to tailor the application to its run-time environment without any intervention or special preparation by the user or application writer.

7.1.1 Performance Optimization Focus

Dynamo [7, 8] is a system developed at Hewlett-Packard Laboratories that provides a software-based mechanism for selecting and optimizing superblocks of HP-UX instructions. Dynamo cached superblocks and employed a preemptive flushing mechanism for cache management, which detected a program phase change and flushed the entire code cache at that point. This policy was found to perform better than a naïve flush of the cache when it became full. Dynamo formed the foundation for several successor systems, including DELI and DynamoRIO that will be discussed in Section 7.1.2.

Wiggins/Redstone [31], developed at Compaq, is a dynamic specialization system that applies data-specific and machine-specific optimizations to Alpha binaries at run time. Wiggins/Redstone also cached optimized superblocks, although they were

characteristically longer (typically 300-500 instructions) than the superblocks built by Dynamo. While the system is described as one that "captures temporal effects" [31] of an application, no specific details on the code cache management policy have been published.

ADORE [74] is a research dynamic optimization system developed at the University of Minnesota. ADORE listens to the performance monitors built into the Itanium processor to guide optimization of Itanium Processor Family (IPF) applications. Frequently-executed superblocks are identified by the hardware and are optimized by the ADORE software system. ADORE limits the creation of superblocks to avoid the need to manage the code cache. While this approach of avoiding code cache management works for a system of this sort (a system that performs optional transformations on native applications) it is not a possible solution for systems that perform binary translation or add any security features to all code prior to execution.

Mojo [22], developed at Microsoft, is a dynamic optimizer that was one of the first infrastructures to specifically target large, interactive Windows applications. Mojo contained two code caches—a basic block cache and a *path cache* which resembles a superblock cache. For cache management, each cache is divided into two coarse-grained cache units. Each unit was fully flushed in an alternating order (i.e. FIFO). Unlike DynamoRIO, which has a separate superblock cache per thread, Mojo shares its path cache (or superblock cache) between all threads in an application. This design decision impacts the code cache manager because synchronization is required before code cache management can occur (to guarantee that threads aren't currently executing code that is being deleted from the code cache).

Finally, Merten et al. [77] propose hardware extensions to aid a dynamic optimization system. A hardware-based *hot-spot detector* locates the frequently-executed program regions by monitoring the retired instruction stream off the critical path. The hot spots are optimized and stored in a *frame cache* in main memory. Frames are long, atomic code regions where highly biased branches are replaced by assertions. Extensions to the branch predictor hardware allow execution to migrate to newly-optimized code frames. While the details of the cache management algorithm in place are not discussed in their technical report, they do mention flushing the code cache to effectively handle self-modifying code.

7.1.2 Security, Sandboxing, and Introspection

DELI [34] is a VLIW version of Dynamo geared toward embedded-processor applications that was developed by Hewlett-Packard in conjunction with ST Microelectronics. The system is designed to be an open interface between the hardware and software layers by providing an API for creating and optimizing a private copy of executing code. The cache management policy implemented in DELI is a user-controlled full cache flush mechanism.

DynamoRIO [14, 15, 36] is a dynamic optimization research infrastructure devel-

oped as a collaboration between Hewlett-Packard and MIT. DynamoRIO executes on the IA-32 architecture in both Windows and Linux. The cache replacement policy in the publicly available version of DynamoRIO defaults to an unbounded code cache. Using an environment variable, a user may impose a cache size limit, and in this case, DynamoRIO employs a fine-grained FIFO replacement algorithm, with the code cache being implemented as a circular buffer similar to that proposed in our earlier study [49] and discussed in Chapter 2.

7.2 Dynamic Binary Translation

Dynamic binary translation has been used to produce fast simulators [25] and efficient emulators [32]. Runtime information is used to avoid the need for source code and the problems of code discovery inherent in static translation techniques. Dynamic binary translation is an invaluable technique for code compatibility and design space exploration.

Several dynamic binary translation systems have been developed over the years to provide code compatibility for new or existing architectures, and we will look at several of them in this section. The systems are organized into two categories: those that target a single, existing platform and those that are designed to target multiple platforms.

Targeting a Single Platform FX!32 [55], developed by Digital, allows IA-32 binaries to execute on Alpha processors. Although FX!32 does not perform dynamic optimization in a strict sense—it optimizes between program runs—it has many features of a true dynamic optimization system. The University of Queensland's Dynamic Binary Translator (UQDBT) dynamically translates IA-32 code to run on SPARC-based processors [92] using an emulation framework. Hewlett-Packard's Shogun system [68] dynamically translates HP-UX binaries to run on a statically-scheduled IA-64-like architecture. Aries [99], a follow-up system, dynamically translates from PA-RISC to IA-64. The DAISY [38] dynamic binary translator was developed at IBM and allows PowerPC binaries to execute on the proprietary DAISY VLIW architecture. A follow-up project at IBM is the BOA framework [2] which allows PowerPC code to execute on a VLIW/EPIC processor. Kim and Smith describe a dynamic binary translation system that targets accumulator-oriented architectures [62, 63] to take advantage of clock-frequency advantages inherent to such architectures. Since the initial study targeted the SPEC 2000 benchmarks, the system performed no code cache management, allowing the cache to grow without bound. The Transmeta Crusoe processor [32, 66] is shipped with CMS (Code-Morphing Software) which performs binary translation from IA-32 to an underlying proprietary VLIW architecture. The code cache management policy in CMS has not been disclosed. Finally, Witchel and Rosenblum [96] propose a software system that uses binary translation to perform full-system simulation to experiment with processor design decisions for a MIPS R3000. Basic blocks are

stored in a translation cache that is flushed whenever self-modifying code is detected or when customized translations are desired.

Targeting Multiple Platforms Strata [88] and Walkabout [24] are research infrastructures for dynamic binary translation that are specifically designed to be retargetable. Strata has been retargeted to run on SPARC, MIPS, and IA-32 architectures. Walkabout, which was based on UQDBT, has been retargeted to execute on both SPARC and IA-32 architectures. The code cache management policy in both Strata and Walkabout is a full code cache flush when it becomes full.

7.3 High-Level Language Virtual Machines

Many just-in-time compilers have been equipped with adaptive systems that allow code to be re-optimized during execution. For example, Jikes RVM [4] is a publicly-available Java Virtual Machine developed at IBM Research. Jikes RVM continuously monitors the performance of compiled code and regenerates optimized versions of the frequently-executed regions. The generated code is not stored in a contiguous memory region, thus the management policy is dictated by the garbage collector.

Berndl and Hendren [11] propose a new algorithm for maintaining high-coverage in the optimized traces in the trace cache, however, like Jikes RVM, the trace cache does not reside in contiguous memory. Bruening and Duesterwald [13] discuss optimal region shapes for an embedded JIT compiler in order to reduce the size of the code cache.

7.4 Hardware Caches

Code caching is somewhat similar to the problems of instruction- and data caching in microprocessors as both problem domains attempt to maintain the program working set in the cache.

Hardware-based trace caches improve instruction fetch bandwidth by grouping together dynamic instruction sequences in a distinct instruction cache [43, 85]. The Pentium-4 microprocessor contains a trace cache that stores micro-operations, which has the added benefit of reducing decode overhead [54].

Trace caches share some of the instruction cache performance benefits of software-based superblock caches. However, trace caches are ill-suited for dynamic optimization because traces are short (roughly eight micro-operations in the Pentium-4) and the traces in the cache have short lifetimes. Management of the trace cache is simplified by the fixed-length, unoptimized nature of the traces. Performance limits of trace caches have been reported by comparing against theoretical limits [81].

Replay [39] is a hardware-based dynamic optimization framework that optimizes single-entry single-exit regions called *frames* that are made up of several basic blocks.

Two designs were proposed for storing the large frame regions in a hardware-based trace cache [30]. One method partitioned the frame into cache-line sized segments and stored each segment in a random cache line, while providing pointers at the ends of each segment to the next segment. Whenever one frame replaces another, it overwrites the identical trace cache lines as the evicted block, thereby maintaining the same next-segment pointers. The second design split the cache into two regions. The first region stored only the first portion (called the *head block*) of the frame, while the second region stored the body of the frame. The cache region storing the frame body has an associated free list. To replace a frame in the cache, you simply overwrite the head block, free the stale body, then store the new body in any free location in the body cache.

Others have performed filtering studies in the context of hardware-based caches [82, 84]. In fact, filter caches [86] greatly resemble the nursery cache in our generational cache architecture.

7.5 Related Areas

Cache management appears in several other research areas, including web caching and memory management. Virtual memory is managed as fixed-sized pages [40, 58, 59], which is a much different problem than code caches that store superblocks of varying sizes. In fact, virtual memory can be considered a degenerative form of code cache management, as it represents the case of an unbounded code cache.

Web caches, on the other hand, store varying-sized elements [6, 12, 18, 94]. Yet, web caches have the added benefit of a backing store. Furthermore, they can be managed during reduced internet activity, relieving the strain on the cache management policy.

.

Chapter 8

Conclusions

Large, interactive applications impose limiting constraints on a code cache management system. The salient features of such applications complicate management heuristics and strain the system due to the frequency and number of superblocks generated at run time. These complications warranted the design of new, sophisticated code cache management policies.

Code cache management policies have been primarily limited to two extreme granularities. To make room for new superblocks in the cache, they evicted either the minimum necessary to make room for the new superblocks, or they flushed the entire cache. Using trace-driven simulation driven by the cache accesses and overheads of an existing dynamic optimizer, we have explored the intermediate granularities of code cache evictions.

Through a combination of simulation and analytical means, we investigated the factors that make up the overhead of code cache management. We determined that the major contributor to cache eviction overhead is a fixed cost to invoke the eviction mechanism, which indicated that it is advantageous to evict larger regions from the code cache at a time. We also determined that implementation details of superblock chaining make a significant impact on the memory and run-time overhead of the implemented eviction policy. In the end, we learned that medium-grained code cache evictions are the most scalable under pressure, ultimately resulting in a balance between cache miss rates, cache management complexity, and run-time overhead.

Generational code caches were also presented as a means for solving the problem of global code cache management in dynamic optimization systems. The motivation was an analysis of the lifetime of code traces residing in the code cache for various applications, and the observation that the majority of code traces were either very short- or very long-lived. Replacing a single cache with multiple generational caches can decrease the miss rates and resulting overhead of nearly all benchmarks, large or small, often significantly.

An implementation of generational code cache management in DynamoRIO allowed direct measurement of the performance advantage of generational code caching

over the standard circular FIFO approach found in DynamoRIO. It furthermore enabled an analysis of the overhead of supporting generational code caches and a characterization of the impact of these insights on other code cache management schemes.

Our breakdown of the run-time performance overhead of generational code caches in a dynamic optimization system presents several major contributions to the dynamic optimization community. The analysis of the run-time overhead provides insights to designers of code cache management algorithms, both in terms of improving future trace-driven code cache simulators, and for fine tuning existing implementations.

In general, as dynamic systems become more prevalent, an increasing need will arise for robust and versatile dynamic systems that support larger and larger applications. This need will drive researchers to search for more sophisticated and effective code cache management schemes, and this dissertation provides the first steps in that direction.

8.1 Broader Impact

While all of the experiments performed for this dissertation were done in the context of one particular dynamic optimization system, the impact of the findings present major contributions to any dynamic optimization system that features a software-based code cache, including most dynamic binary translation and adaptive optimization systems.

The base system, DynamoRIO, was developed during 2001–2003, benefiting from roughly seven years of research on its predecessor system, Dynamo. Thus, DynamoRIO's internal design is a bit of a conglomeration Dynamo and of the many other systems of the time. While the specific tuning of DynamoRIO will undoubtedly differ from other systems, the high level design is very similar to the others. Therefore, the local and global code cache management policies presented in this dissertation can be directly implemented in most of these other systems.

We expect the FIFO replacement policy to perform well regardless of the dynamic optimization infrastructure or the input application. Furthermore, we expect added benefits from choosing to evict larger-than-necessary code regions. The particular size of those cache regions will, of course, vary by input program and the overall code cache size. For larger applications, generational code caches will provide an added benefit by maintaining long-lived code. The best configuration of generational caches will vary by application as we found in Chapter 6, but for Windows applications the lifetime of the code leads us to suggest a balanced implementation.

The scalable code cache management policies presented in this dissertation also bring us one step closer to the possibility of incorporating dynamic optimization systems into the mainstream of hardware-software co-design.

8.2 Future Work

This dissertation enables a great deal of follow-up research in code caching, and this section provides an overview of a few possible avenues for future work that are either directly related to or enabled by this dissertation. We will begin by thinking about techniques for managing the code cache from the front end. This includes both managing cache insertions to relieve the pressure on the code cache management system and making more informed decisions about layout in the code cache during superblock insertion. We will then discuss opportunities for improvement if we could add hardware or operating system support for dynamic optimization systems. Next, we'll discuss opportunities for persistent caching, which will be continued in Appendix A. Finally, we'll investigate some of the technologies that are enabled by efficient, scalable dynamic optimization systems.

8.2.1 Code Cache Insertion Policies

Now that effective code cache replacement policies have been identified, it becomes important to look at the policies for inserting code into the code cache. There are several open questions in this regard.

Insertion Rates The threshold for determining that a code region is *hot* is fixed at 50 executions for both Windows and Linux applications in DynamoRIO. It would be interesting to investigate this issue to determine if motivation exists for setting unique thresholds for Windows versus Linux applications, dynamically-loaded libraries versus static code, or even side-exits versus fall-through exits for existing superblocks.

Superblock Placement within a Code Cache A related issue concerns the spatial locality of superblocks as they appear in the code cache. The current DynamoRIO thresholds result in placement of superblocks within the code cache that is not conducive for execution to flow directly from one superblock to the next. In fact, a simple analysis of the links within a code cache show that the links resemble a tangled web (see Figure B.3 in Appendix B), therefore execution will branch great distances within the code cache. This effect undoubtedly has negative impacts on the instruction cache performance of the code in the code cache. Investigating more sophisticated layouts of superblocks in the code cache should lead to added performance gains in superblock execution within code caches in dynamic optimization systems.

8.2.2 Hardware and OS Support for Code Caches

An interesting issue to consider is the range of hardware techniques that would benefit code caches, or dynamic optimization systems in general. Kim and Smith [63], for example, looked at hardware support for indirect branches in code caches. Other

researchers have used the existing performance monitor registers in the IPF architecture to locate hot code regions [74]. It would be interesting to determine whether the opportunity exists for communication to flow in the other direction—whether dynamic optimization systems can provide information to aid in hardware decisions. For instance, the information acquired by the software-based code cache could potentially aid in hardware-based instruction cache replacement decisions.

Code caches could also benefit from operating system support. One interesting idea is to provide support for memory-protection changes necessary to execute cached code and to detect self-modifying code, which is currently very expensive to a dynamic optimization system.

8.2.3 Persistent Storage of Code Cache Contents

Techniques for storing optimized or translated code between program executions could be an interesting research area. The trade-offs involved in storing optimization hints versus previously-optimized code between executions are not clear at this point, although in Appendix A shows that there is certainly a potential opportunity for storing information between executions.

8.2.4 Enabling Other Technologies

Finally, with reliable, scalable dynamic optimization systems comes the opportunity to look more seriously at other technologies that these systems can benefit. In particular, there appears to be a clear potential for dynamic optimization systems to aid in areas such as power-aware computing and computer security.

Power-Aware Computing Dynamically tailoring an application to a low-power environment seems a natural application of dynamic optimization systems. While some researchers have started to investigate these opportunities [1, 83], there are still a large number of unknowns in terms of the potential or the possible techniques for low-power dynamic optimization. From optimizations that specifically target low-power to the effects of the dynamic optimization system itself on the power profile, there is a world of research ideas waiting to be explored.

Security The transparent and introspective nature of many dynamic optimization systems means that security features can be added or validated on the fly. Some researchers have already begun investigating this potential [64].

Appendix A

Persistent Caching

A natural extension to generational code caches is the notion of storing the most persistent code (residing in the persistent cache) beyond the end of the program execution, and allowing the cost of transformation to be amortized not only over an entire program execution, but over several executions.

As previously discussed, most modern dynamic optimization systems cache the code sequences generated during a program's execution for that execution. We refer to this approach as one that exploits *intra-execution persistence*. This approach works well when applications exhibit a sizable amount of code reuse and a large amount of reuse for individual sequences. Long-running programs often exhibit such characteristics, but this is not typically the case for short-running programs. Yet, the performance of short-running programs can be important if most of the application workload comprises such applications or if long-running scripts repeatedly invoke short-running programs.

To address the problem of short-running applications, a dynamic binary translator or optimizer will need to amortize its costs over multiple executions of the short-running application, an approach we refer to as one that exploits *inter-execution persistence*. Though some binary translation systems [55] reuse translations from earlier program runs, current dynamic optimization infrastructures do not support inter-execution persistence of optimized code sequences. We refer to persistence of optimized code sequences as *optimization persistence*. Understanding optimization persistence is an important step in building such functionality into dynamic optimization systems.

In this work, we find that there is a sizable amount of inter-execution optimization persistence in SPEC 2000 under the DynamoRIO dynamic optimization system. By understanding the type of inter-execution optimization persistence that exists, we can, in the future, consider techniques that reduce the overhead of dynamic optimization by replacing the cost of start-up profiling and initial optimization generation with the cost of priming the dynamic optimizer's code cache with persistent optimizations.

Once we are focused on code reuse beyond the boundaries of a program's exe-

cution, we can consider using optimizations and translations from past application executions for new programs that have never previously executed. Interestingly, we show that a non-trivial amount of optimized code persists across program boundaries. That is, identical superblocks appear in the DynamoRIO code cache despite the fact that different programs are executing. This result points out the potential benefit of inter-application persistence.

We can explain inter-application optimization persistence by realizing that the DynamoRIO optimizes across module boundaries. Therefore, by priming the code cache with optimized superblocks from commonly executed shared libraries, we may achieve overhead reductions even during the first execution of a new application.

In this work, we focus on systems for dynamic optimization, because the least is known about the beneficial effects of inter-execution and inter-application optimization persistence on these types of systems. We analyze the potential overhead reduction possible in the DynamoRIO dynamic optimization system when dynamically optimized portions of code are maintained after program termination for use in successive executions. This is accomplished by analyzing the amount of potential persistence that occurs in a dynamic optimizer, first for successive executions of the same program (with changing inputs), then for different programs.

Just as poor profile data can adversely affect the performance of an executable produced by a static compiler, low correlation between successive runs of an executable could result in poor dynamic optimization performance if persistent optimization state is maintained.

Therefore, before it will be possible for researchers to propose or design an optimization caching technique that will store optimizations between program executions, it is first worthwhile to clearly understand how much opportunity exists. We did so by measuring exactly how much work was duplicated during successive program executions. In this case, *program executions* can be defined as either successive executions of the *same* program or successive executions of *different* programs. We refer to the former case as inter-execution persistence, and the latter case as inter-application persistence. We investigated both opportunities. Our results indicate that there is a significant opportunity for leveraging inter-execution persistence, and there is even a small opportunity for inter-application persistence for small programs.

A.1 Inter-Execution Optimization Persistence

Our first study looked into the correlation of frequently executed instruction sequences from one execution of a single program to the next with changing program inputs. We performed this study by running the SPEC 2000 benchmarks to completion under the control of the DynamoRIO dynamic optimization system.

By setting an environment variable, we were able to request that DynamoRIO log all of the hot superblocks after program completion. Superblocks chosen for optimiza-

tion were compared with one another as we varied the input set. This comparison was performed offline after the benchmarks had completed execution.

The SPEC 2000 benchmarks have three official input sets per benchmark – test, train, and reference. We used these three input sets to study the effect of changing input sets on dynamic optimization decisions. For these experiments, we performed comparisons of all permutations of the input sets, then calculated the overlapping percentage using Equation A.1.

Comparison
`test` vs. `train`
`train` vs. `test`
`test` vs. `reference`
`reference` vs. `test`
`train` vs. `reference`
`reference` vs. `train`

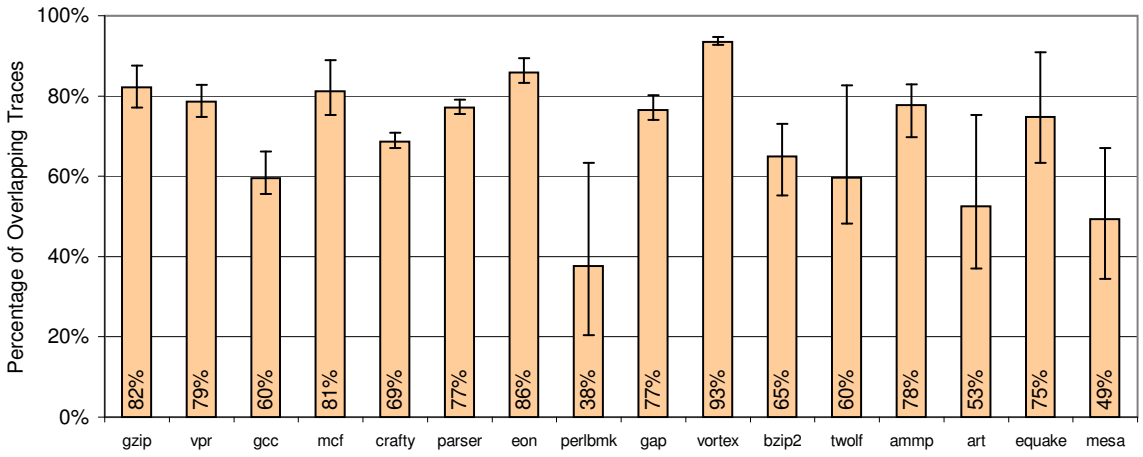
Our `A` vs. `B` notation represents the following calculation:

$$\frac{|hotTraces_A \cap hotTraces_B|}{|hotTraces_A|} \tag{A.1}$$

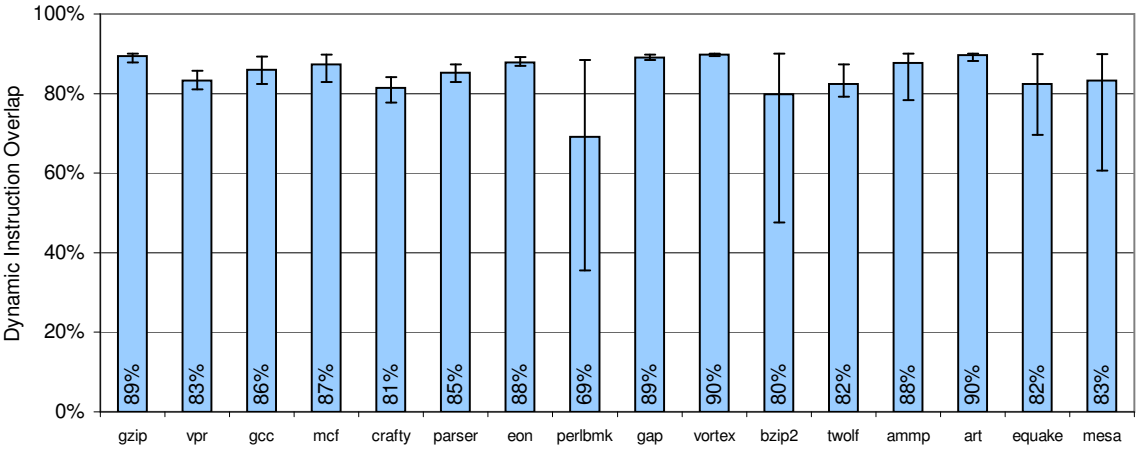
We then averaged the results of the six permutations to form the results shown in Figure A.1. The average values are shown in the wide graph bars, while the ranges are depicted in the error bars. From Figure A.1(a), we can make the overall observation that on average 70% of the optimized code produced by DynamoRIO is regenerated every execution, regardless of program inputs. In the best case, `vortex` resulted in 93% inter-execution persistence, while in the worst case, `perlbmk` showed only 38%. As `perlbmk` is an interpreter program, its dynamic behavior is highly dependent on the program input, therefore the low persistence and high variance is not surprising. Additionaly, `art` and `mesa` resulted in less-than-average persistence; however, in their case, this can be attributed to the fact that they are small programs (producing only 269 and 311 superblocks, respectively) and therefore small changes in their code resulted in a higher percentage of superblock changes.

Figure A.1(b) shows the percentage of dynamic instructions that can be attributed to the persistent code. Interestingly, a consistently high percentage of time is spent in the code that persists from one run to the next. Even in the case of `perlbmk`, where only 38% of the superblocks were regenerated during successive executions, that 38% accounted for 69% of the dynamic instruction count. Similarly, for the small benchmarks `art` and `mesa`, their less-than-average superblock persistence ended up accounting for 90% and 83% of the dynamic instruction count, respectively.

While the original hope was to see a high inter-execution optimization persistence to reduce the overhead of short-running programs, we actually found high persistence regardless of program size or execution duration.



(a) Percentage of Common Superblocks



(b) Percentage of Overlapping Dynamic Instructions

Figure A.1: This figure depicts the inter-execution persistence occurring between the different input sets of the SPEC 2000 benchmarks. On the top, we show the percentage of superblocks that reappear in the code cache during successive executions, regardless of the changing input set. On the bottom, we show the percentage of dynamic instruction count spent in the superblocks that reappear during successive executions.

A.2 Inter-Application Persistence

Our next set of experiments attempted to quantify the amount of persistence that occurs between different programs. Using a technique similar to that of Section A.1, we ran each of the SPEC 2000 benchmarks using the reference input set under the control of DynamoRIO. Again, we instructed DynamoRIO to output a log of all optimized superblocks. We compared the log files of every permutation of the benchmarks to determine the overlap that occurred between different programs.

Intuitively, one might assume that there is no common code between any of the SPEC 2000 benchmarks. Encouraged by the work of Calder et al. [17] who showed that shared libraries exhibit very predictable branch behavior from one execution to the next, we were interested in determining if similar conclusions could be made in the optimization domain. Our results show that up to 32% of optimized superblocks generated by DynamoRIO are also produced while executing a different benchmark. Since the superblocks generated by DynamoRIO can span procedure boundaries, the dynamic optimizer has the ability to optimize code contained inside system libraries. Therefore, optimized code from the standard C libraries (such as the `printf` routine) will often be duplicated in many different applications.

Figure A.2 shows the number of common superblocks between each of the SPEC 2000 benchmarks using the reference inputs. The top half (Figure A.2(a)) shows the number of overlapping superblocks, and the bottom half (Figure A.2(b)) shows the resulting dynamic instruction percentage that is spent executing the overlapping code. From the figure, we see that there is up to 32% superblock commonality between different programs. We see that while the superblocks generated for `gzip` account for 27.3% of the superblocks executed in `art`, only 0.0008% of `gcc`'s execution can be attributed to superblocks also appearing in `gzip`.

If we categorize each benchmark according to the number of superblocks generated during execution, we begin to see an interesting trend. Figure A.3 groups the benchmarks into five categories: very small (0-400 superblocks), small (401-800 superblocks), medium (801-2000 superblocks), large (2001-4000 superblocks) and very large (4001+ superblocks.) It then shows the overlapping dynamic instruction count experienced by each of the benchmarks falling into the respective buckets. As program size increases (as measured by the number of superblocks produced during execution), the opportunity for inter-application persistence diminishes. While programs in the *very small* category experienced up to 27% persistence, those in the *very large* category never experience more than 3% inter-application persistence. Therefore, we can conclude that while opportunities do exist for leveraging inter-application persistence, the benefits are skewed toward smaller programs.

	gzip	vpr	gcc	mcf	crafty	parser	eon	perlbmk	gap	vortex	bzip2	twolf	ammp	art	equake	mesa
gzip	440															
vpr	12	1184														
gcc	15	135	18450													
mcf	11	77	53	240												
crafty	23	128	97	46	2027											
parser	26	134	146	55	128	2883										
eon	11	14	21	12	13	13	861									
perlbmk	11	188	214	51	128	122	21	4345								
gap	16	38	61	28	31	68	17	61	2692							
vortex	13	136	169	58	85	94	15	159	45	2198						
bzip2	22	13	20	16	19	20	8	12	16	10	442					
twolf	14	229	162	81	131	111	16	188	37	135	13	1573				
ammp	12	61	58	33	33	52	12	125	26	47	8	60	414			
art	18	112	54	45	79	48	12	97	16	49	15	121	43	269		
equake	13	140	84	71	90	69	11	146	21	62	14	144	91	99	400	
mesa	11	79	67	67	48	54	13	56	21	66	9	89	29	41	92	311

(a) Count of Common Superblocks

	gzip	vpr	gcc	mcf	crafty	parser	eon	perlbmk	gap	vortex	bzip2	twolf	ammp	art	equake	mesa
gzip	100%	2.51%	0.00%	0.02%	3.05%	0.75%	0.00%	0.00%	0.09%	0.01%	0.39%	0.02%	0.00%	27.30%	0.03%	0.00%
vpr	3.64%	100%	0.40%	0.57%	0.38%	1.61%	0.00%	1.82%	0.02%	0.84%	0.20%	0.83%	0.04%	0.01%	1.36%	0.95%
gcc	0.29%	0.34%	100%	1.41%	0.75%	6.76%	0.56%	2.93%	0.75%	3.08%	1.42%	0.57%	0.27%	0.00%	15.01%	1.54%
mcf	0.00%	0.26%	0.30%	100%	0.02%	0.02%	0.00%	0.08%	0.01%	0.01%	2.03%	2.19%	0.02%	4.20%	1.25%	0.84%
crafty	1.81%	3.38%	0.25%	0.03%	100%	1.14%	0.00%	0.14%	0.50%	0.10%	6.31%	9.98%	0.01%	4.17%	0.07%	0.93%
parser	5.21%	0.49%	0.56%	0.26%	5.44%	100%	0.00%	1.31%	0.16%	0.18%	3.25%	1.38%	14.77%	0.01%	0.54%	0.88%
eon	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	100%	0.01%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
perlbmk	0.00%	4.44%	1.08%	0.15%	0.20%	0.96%	0.29%	100%	0.21%	2.35%	0.45%	2.39%	0.45%	0.01%	0.46%	1.16%
gap	0.04%	0.03%	0.02%	0.27%	1.02%	1.02%	0.14%	1.41%	100%	0.36%	0.34%	1.41%	0.02%	0.00%	0.14%	0.00%
vortex	0.02%	0.34%	1.02%	0.49%	0.28%	0.70%	0.68%	1.43%	0.16%	100%	0.19%	0.13%	3.23%	0.00%	1.00%	1.33%
bzip2	1.37%	0.00%	0.00%	0.96%	1.34%	0.10%	0.00%	0.01%	0.04%	0.00%	100%	0.55%	0.00%	11.17%	0.04%	0.01%
twolf	4.35%	2.44%	0.42%	0.60%	0.56%	0.77%	0.02%	1.92%	0.14%	0.98%	0.52%	100%	0.03%	9.13%	4.69%	0.96%
ammp	0.00%	0.09%	0.33%	0.37%	0.00%	1.21%	0.00%	1.24%	0.01%	0.11%	0.00%	0.04%	100%	0.08%	0.45%	0.16%
art	7.24%	0.26%	0.19%	3.68%	0.09%	0.02%	0.00%	0.36%	0.00%	0.03%	0.37%	0.03%	0.01%	100%	6.91%	0.88%
equake	0.09%	0.27%	0.41%	0.50%	0.14%	0.03%	0.00%	0.47%	0.00%	0.03%	0.87%	0.10%	0.20%	9.18%	100%	0.93%
mesa	0.00%	0.27%	0.91%	0.57%	0.07%	0.01%	0.00%	0.49%	0.00%	0.89%	0.06%	0.12%	0.02%	0.00%	1.50%	100%

(b) Percentage of Overlapping Dynamic Instructions

Figure A.2: These figures depict the inter-application persistence occurring between the different benchmarks in SPEC 2000. On the top, we show the number of superblocks that appear in code caches of both benchmark programs. On the bottom, we show the percentage of dynamic instructions corresponding to the overlapping superblocks. Each column lists the percentage of dynamic instructions spent executing code that appeared in each row.

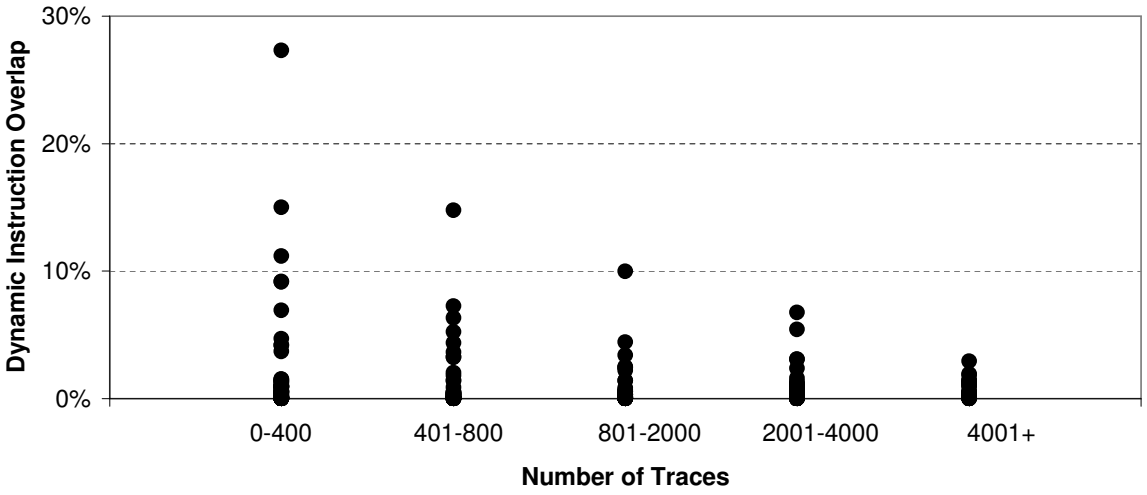


Figure A.3: In this scatter plot, benchmarks are bucketed based on the number of superblocks produced during their execution, then their overlapping dynamic instructions are shown as graph points.

A.3 Discussion

Analysis of the amount of optimization persistence that occurs can improve dynamic optimization systems in several ways. Whether the end goal is one of optimization or compatibility, it is certainly useful to recognize whether or not earlier efforts can be reused to reduce future overhead. In this work, we analyzed the number of optimizations that were repeated during successive program executions, both for the same and for different programs. We found that regardless of changing inputs, on average 70% of the optimizations performed during one execution are repeated during successive executions, and those optimized superblocks account for roughly 85% of the dynamic instruction count. Furthermore, we found that up to 27% of a short-running program's dynamic instructions were identical to that of a different program.

Appendix B

Experimental Approach

All of the experiments throughout this dissertation were performed in the context of the DynamoRIO research infrastructure from Hewlett-Packard and MIT. This chapter provides further details on the simulation and execution environments used to gather all results. We will start out with a description of the code cache simulator and the benchmarks used. Next, we will outline our approach for measuring the overhead of various dynamic optimization tasks using the PAPI performance monitors. Finally, we will provide raw data for many of the graphs shown in the earlier chapters.

Simulations The simulations executed for Chapters 3–5 were all trace driven using the process shown in Figure B.1. As shown in the figure, the benchmarks described in Sections B.1 and B.2 were executed under the complete control of DynamoRIO version 0.9.3. Environment variables were set to indicate that DynamoRIO was to output a trace of relevant events, including cache accesses, insertions, deletions, and details of every superblock encountered. These cache traces were stored and reused for all simulations. The simulations were then performed using my own trace-driven code cache simulator, written in `C`. Implementation of the code cache simulators was straightforward, generally consisting of less than 1,500 lines of C code. Simulated miss rates were validated in Section 6.5.

Implementation The implementation in Chapter 6 was performed directly in the source code of DynamoRIO version 0.9.3. This version consists of 47 source files and 39 header files comprising roughly 63,000 lines of code (including comments.) The



Figure B.1: Trace-driven simulation framework used in Chapters 3–5.

Name	Superblocks	Description
gzip	301	Compression
vpr	449	FPGA Place+Route
gcc	8751	C Compiler
mcf	158	Combinatorial Optimization
crafty	1488	Chess Game
parser	2418	Word Processing
eon	448	Computer Visualization
perlbmk	2144	PERL Language
gap	667	Group Theory Interpreter
vortex	1985	Object-Oriented Database
bzip2	224	Compression
twolf	574	Place+Route

Table B.1: Benchmarks used in our evaluation. The middle column lists the number of hot superblocks that must be managed in the code cache.

implementation included adjustments to 11 of the source and 6 of the header files.

B.1 SPEC 2000 Benchmarks

The official SPEC CPU 2000 benchmarks were used in many of the experiments in this dissertation. Chapters 3, 4, and 6 used the integer benchmarks, while Chapter 5 also incorporated the floating-point benchmark suite.

Unless otherwise indicated, all inputs to the benchmarks were the official reference inputs, and all benchmarks were executed to completion. The SPEC 2000 integer benchmarks are described in Table B.1. The table also lists the number of superblocks generated during their execution.

B.2 Interactive Benchmarks

Interactive applications typically comprise several threads of execution—a *main thread* and several *helper threads*. DynamoRIO maintains a separate code cache for each thread because an earlier study [14] found that the amount of sharing between threads was too small to warrant the synchronization overhead of a shared code cache. Since the bulk of execution actually occurs in the main thread we therefore gathered all results with respect to the main thread of execution.

The interactive benchmarks used in this dissertation and their inputs are listed in Table B.2. Unfortunately, Windows benchmarking has not matured enough to provide for repeatable, standardized benchmark scripts. Therefore, the user interaction in

Name	Secs	Description	Inputs
access	202	Database App	Northwind Sample Database (2.3 MB)
acroread	376	PDF Viewer	Kim Hazelwood's MS Thesis (415 KB)
defrag	46	System Util	4 GB Hard Drive
excel	208	Spreadsheet App	Performance Data (218 KB)
iexplore	247	Web Browser	www.harvard.edu + 5 subpages
mpeg	257	Media Player	Fatboy Slim Video (17 MB)
outlook	196	E-Mail App	Set-up Account; Sent message
pinball	372	3D Game	Game Demo
powerpt	173	Presentation	MS Thesis Presentation (348 KB)
solitaire	335	Game	Manual five minute game
winzip	92	Compression	Compressed Documents Folder (21 MB)
word	212	Word Processor	MS Thesis (1.5 MB)

Table B.2: Interactive Windows benchmarks used during evaluations.

the Windows applications was manual, performing everyday tasks for the duration of time listed in Table B.2. For the chapters that include Windows application results (Chapters 4 and 5) the Windows applications were executed once, and the verbose output from DynamoRIO was saved and reused for all subsequent experiments.

B.3 Using the PAPI Interface

We used the PAPI performance counter interface [72] to the Pentium hardware performance monitors to study the overhead of various activities in the DynamoRIO framework. The relevant PAPI calls were:

- `PAPI_add_event()` for adding a listener to track either the `PAPI_TOT_INS` (total instructions executed) or the `PAPI_TOT_CYC` (total cycles executed).
- `PAPI_start()` to start the listener.
- `PAPI_stop()` to stop the listener and print the instructions/cycles executed.

We collected a log of over 30,000 occurrences of each type of event (cache evictions, misses, etc.), including the superblock size (in bytes) and the number of instructions required to perform the action. We then used a least-squares linear regression trend-line to produce the overhead estimates used throughout the dissertation. An example of this process is illustrated in Figure B.2, which shows our approach for determining the overhead of evicting code from the code cache (used in Chapters 4 and 5). This process was repeated to determine the overhead of code cache misses, link repair, and context switches.

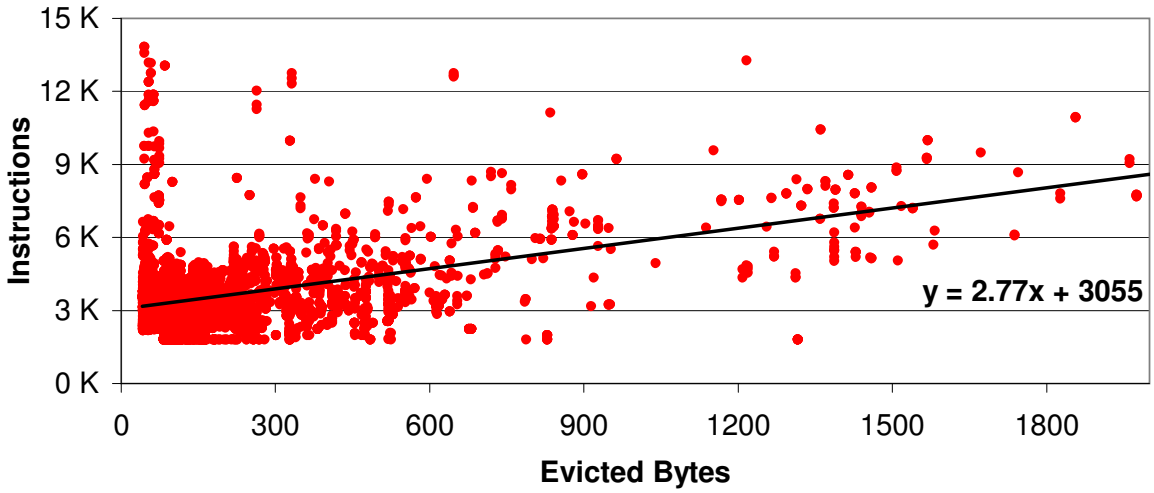


Figure B.2: Overhead, measured in instruction count, for code cache evictions. Each point in this figure represents one overhead reading from the Pentium performance monitors.

In Chapter 6, the Pentium performance monitors were again used to measure the overhead of our actual implementation of generational code caches, and the results were reported in Table 6.1. Therefore, the PAPI interface was not only useful for estimating overhead, but measuring the final overhead as well.

B.4 Raw Data

Many of the figures shown throughout the text of this dissertation were averages shown across a benchmark family. We chose to summarize our results throughout the text for both brevity and clarity. This section is intended as a supplement to the text in that it should answer any questions about the variance of individual benchmarks in a benchmark family for the figures shown throughout.

Chapter 3 Data The next two tables are intended to be a supplement to Chapter 3 where we investigated traditional replacement policies in the context of code caches. Table B.3 shows the raw data for Figure 3.2, which reported the miss rates of various traditional replacement policies under low cache pressure (factor=2). Furthermore, Table B.4 shows the raw data for Figure 3.3, which reported the miss rates under high cache pressure (factor=10).

Chapter 4 Data To provide more of an intuitive sense of chaining within a code cache, we show a visualization of actual superblocks and links in Figure B.3. As the figure shows, superblock links resemble a tangled web with very little spatial locality.

	LRU	**FIFO**	**LFU**	**BFE**	**Flush**	**Random**	**Optimal**
gzip	0.00%	0.00%	2.33%	0.00%	0.00%	0.00%	0.00%
vpr	0.00%	0.00%	7.85%	0.01%	0.01%	0.00%	0.00%
gcc	0.05%	0.08%	4.51%	2.34%	0.16%	0.09%	0.04%
crafty	0.00%	0.00%	0.31%	3.98%	0.01%	0.00%	0.00%
mcf	0.01%	0.01%	0.60%	0.02%	0.02%	0.01%	0.01%
parser	0.00%	0.00%	0.54%	1.49%	0.01%	0.00%	0.00%
eon	0.11%	0.12%	1.16%	0.41%	0.23%	0.12%	0.11%
perlbmk	1.80%	2.07%	10.93%	7.15%	4.13%	2.40%	1.58%
vortex	0.00%	0.00%	1.64%	0.03%	0.01%	0.00%	0.00%
gap	0.01%	0.02%	2.62%	0.30%	0.03%	0.02%	0.01%
bzip2	0.00%	0.00%	1.15%	0.00%	0.00%	0.00%	0.00%
twolf	0.03%	0.04%	0.33%	0.09%	0.07%	0.04%	0.03%
average	0.17%	0.20%	2.83%	1.32%	0.39%	0.23%	0.15%

Table B.3: Miss rates of traditional replacement policies. Cache pressure = 2.

	LRU	**FIFO**	**LFU**	**BFE**	**Flush**	**Random**	**Optimal**
gzip	0.04%	0.04%	1.36%	2.09%	0.30%	0.03%	0.01%
vpr	0.03%	0.04%	39.69%	44.27%	0.32%	0.02%	0.01%
gcc	0.69%	0.74%	24.83%	34.38%	1.86%	0.88%	0.53%
crafty	2.02%	3.19%	32.25%	45.27%	10.65%	3.21%	0.81%
mcf	0.66%	0.67%	3.03%	7.16%	2.66%	0.27%	0.14%
parser	0.31%	0.32%	11.12%	8.81%	1.11%	0.32%	0.18%
eon	11.27%	13.41%	23.59%	18.98%	15.86%	7.85%	4.12%
perlbmk	12.00%	13.39%	42.95%	37.56%	16.96%	13.40%	7.00%
vortex	0.39%	0.74%	26.04%	22.42%	2.02%	0.52%	0.15%
gap	0.18%	0.27%	16.46%	24.18%	1.28%	0.29%	0.09%
bzip2	0.00%	0.00%	4.51%	0.76%	0.09%	0.00%	0.00%
twolf	0.45%	0.45%	17.60%	45.68%	1.10%	0.42%	0.21%
average	2.34%	2.77%	20.29%	24.30%	4.52%	2.27%	1.10%

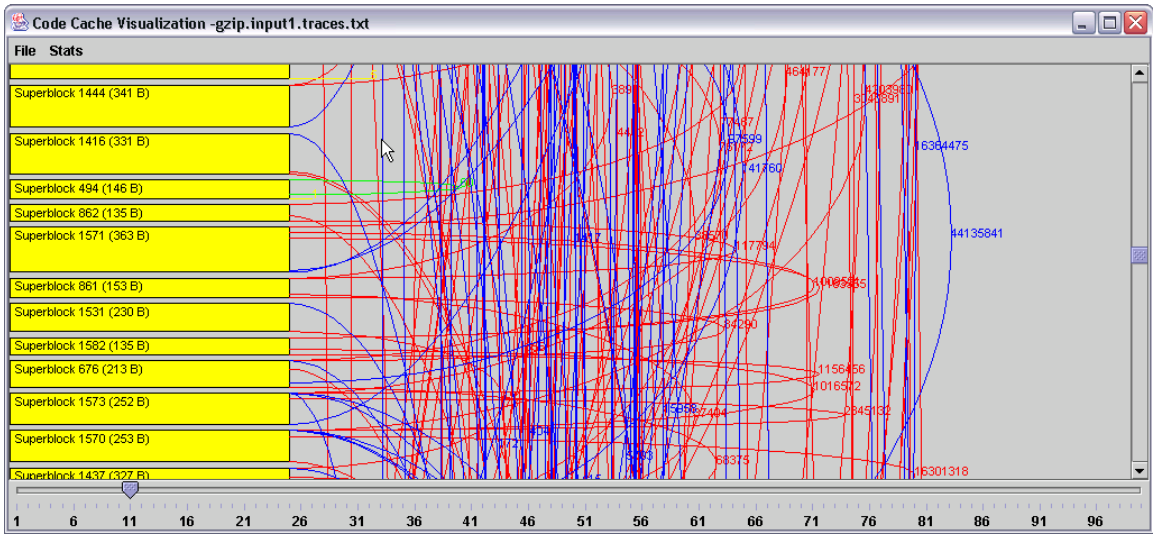
Table B.4: Miss rates of traditional replacement policies. Cache pressure = 10.

Benchmark	FLUSH Miss Rate	2-Unit Miss Rate	FIFO Miss Rate
gzip	0.04%	0.04%	0.03%
vpr	23.30%	18.38%	14.47%
gcc	0.50%	0.39%	0.29%
mcf	3.75%	3.59%	3.56%
crafty	31.43%	27.45%	24.07%
parser	0.30%	0.27%	0.27%
eon	24.74%	22.09%	20.05%
perlbmk	6.78%	5.42%	4.72%
gap	17.51%	14.83%	13.82%
vortex	7.20%	6.30%	5.62%
bzip2	3.78%	3.25%	2.77%
twolf	21.39%	19.87%	17.95%
illustr	0.08%	0.07%	0.06%
iexplore	5.45%	4.46%	3.60%
outlook	6.15%	5.06%	4.28%
photoshop	0.20%	0.17%	0.14%
pinball	2.17%	1.72%	1.51%
powerpt	1.70%	1.40%	1.21%
vstudio	1.39%	1.15%	0.97%
winzip	0.18%	0.14%	0.12%
word	4.46%	3.84%	3.35%

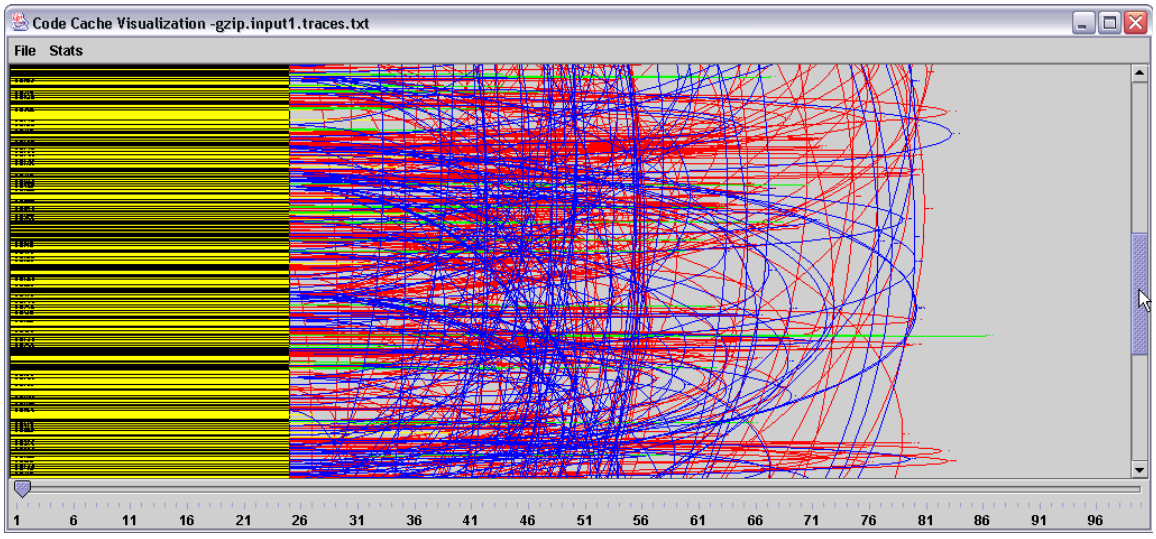
Table B.5: Details of medium-grained evictions from Chapter 4.

Table B.5 shows the raw data for many of the graphs that were shown in Chapter 4. Throughout the chapter these results were shown as averages across all benchmarks in a benchmark family.

Chapter 5 Data When we empirically studied generational code caches in Chapter 5, we assumed that superblock lifetimes were a good predictor of the important superblocks during execution. Figures B.4–B.5 validate this claim by showing the accesses over time of the five oldest and hottest superblocks, respectively.

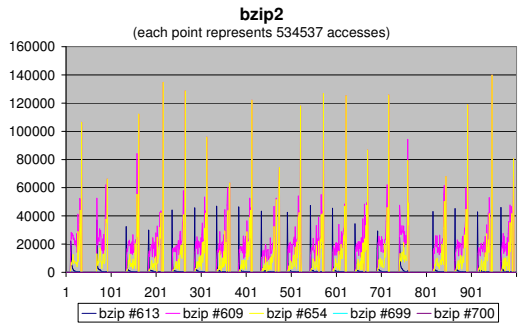


(a) Zoomed View

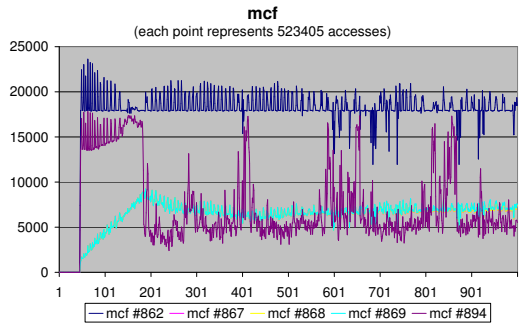


(b) Regular View

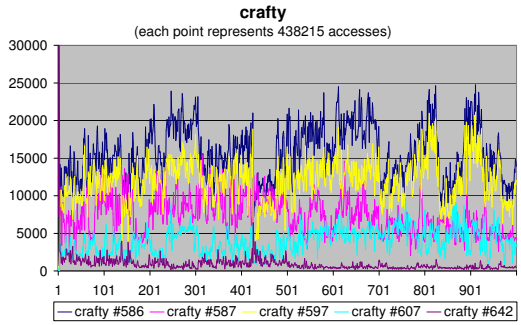
Figure B.3: Interconnectivity of superblocks within a code cache for the gzip benchmark. The yellow boxes represent actual superblocks, with their ID and size noted. Blue links target superblocks appearing above the source superblock. Red links target superblocks below the source. Green links represent self-links (loops). The dynamic execution count of each link shown next to the arc of a link. (This visualization tool was developed by Brian Boysen. Thank you Brian!)



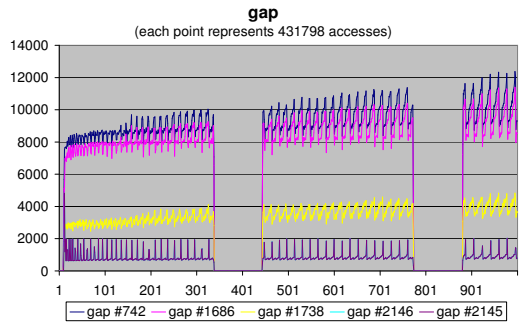
(a) Bzip2



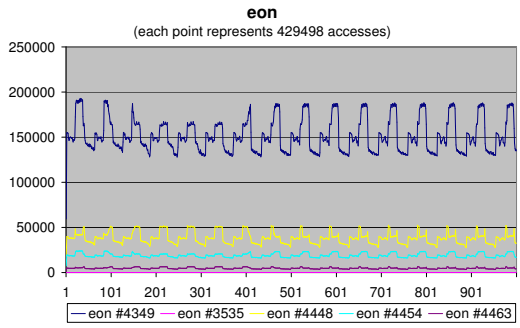
(b) Mcf



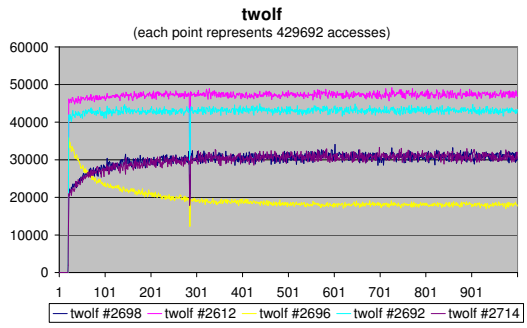
(c) Crafty



(d) Gap

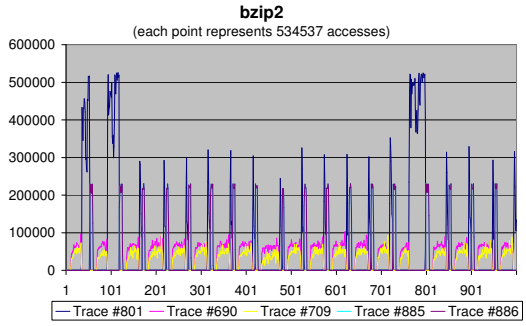


(e) Eon

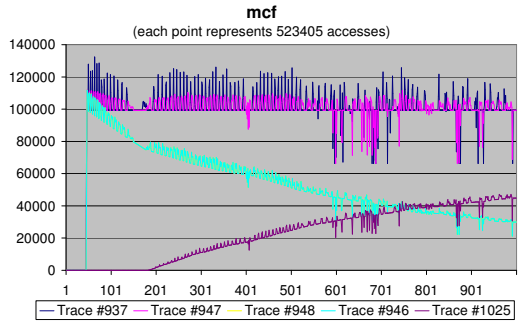


(f) Twolf

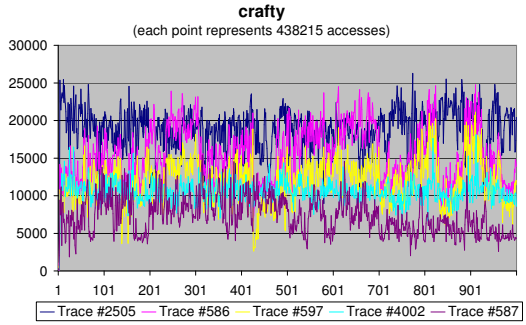
Figure B.4: Accesses over time for the five oldest superblocks per benchmark. The horizontal axis represents time (in increments shown at the top of each figure). The vertical axis shows the dynamic execution count of the five superblocks during each time increment.



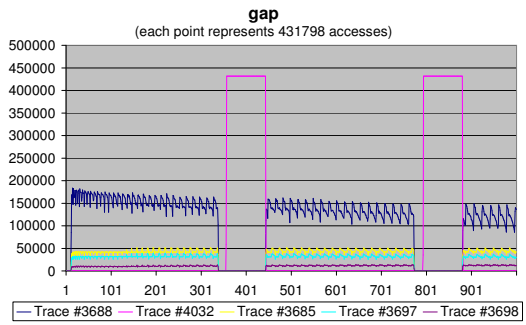
(a) Bzip2



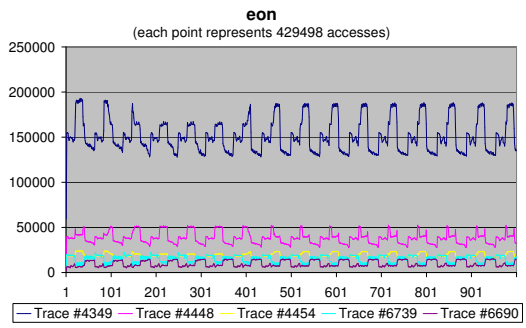
(b) Mcf



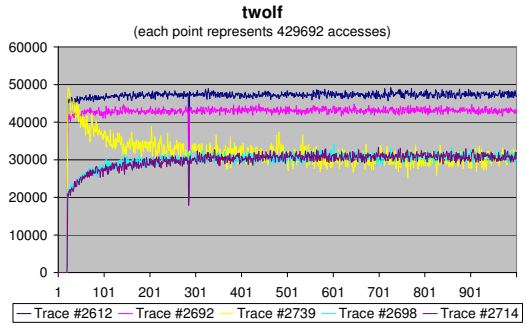
(c) Crafty



(d) Gap



(e) Eon



(f) Twolf

Figure B.5: Accesses over time for the five hottest superblocks per benchmark. The horizontal axis represents time (in increments shown at the top of each figure). The vertical axis shows the dynamic execution count of the five superblocks during each time increment.

Bibliography

[1] Yoav Almog, Roni Rosner, Naftali Schwartz, and Ari Schmorak. Specialized dynamic optimizations for high-performance energy-efficient microarchitecture. In *2nd International Symposium on Code Generation and Optimization*, pages 137–150, March 2004.

[2] Erik R. Altman, Michael Gschwind, Sumedh Sathaye, Stephen Kosonocky, Arthur Bright, Jason Fritz, Paul Ledak, David Appenzeller, Craig Agricola, and Zachary Filan. BOA: The architecture of a binary translation processor. *IBM Research Report RC 21665*, December 2000.

[3] Andrew W. Appel. Simple generational garbage collection and fast allocation. *Software Practice and Experience*, 19(2):171–183, 1989.

[4] Matthew Arnold, Stephen Fink, David Grove, Michael Hind, and Peter F. Sweeney. Adaptive optimization in the Jalapeno JVM. In *15th Conference on Object-Oriented Programming, Systems, Languages, and Applications*, pages 47–65, 2000.

[5] Matthew Arnold and Barbara Ryder. A framework for reducing the cost of instrumented code. In *ACM Conference on Programming Language Design and Implementation*, pages 168–179, 2001.

[6] Omri Bahat and Armand M. Makowski. Optimal replacement policies for non-uniform cache objects with optimal eviction. In *IEEE INFOCOM Conference on Computer Communications*, April 2001.

[7] Vasanth Bala, Evelyn Duesterwald, and Sanjeev Banerjia. Transparent dynamic optimization. Technical Report HPL-1999-77, Hewlett Packard, June 1999.

[8] Vasanth Bala, Evelyn Duesterwald, and Sanjeev Banerjia. Dynamo: A transparent dynamic optimization system. In *ACM Conference on Programming Language Design and Implementation*, pages 1–12, 2000.

[9] Leonid Baraz, Tevi Devor, Orna Etzion, Shalom Goldenberg, Alex Skaletsky, Yun Wang, and Yigel Zemach. Ia-32 execution layer: a two-phase dynamic

translator designed to support ia-32 applications on itanium-based systems. In *36th International Symposium on Microarchitecture*, pages 191–201, December 2003.

[10] L. A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.

[11] Marc Berndl and Laurie Hendren. Dynamic profiling and trace cache generation. In *First International Symposium on Code Generation and Optimization*, pages 276–288, March 2003.

[12] Lee Breslau, Pei Cao, Li Fan, Graham Phillips, and Scott Shenker. Web caching and zipf-like distributions: Evidence and implications. In *INFOCOM*, pages 126–134, 1999.

[13] Derek Bruening and Evelyn Duesterwald. Exploring optimal compilation unit shapes for an embedded just-in-time compiler. In *3rd ACM Workshop on Feedback-Directed and Dynamic Optimization*, pages 13–20, December 2000.

[14] Derek Bruening, Evelyn Duesterwald, and Saman Amarasinghe. Design and implementation of a dynamic optimization framework for windows. In *4th ACM Workshop on Feedback-Directed and Dynamic Optimization (FDDO-4)*, December 2001.

[15] Derek Bruening, Timothy Garnett, and Saman Amarasinghe. An infrastructure for adaptive dynamic optimization. In *First International Symposium on Code Generation and Optimization*, pages 265–275, March 2003.

[16] Bryan Buck and Jeffrey K. Hollingsworth. An API for runtime code patching. *The International Journal of High Performance Computing Applications*, 14(4):317–329, Winter 2000.

[17] Brad Calder, Dirk Grunwald, and Amitabh Srivastava. The predictability of branches in libraries. In *28th International Symposium on Microarchitecture*, pages 24–34, 1995.

[18] Pei Cao and Sandy Irani. Cost-aware WWW proxy caching algorithms. In *Usenix Symposium on Internet Technologies and Systems*, Monterey, CA, 1997.

[19] Jason P. Casmira, David R. Kaeli, and David P. Hunter. Tracing and characterization of NT-based system workloads. *Digital Technical Journal*, 10(1):6–21, December 1998.

[20] Mark J. Charney and Thomas R. Puzak. Prefetching and memory system behavior of the SPEC95 benchmarks. *IBM Journal of Research and Development*, 41(3), May 1997.

[21] Howard Chen, Wei-Chung Hsu, Jiwei Lu, Pen-Chung Yew, and Dong-Yuan Chen. Dynamic trace selection using performance monitoring hardware sampling. In *First International Symposium on Code Generation and Optimization*, pages 79–90, March 2003.

[22] Wen-Ke Chen, Sorin Lerner, Ronnie Chaiken, and David Gillies. Mojo: A dynamic optimization system. In *4th ACM Workshop on Feedback-Directed and Dynamic Optimization*, pages 81–90, 2000.

[23] Anton Chernoff, Mark Herdeg, Ray Hookway, Chris Reeve, Norman Rubin, Tony Tye, S. Bharadwaj Yadavalli, and John Yates. FX!32: A profile-directed binary translator. *IEEE Micro*, 18(2):56–64, March-April 1998.

[24] C. Cifuentes, B. Lewis, and D. Ung. Walkabout - a retargetable dynamic binary translation framework. Technical Report TR2002-106, Sun Microsystems Laboratories, January 2002.

[25] Bob Cmelik and David Keppel. Shade: A fast instruction-set simulator for execution profiling. *ACM SIGMETRICS Performance Evaluation Review*, 22(1):128–137, May 1994.

[26] Robert Cohn, David Goodwin, P. Geoffrey Lowney, and Norman Rubin. Spike: An optimizer for alpha/nt executables. In *USENIX Windows NT Workshop*, pages 17–24, 1997.

[27] Thomas M. Conte, Sumedh W. Sathaye, and Sanjeev Banerjia. A persistent rescheduled-page cache for low-overhead object code compatibility. In *29th International Symposium on Microarchitecture*, pages 4–13, 1996.

[28] Fernando J. Corbato and Victor A. Vyssotsky. Introduction and overview of the multics system. *AFIPS Fall Joint Computer Conference*, 27(1):185–196, 1965.

[29] Standard Performance Evaluation Corporation. SPEC CPU2000 benchmark suite. http://www.spec.org/cpu2000/, December 1999.

[30] Matthew Martin Crum. Design alternatives for caching long regions of the dynamic instruction stream. Master's thesis, University of Illinois at Urbana-Champaign, 2001.

[31] Dean Deaver, Rick Gorton, and Norm Rubin. Wiggins/redstone: An on-line program specializer. In *IEEE Hot Chips XI*, 1999.

[32] James C. Dehnert, Brian K. Grant, John P. Banning, Richard Johnson, Thomas Kistler, Alexander Klaiber, and Jim Mattson. The transmeta code morphing software: Using speculation, recovery, and adaptive retranslation to address real-life

challenges. In *First International Symposium on Code Generation and Optimization*, pages 15–24, March 2003.

[33] Peter J. Denning. Before memory was virtual. *In the Beginning: Personal Recollections of Software Pioneers*, 1997.

[34] Giuseppe Desoli, Nikolay Mateev, Evelyn Duesterwald, Paolo Faraboschi, and Joseph A. Fisher. Deli: A new run-time control point. In *35th International Symposium on Microarchitecture*, pages 257–268, 2002.

[35] Ashutosh S. Dhodapkar and James E. Smith. Comparing program phase detection techniques. In *Proceedings of the 36th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 217–227, 2003.

[36] Evelyn Duesterwald and Saman Amarasinghe. On the run: Building dynamic program modifiers for optimization, introspection and security. Tutorial at Conference on Programming Language Design and Implementation, June 2002.

[37] Evelyn Duesterwald and Vasanth Bala. Software profiling for hot path prediction: Less is more. In *12th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 202–211, October 2000.

[38] Kemal Ebcioglu and Eric Altman. DAISY: Dynamic compilation for 100% architectural compatibility. In *24th International Symposium on Computer Architecture*, pages 26–37, June 1997.

[39] Brian Fahs, Satarupa Bose, Matthew Crum, Brian Slechta, Francesco Spadini, Tony Tung, Sanjay J. Patel, and Steven S. Lumetta. Performance characterization of a hardware mechanism for dynamic optimization. In *34th International Symposium on Microarchitecture*, pages 16–27, December 2001.

[40] Amos Fiat and Anna R. Karlin. Randomized and multipointer paging with locality of reference. In *27th ACM Symposium on Theory of Computing*, pages 626–634. ACM Press, 1995.

[41] Joseph A. Fisher. Trace scheduling: A technique for global microcode compaction. *IEEE Transactions on Computers*, 30(7):478–490, July 1981.

[42] Joseph A. Fisher. Walk-time techniques: Catalyst for architectural change. *IEEE Computer*, 30(9):40–42, September 1997.

[43] Daniel H. Friendly, Sanjay J. Patel, and Yale N. Patt. Putting the fill unit to work: Dynamic optimizations for trace cache microprocessors. In *31th International Symposium on Microarchitecture*, pages 173–181, 1998.

[44] Brian Grant, Markus Mock, Matthai Philipose, Craig Chambers, and Susan J. Eggers. DyC: an expressive annotation-directed dynamic compiler for C. *Theoretical Computer Science*, 248(1–2):147–199, 2000.

[45] William A. Havanki, Sanjeev Banerjia, and Thomas M. Conte. Treegion scheduling for wide issue processors. In *4th International Conference on High Performance Computer Architecture*, pages 266–276, 1998.

[46] Kim Hazelwood and Thomas M. Conte. A lightweight algorithm for dynamic if-conversion during dynamic optimization. In *9th Conference on Parallel Architectures and Compilation Techniques*, pages 71–80, 2000.

[47] Kim Hazelwood and David Grove. Adaptive online context-sensitive inlining. In *First International Symposium on Code Generation and Optimization*, pages 253–264, March 2003.

[48] Kim Hazelwood and James E. Smith. Exploring code cache eviction granularities in runtime systems. In *2nd International Symposium on Code Generation and Optimization*, pages 89–99, Palo Alto, CA, March 2004.

[49] Kim Hazelwood and Michael D. Smith. Code cache management schemes for dynamic optimizers. In *6th Workshop on Interaction between Compilers and Computer Architectures*, pages 102–110, February 2002.

[50] Kim Hazelwood and Michael D. Smith. Characterizing inter-execution and inter-application optimization persistence. In *First Workshop on Exploring the Trace Space for Dynamic Optimization Techniques*, pages 51–58, San Francisco, CA, June 2003.

[51] Kim Hazelwood and Michael D. Smith. Generational cache management of code traces in dynamic optimization systems. In *36th International Symposium on Microarchitecture*, pages 169–179, San Diego, CA, December 2003.

[52] Kim Hazelwood, Mark C. Toburen, and Thomas M. Conte. A case for exploiting memory-access persistence. In *First Workshop on Memory Performance Issues*, June 2001.

[53] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 3rd edition, 2003.

[54] Glenn Hinton, Dave Sager, Mike Upton, Darrell Boggs, Doug Carmean, Alan Kyker, and Patrice Roussel. The Microarchitecture of the Pentium 4 Processor. *Intel Technology Journal*, 2001.

[55] Raymond J. Hookway and Mark A. Herdeg. Digital FX!32: Combining emulation and binary translation. *Digital Technical Journal*, pages 3–12, February 1997.

[56] Zhigang Hu, Stefanos Kaxiras, and Margaret Martonosi. Timekeeping in the memory system: predicting and optimizing memory behavior. pages 209–220, 2002.

[57] Wen-Mei Hwu, Scott A. Mahlke, William Y. Chen, Pohua P. Chang, Nancy J. Warter, Roger A. Bringmann, Roland G. Ouellette, Richard E. Hank, Tokuzo Kiyohara, Grant E. Haab, John G. Holm, and Daniel M. Lavery. The superblock: an effective technique for VLIW and superscalar compilation. *The Journal of Supercomputing*, 7:229–248, July 1993.

[58] Sandy Irani. Page replacement with multi-size pages and applications to web caching. In *29th ACM Symposium on Theory of Computing*, pages 701–710, May 1997.

[59] Sandy Irani, Anna Karlin, and Steven Phillips. Strongly competitive algorithms for paging with locality of reference. *SIAM Journal on Computing*, 25(3):477–497, June 1996.

[60] Rahul Joshi, Michael D. Bond, and Craig Zilles. Targeted path profiling: Lower overhead path profiling for staged dynamic optimization systems. In *2nd International Symposium on Code Generation and Optimization*, pages 239–250, 2004.

[61] Stefanos Kaxiras, Zhigang Hu, and Margaret Martonosi. Cache decay: exploiting generational behavior to reduce cache leakage power. pages 240–251, 2001.

[62] Ho-Seop Kim and James E. Smith. Dynamic binary translation for accumulator-oriented architectures. In *First International Symposium on Code Generation and Optimization*, pages 25–35, March 2003.

[63] Ho-Seop Kim and James E. Smith. Hardware support for control transfers in code caches. In *36th International Symposium on Microarchitecture*, December 2003.

[64] Vladimir Kiriansky, Derek Bruening, and Saman Amarasinghe. Secure execution via program shepherding. In *11th USENIX Security Symposium*, San Francisco, August 2002.

[65] Thomas Kistler. Dynamic runtime optimization. Technical Report ICS-TR-96-54, University of California, Irvine, Department of Information and Computer Science, November 1996.

[66] Alexander Klaiber. The technology behind crusoe processors. Transmeta Corporation White Paper, January 2000.

[67] Chris Lattner and Vikram Adve. LLVM: A compilation framework for lifelong program analysis and transformation. In *2nd International Symposium on Code Generation and Optimization*, pages 75–86, 2004.

[68] Bich C. Le. An out-of-order execution technique for runtime binary translators. In *8th international conference on Architectural support for programming languages and operating systems*, pages 151–158. ACM Press, 1998.

[69] Dennis C. Lee, Patrick Crowley, Jean-Loup Baer, Thomas E. Anderson, and Brian N. Bershad. Execution characteristics of desktop applications in windows NT. In *25th International Symposium on Computer Architecture*, pages 27–38, 1998.

[70] Mark Leone and R. Kent Dybvig. Dynamo: A staged compiler architecture for dynamic program optimization. Technical Report 490, Indiana University, September 1997.

[71] Jin Lin, Tong Chen, Wei Hsu, Pen-Chung Yew, and Roy Ju. A compiler framework for speculative analysis and optimizations. In *ACM Conference on Programming Language Design and Implementation*, June 2003.

[72] Kevin London, Jack Dongarra, Shirley Moore, Philip Mucci, Keith Seymour, and Thomas Spencer. End-user tools for application performance analysis using hardware counters. In *14th Conference on Parallel and Distributed Computing Systems*, August 2001.

[73] P. Geoffrey Lowney, Stefan M. Freudenberger, Thomas J. Karzes, W. D. Lichtenstein, Robert P. Nix, John S. O'Donnell, and John Ruttenberg. The multiflow trace scheduling compiler. *Journal of Supercomputing*, 7(1-2):51–142, 1993.

[74] Jiwei Lu, Howard Chen, Rao Fu, Wei Hsu, Pen-Chung Yew, and Dong-Yuan Chen. Performance of runtime data cache prefetching in a dynamic optimization system. In *36th International Symposium on Microarchitecture*, pages 180–190, December 2003.

[75] Chi-Keung Luk, Robert Muth, Harish Patil, Robert Cohn, , and P. Geoffrey Lowney. Ispike: A post-link optimizer for the intel itanium architecture. In *2nd International Symposium on Code Generation and Optimization*, pages 15–26, 2004.

[76] Scott A. Mahlke, David C. Lin, William Y. Chen, Richard E. Hank, and Roger A. Bringmann. Effective compiler support for predicated execution using the hyperblock. In *25th International Symposium on Microarchitecture*, pages 45–54, 1992.

[77] Matthew C. Merten, Andrew R. Trick, and Ronald D. Barnes. An architectural framework for runtime optimization. *IEEE Transactions on Computers*, 50(6):567–589, 2001.

[78] Matthew C. Merten, Andrew R. Trick, Christopher N. George, John C. Gyllenhaal, and Wen mei W. Hwu. A hardware-driven profiling scheme for identifying program hot spots to support runtime optimization. In *26th International Symposium on Computer Architecture*, pages 136–147, 1999.

[79] Gordon E. Moore. Cramming more components onto integrated circuits. *Electronics Magazine*, 38(8):114–117, April 1965.

[80] Robert Muth, Saumya K. Debray, Scott A. Watterson, and Koenraad De Bosschere. alto: a link-time optimizer for the compaq alpha. *Software - Practice and Experience*, 31(1):67–101, 2001.

[81] Matt Postiff, Gary Tyson, and Trevor Mudge. Performance limits of trace caches. Technical Report CSE-TR-373-98, University of Michigan, August 1998.

[82] Alex Ramirez, Josep-Lluis Larriba-Pey, and Mateo Valero. Trace cache redundancy: Red & blue traces. In *6th International Conference on High Performance Computer Architecture*, pages 325–340, 2000.

[83] Roni Rosner, Yoav Almog, Naftali Schwartz, and Avi Mendelson. Power awareness through selective dynamically optimized traces. In *31st International Symposium on Computer Architecture*, June 2004.

[84] Roni Rosner, Avi Mendelson, and Ronny Ronen. Filtering techniques to improve trace-cache efficiency. In *10th Conference on Parallel Architectures and Compilation Techniques*, pages 37–48, September 2001.

[85] Eric Rotenberg, Steve Bennett, and James E. Smith. Trace cache: A low latency approach to high bandwidth instruction fetching. In *29th International Symposium on Microarchitecture*, pages 24–35, 1996.

[86] Julio Sahuquillo and Ana Pont. The filter cache: A run-time cache management approach. In *25th Euromicro Conference*, pages 1416–1423, 1999.

[87] Kevin Scott, Naveen Kumar, Siva Velusamy, Bruce Childers, Jack Davidson, and Mary Lou Soffa. Reconfigurable and retargetable software dynamic translation. In *First International Symposium on Code Generation and Optimization*, pages 36–47, March 2003.

[88] Kevin Scott, Naveen Kumar, Siva Velusamy, Bruce Childers, Jack W. Davidson, and Mary Lou Soffa. Retargetable and reconfigurable software dynamic translation. In *First International Symposium on Code Generation and Optimization*, pages 36–47, March 2003.

[89] Michael D. Smith. Dynamic optimization: An online opportunity. Keynote Address at the 9th Conference on Parallel Architectures and Compilation Techniques, October 2000.

[90] Michael D. Smith. Overcoming the challenges to feedback-directed optimization. In *Workshop on Dynamic and Adaptive Compilation and Optimization (Dynamo'00)*, pages 1–11, 2000.

[91] Amitabh Srivastava and David W. Wall. Link-time optimization of address calculation on a 64-bit architecture. In *SIGPLAN Conference on Programming Language Design and Implementation*, pages 49–60, 1994.

[92] David Ung and Cristina Cifuentes. Optimising hot paths in a dynamic binary translator. In *2nd Workshop on Binary Translation*, October 2000.

[93] Dave Ungar. Generation scavenging: a non-disruptive high performance storage reclamation algorithm. *ACM SIGPLAN Notices*, 19(5):157–167, May 1984.

[94] Jia Wang. A survey of web caching schemes for the internet. *ACM Computer Communication Review*, 25(9):36–46, 1999.

[95] Zheng Wang and Norm Rubin. Evaluating the importance of user-specific profiling. In *2nd USENIX Windows NT Symposium*, 1998.

[96] Emmett Witchel and Mendel Rosenblum. Embra: Fast and flexible machine simulation. In *Measurement and Modeling of Computer Systems*, pages 68–79, 1996.

[97] Reginald Clifford Young. *Path-based Compilation*. PhD thesis, Harvard University, 1997.

[98] Catherine Xiaolan Zhang, Zheng Wang, Nicholas C. Gloy, J. Bradley Chen, and Michael D. Smith. System support for automated profiling and optimization. In *Symposium on Operating Systems Principles*, pages 15–26, 1997.

[99] Cindy Zheng and Carol Thompson. PA-RISC to IA-64: Transparent Execution, No Recompilation. *IEEE Computer*, 33(3):47–52, March 2000.